

NetMamba+ — both CSVs, both repositories, one packet workflow

Codex-authored handbook, narration script and every-file appendix

This handbook follows the single 01:00:00 video and its slide deck. The video uses synthetic narration. Automated checks and their limits are recorded separately; no complete human watch-through is claimed.

Both uploaded CSVs feed the native packet adaptation. The selected joint model measured 97.56% CIC and 94.29% UNSW group-weighted balanced accuracy; the paper's flow result was not reproduced by this different task. This lesson explains both repositories, setup, recorded tests, predictions and remaining gaps.

Project repositories: [Reproduction repository](#) · [Client repository](#).

Slide images in the PowerPoint match the video; their narration is available as editable speaker notes. This handbook provides searchable text and a complete repository-file appendix. The original customer presentation remains a separate historical package.

Source SHA-256: c330713ea4838877baf0fc2943112c593fd7322e737a91b721511e0a8f5c7528

Course chapters

00:00:00 — The client problem and the two repositories

00:05:16 — The paper's idea and our packet adaptation

00:10:44 — Read the CICIDS2017 packet CSV

00:15:46 — Add UNSW and protect the experiment's split

00:20:59 — Follow one packet through the model

00:26:06 — Understand how the packet model learns

00:31:08 — Read the code behind training and prediction

00:35:41 — Explain the measured results and their limits

00:40:44 — Show the packet demo and explain a mistake

00:45:22 — Set up, run and test the same packet workflow

00:49:57 — Understand both repositories and their updates

00:54:57 — Hardware roadmap, remaining work and your handoff

1. The client problem and the two repositories

00:00:00 — Follow one packet to a decision

Imagine a client asking what this project actually does. Computers send information in small units called packets. This project takes the recorded content of one packet and gives it scores for two supplied categories: benign and attack. Benign is the category used for ordinary traffic in these examples. Attack is the other training category. The model learns patterns in numerical examples; it does not investigate an incident or know the customer's business. Our job throughout this video is to connect that simple description to the files, the trained model, and the evidence behind its measured behavior.

On screen: Stored packet bytes · Trained numerical model · Benign or attack scores

Evidence: [project-guide](#), [packet-report](#)

00:00:38 — The paper, the implementation, and the evidence

Three things play different roles here. The research paper explains a proposed method, much as a recipe explains how a dish could be made. The repository contains the programs and instructions used to put the method into practice. The evidence records what happened when those programs were run. The two uploaded tables supply the packet examples. They are not instructions, and the paper is not fed into the classifier. When presenting a result to a client, connect the claim to the matching evidence. A diagram explains an idea; a saved measurement supports a claim about an executed experiment.

On screen: Paper: research method · Repository: executable work · Evidence: recorded outcomes

Supplied file — Role in the project

2601.21792v1.pdf — Method to inspect; never model input

Payload_data_CICIDS2017.csv — CIC packet bytes and labels

Payload_data_UNSW.csv — UNSW packet bytes and labels

Evidence: [project-guide](#), [paper-notes](#), [packet-report](#)

00:01:16 — One supported route from both CSVs

Follow the same route whenever you explain or run the client delivery. Start with both supplied packet files, the C I C I D S twenty seventeen export and the U N S W export. Preparation checks their contents and organizes repeated packet payloads. Training then uses selected examples from both sources. Validation chooses the saved version that will be used for prediction. That chosen classifier is called the joint pretrained model. Later, a separate unlabeled packet file goes through that saved model to produce scores. Every step in this course belongs to that single supported packet workflow.

On screen: Both uploaded CSVs · Prepare and train packet adaptation · Use the joint pretrained model

Evidence: [project-guide](#), [packet-report](#), [packet-data](#)

00:01:57 — What joint pretrained means

The model's name tells you two useful facts. Joint means that its supervised training includes examples from both uploaded sources. Pretrained means that the central encoder begins with released weights learned earlier. An encoder turns input numbers into a representation that later layers can use. A new final layer learns the two packet categories. The released initialization file is not the finished joint classifier. The repository also trained comparison models and simpler controls to investigate this choice. Those experiments help explain the evidence. They do not give the client several competing setup routes. For the supported demonstration and prediction commands, we use the validation selected joint pretrained classifier.

On screen: Joint: examples from both sources · Pretrained: transferred starting weights · One selected packet classifier

Evidence: [project-guide](#), [packet-report](#), [packet-protocol](#)

00:02:45 — Learning a rule and using a saved rule

Think of training as practice with an answer key. The model makes a prediction, compares it with the supplied answers, and adjusts numbers called weights. A checkpoint saves those learned numbers together with information needed to interpret them. Inference means using a selected checkpoint while keeping its weights fixed. It resembles applying an established rule to another example. During inference the packet bytes are available, but the true answer need not be. Without a known answer, we can report a prediction, but we cannot measure whether that prediction was correct. Downloading source code alone does not supply a trained checkpoint.

On screen: Training changes weights · Checkpoint saves a model · Inference applies fixed weights

Evidence: [project-guide](#), [packet-report](#)

00:03:24 — Two repositories, one packet workflow

There are two repositories because different readers need different packages. Net Mamba Plus reproduction is the full development, research, and learning source. It includes the implementation, measured evidence, explanations, and teaching media. Net Mamba Plus client is the concise executable delivery, with setup instructions, packet programs, tests, and the packet evidence needed for handoff. The client edition omits the teaching videos, PowerPoint decks, and presenter scripts. Both describe the same packet model workflow. The client has separate Git history and receives verified exports; its client manifest identifies the source revision and delivered files. Send the client repository together with the client project guide. Use the reproduction repository when someone wants to understand the broader development and teaching record.

On screen: Full source and teaching record · Concise executable client delivery

Repository — Purpose

netmambaplus-reproduction — Development, research, evidence, media

netmambaplus-client — Setup, execution, tests, packet evidence

Evidence: [project-guide](#)

00:04:19 — Explain the delivered capability precisely

A useful client explanation separates a working capability from the environment around it. This delivery contains runnable packet classification, verification tools, and an offline viewer of recorded predictions. The viewer helps inspect the experiment; opening it does not capture traffic or run fresh model inference. The measured results describe selected examples from the supplied exports. They do not establish detection quality on every real network. As we continue, keep following one question: what enters each step, what leaves it, and which saved record supports that description? That habit lets you explain the repository confidently without needing to become its programmer.

On screen: Runnable packet classification · Saved evidence and offline viewer · Performance tied to the tested exports

Evidence: [project-guide](#), [packet-report](#)

00:05:06 — Explain it back

Practice for 10 seconds: Explain the packet route and the purpose of each repository.

On screen: Explain the packet route and the purpose of each repository.

Evidence: [project-guide](#)

2. The paper's idea and our packet adaptation

00:05:16 — Three views of traffic in the paper

The paper asks how a model can classify network traffic while processing sequences efficiently. Its Net Mamba Plus representation combines three views of related traffic. Bytes represent recorded content. Packet sizes describe how large the pieces are. Arrival intervals describe the gaps between them. Imagine observing deliveries: what each package contains, how large it is, and how deliveries are spaced can provide different clues. The technical word multimodal means these multiple traffic views. It does not mean a conversation involving pictures and audio. Before copying an architecture diagram, we must check which of those observations our actual data can support.

On screen: Bytes: recorded content · Sizes: packet lengths · Timing: arrival intervals

Evidence: [paper-notes](#), [upstream-comparison](#)

00:05:58 — Mamba carries a numerical state forward

Mamba is a selective state space model. The name sounds difficult, but begin with a simple picture: reading a sequence while continually updating a small numerical notebook. The notebook is the state. Each new input influences how that state changes and which information is carried forward. Selective means that this behavior depends on the input, using learned calculations. The notebook analogy explains information flow; the actual state is a collection of numbers. It is neither written reasoning nor a live connection database. In our project this encoder processes packet byte representations, then passes useful numerical summaries to a classifier.

On screen: Read the next input · Update a learned state · Carry information onward

Evidence: [paper-notes](#), [upstream-comparison](#), [packet-report](#)

00:06:41 — A packet row cannot establish a whole flow

The paper's flow representation groups related packets according to connection and ordering rules. Picture sorting letters into conversations: you need reliable information about who wrote to whom and which message came first. The supplied packet tables lack the addresses, ports, connection identifiers, and established ordering needed for that reconstruction. Five neighboring spreadsheet rows might describe unrelated conversations. Matching their labels or protocol names does not repair the missing information. Even the recorded time difference has an undocumented interpretation. We therefore treat each row's stored payload as one packet example. This explicit choice determines what the resulting measurements can mean.

On screen: Packet: one network unit · Flow: related packets with defined order · Connection and ordering evidence are missing

Evidence: [paper-notes](#), [csv-profile](#), [packet-report](#)

00:07:28 — Pretraining learns to reconstruct hidden content

Think of pretraining as practice with part of each example hidden. The model sees the remaining information and tries to reconstruct what was hidden. Its answer key is the original input content, rather than an attack category. The reconstruction error guides adjustments to the weights. This can produce a useful starting representation before a later classification task has been learned. Our packet model loads the encoder weights released by the paper's authors. We did not recreate their complete earlier training corpus. The checkpoint's full exposure history remains unknown, so transferring those weights does not establish that earlier examples were independent.

On screen: Hide selected input content · Predict what was hidden · Adjust the encoder's weights

Evidence: [paper-notes](#), [packet-report](#), [project-guide](#)

00:08:10 — Supervised learning teaches the packet categories

Supervised classification asks a different question from reconstruction. Instead of filling in missing input content, the model learns which supplied category belongs with an example. Here those categories are benign and attack. The pretrained encoder provides a starting point, and a fresh classification head learns how to turn its summaries into two scores. A head is simply the final decision layer. During training the known labels guide weight changes; they are not inserted into the encoder's input. This distinction matters when explaining the research connection: we reuse the learning machinery while adapting its observation unit and its final classification task.

On screen: Packet bytes enter the encoder · New head produces two scores · Supplied labels guide training

Evidence: [project-guide](#), [packet-report](#), [upstream-comparison](#)

00:08:52 — What the packet adaptation actually observes

In the packet adaptation, the actual observations are the fifteen hundred stored payload bytes. The size and timing branches contain no observed sequences. Two learned prefix components remain, but they are model parameters, not measurements taken from the spreadsheet. The recorded time to live, length, protocol, time difference, dataset identity, and label never enter the neural forward pass, meaning the calculation that produces its scores. Metadata is examined separately and used in diagnostic controls. This is why a drawing with three traffic views needs a careful caption: it describes the paper's architecture, while this adaptation supplies only the packet bytes.

On screen: All 1,500 stored payload bytes · No observed size or interval sequences · Metadata stays outside the neural call

Evidence: [packet-report](#), [project-guide](#), [packet-protocol](#)

00:09:36 — A similar percentage is not the same experiment

The paper reports ninety seven point five zero percent flow accuracy on C I C I o T twenty twenty two in Table Four. That dataset is different from the uploaded C I C I D S twenty seventeen export. Our packet study measures a different observation unit, different data, a binary target, and group weighted balanced accuracy. Balanced accuracy averages how well the two categories are recovered. Ordinary accuracy counts the overall fraction correct. Similar percentages do not make those measurements interchangeable. The repository has not reproduced the paper's flow result through these CSVs. When a client asks whether the adaptation matches the paper, explain the changed task first. Likewise, the paper's efficiency or deployment claims do not automatically describe our checked GPU execution.

On screen: Paper Table IV: 97.50% CICIOT2022 flow accuracy · Packet study: a different task and metric · Compare the conditions before the numbers

Evidence: [project-guide](#), [packet-report](#), [paper-notes](#), [upstream-comparison](#)

00:10:34 — Explain it back

Practice for 10 seconds: Why is this a packet adaptation rather than the paper's flow experiment?

On screen: Why is this a packet adaptation rather than the paper's flow experiment?

Evidence: [packet-report](#), [paper-notes](#)

3. Read the CICIDS2017 packet CSV

00:10:44 — A text table of packet examples

Now open the first source conceptually, as though we were looking across a very wide spreadsheet. C S V means comma separated values, a text format containing rows and columns. This supplied C I C I D S twenty seventeen export contains one million, four hundred ten thousand, two hundred fifty five packet rows and fifteen original labels. Each row is one stored packet example. The row count does not count connections, distinct payloads, or successful predictions. The dataset name identifies the supplied export under study; it does not establish every detail of how that export was originally produced.

On screen: CICIDS2017 supplied export · 1,410,255 packet rows · 15 original labels

Evidence: [project-guide](#), [paper-notes](#), [csv-profile](#), [packet-manifest](#)

00:11:27 — Read one row in three parts

Move across a single row from left to right. The first fifteen hundred columns are ordered payload byte slots. They are followed by four metadata fields and then the label, giving fifteen hundred and five columns altogether. Metadata means recorded information about the example. The label is its supplied answer. Keep these three roles separate even though the spreadsheet places them side by side. The preparation code expects the exact header sequence, so swapping columns changes the input contract. Fifteen hundred byte columns describe positions within one packet representation; they are not fifteen hundred independent packets or separate training examples.

On screen: One row represents one packet example · 1,505 columns in a fixed order

Part — Contents

Payload — 1,500 ordered byte slots

Metadata — ttl, total_len, protocol, t_delta

Target — label

Evidence: [paper-notes](#), [packet-data](#), [packet-manifest](#)

00:12:08 — Byte slots contain numbers, including stored zeros

A byte is a small numerical value from zero through two hundred fifty five. In the CSV it is written as text, and preparation checks that it represents an exact valid integer before conversion. Think of an ordered strip of numbered tiles. The model receives the strip's numerical pattern; this is not a promise to translate its contents into readable messages. A tile containing zero is still part of the stored representation. The export does not reliably identify which zeros are padding. We preserve all the stored slots, rather than guessing that a zero or recorded length justifies removing them.

On screen: Integer values from 0 through 255 · Exact order is preserved · Stored zeros remain in the input

Evidence: [packet-data](#), [packet-report](#), [paper-notes](#), [packet-manifest](#)

00:12:47 — Four recorded properties, with specific limits

The next fields describe recorded properties. Time to live relates to a packet's lifetime across the network. Total length records a packet length, but does not identify meaningful payload slots. Protocol records a carrying category such as T C P or U D P; it is not an attack answer. Time delta records a difference whose units and reference event are not established here. These values are profiled, and the separate metadata controls validate and use them. They never enter the native packet classifier. None supplies the missing connection identity or proves how neighboring rows should be ordered.

On screen: Profiled and checked for separate controls · Never inputs to the native neural call

Field — Recorded meaning

t_{tl} — Time to live; network lifetime field

total_len — Recorded packet length

protocol — Traffic carrying protocol

t_delta — Time difference; interpretation unestablished

Evidence: [paper-notes](#), [project-guide](#), [packet-report](#), [packet-protocol](#), [packet-manifest](#)

00:13:28 — Keep the answer outside the model input

The last column supplies the example's label. In this source, the registered label benign maps to class zero. The other explicitly registered labels map to class one, attack. Unknown spellings are rejected instead of silently becoming an attack example. During supervised training, these answers guide learning after the model produces its scores. Giving the answer to the model as an input would defeat the purpose, like leaving the solution printed on an exam question. Later, known labels can score held out predictions. The separate prediction command requires an unlabeled input, so its model must decide from the stored bytes alone.

On screen: BENIGN maps to benign, class 0 · Registered attack labels map to class 1 · Unknown labels fail validation

Evidence: [packet-data](#), [project-guide](#), [packet-report](#), [packet-manifest](#)

00:14:10 — Many rows repeat the same stored payload

A large spreadsheet can repeat the same input many times. In this source, the original rows reduce to four hundred thirty eight thousand, eight hundred eighty six distinct stored payload groups. A group collects rows whose entire payload byte sequence is identical. Imagine receiving repeated photocopies of the same page: counting copies and counting distinct pages answer different questions. Preparation keeps the original label counts within each group, including disagreements. It does not pretend that repeated rows supply the same diversity as new payloads. This distinction will matter both when dividing the data and when interpreting a model's reported score.

On screen: 1,410,255 rows · 438,886 distinct payload groups · Every original label count is retained

Evidence: [packet-report](#), [packet-data](#), [packet-manifest](#)

00:14:54 — Check what the selected test set contains

Support means how many examples actually stand behind a measurement. The export has fifteen original categories, but that does not guarantee useful test coverage for every one. After duplicate grouping and the study's selection limits, the selected C I C test set contains no port scan rows and only four distributed denial of service rows. We therefore have no measured port scan performance there, and four rows cannot support a broad claim about distributed attacks. Preparation accounts for the whole file, while training uses selected and sampled groups. Always ask what reached the relevant experiment, not just what arrived originally.

On screen: Counts are not performance · No PortScan test rows · Four DDoS test rows

Evidence: [packet-report](#), [project-guide](#), [packet-protocol](#), [packet-manifest](#)

00:15:36 — Explain it back

Practice for 10 seconds: Explain the three parts of a row and which part reaches the model.

On screen: Explain the three parts of a row and which part reaches the model.

Evidence: [packet-data](#), [project-guide](#), [packet-manifest](#)

4. Add UNSW and protect the experiment's split

00:15:46 — UNSW uses the same row structure

The second supplied export, U N S W, contains seventy nine thousand, eight hundred eighty one packet rows and ten original labels. Its rows follow exactly the same column structure as the first file: fifteen hundred payload byte slots, four metadata fields, and a label. That shared structure makes a common packet preparation route possible. The examples and original category vocabulary are different. For instance, ordinary traffic is labeled normal here, while other registered names include generic and exploits. A familiar table layout is useful compatibility evidence, but it does not make the underlying traffic or its supplied annotations identical.

On screen: 79,881 packet rows · The same 1,505-column schema · 10 original labels

Evidence: [csv-profile](#), [paper-notes](#), [packet-data](#), [packet-manifest](#)

00:16:30 — Map source labels to the shared binary task

To train one binary classifier, preparation gives both sources a shared target definition. C I C's benign label and U N S W's normal label become the benign category. The other registered labels become attack. This mapping creates a compatible learning question; it does not assert that all the original attack names mean the same thing. Their source names and counts remain available for analysis. Dataset identity stays outside the neural forward pass. We have also not independently established that every supplied annotation reflects operational maliciousness. The model learns the provided labeling task, with those limits carried into its evaluation.

On screen: Keep original source annotations · Use one benign or attack target

Source — Benign — Attack

CICIDS2017 — BENIGN — Other registered CIC labels

UNSW — normal — Other registered UNSW labels

Evidence: [packet-data](#), [packet-report](#), [project-guide](#), [packet-manifest](#)

00:17:15 — Combine packet examples without inventing connections

Combining the files means bringing compatible packet examples into the same study. Conceptually, we place their example collections together, while preserving the source records. We do not match row one in one file with row one in the other, or append one packet's bytes to another packet's bytes. That would invent a relationship the data never established. The joint training procedure draws selected groups from both sources to teach one classifier. Think of combining two sets of practice cards that ask the same kind of question. Each card remains a separate example, even though both sets contribute to learning.

On screen: Compatible examples from both sources · Shared preparation and grouping · Joint packet training

Evidence: [packet-report](#), [packet-data](#), [packet-protocol](#), [packet-manifest](#)

00:17:54 — Find exact payload duplicates across both sources

Duplicate checking must cover both files together. The complete preparation identifies four hundred seventy eight thousand, forty four globally distinct payloads. Global means the same stored bytes count as one identity even if they appear in different sources. The code computes a fingerprint called S H A two fifty six from all fifteen hundred canonical bytes. This provides a practical identity for grouping exact payloads, and the preparation checks matching payload content. Original row counts and source labels remain associated with that identity. A fingerprint describes exact stored content; it does not reveal which physical connection or capture produced it.

On screen: Compare all 1,500 stored bytes · Assign a SHA-256 fingerprint · 478,044 global payload groups

Evidence: [packet-data](#), [packet-report](#), [project-guide](#), [packet-manifest](#)

00:18:38 — Shared payloads can carry conflicting answers

Ten exact payloads appear in both supplied files. Nine of those shared payloads have conflicting binary labels. That means identical model input can be associated with benign in some rows and attack in others. The model is missing the context that might explain the difference, and the labels have not been independently adjudicated. We do not fix this by deleting the inconvenient answer or declaring one source correct. The original counts remain visible. These nine are the conflicting groups among the shared payloads; this is not a claim that the whole combined dataset contains only nine disagreements of this kind.

On screen: 10 payloads appear in both files · 9 of those have conflicting binary labels · Disagreements remain in the evidence

Evidence: [packet-report](#), [packet-data](#), [project-guide](#), [packet-manifest](#)

00:19:17 — Preserve disagreement as label proportions

Use an illustrative group to see how the training handles disagreement. Suppose identical bytes occur in nine rows labeled benign and one row labeled attack. The group's target is ninety percent benign and ten percent attack. Those are proportions of supplied labels, not a newly verified probability about network behavior. Each sampled group has equal total weight in the learning calculation, with that weight divided according to its answers. This preserves the contradiction while preventing a highly repeated payload from automatically dominating the training loss. Later, the evidence can distinguish weighting original rows from giving each payload one total vote.

On screen: Illustration: nine benign rows, one attack row · Training target: 90% benign, 10% attack · Each sampled group has equal total weight

Evidence: [packet-report](#), [packet-protocol](#), [project-guide](#), [packet-manifest](#)

00:20:01 — Keep exact duplicates inside one partition

Each global payload group receives a partition through a hash rule using split seed seventeen. Training groups teach the weights. Validation groups select the saved checkpoint. Test groups measure it after selection has frozen. The target proportions are seventy, fifteen, and fifteen percent, with later caps selecting groups inside those partitions. Keeping identical payloads together prevents exact copies from leaking from practice into the exam, including across both sources. The animated symbols represent whole groups that stay intact. This does not remove every near duplicate or prove capture independence. These results describe the defined export study, not guaranteed performance on a client's network.

On screen: Group first, using both sources · Seed 17: target 70% / 15% / 15% · Train, validation, and test stay distinct

Exact payloads: Same bytes stay together

Train: Target 70% · learn weights

Validation: Target 15% · select checkpoint

Test: Target 15% · measure afterward

Seed 17 · target proportions before caps · identical payloads stay in one split; capture independence unknown.

Evidence: [packet-data](#), [packet-report](#), [project-guide](#), [packet-manifest](#)

00:20:49 — Explain it back

Practice for 10 seconds: Why group both sources before splitting, and what uncertainty remains?

On screen: Why group both sources before splitting, and what uncertainty remains?

Evidence: [packet-data](#), [packet-report](#), [packet-manifest](#)

5. Follow one packet through the model

00:20:59 — One packet supplies the model input

Follow one table row from the supplied CSV into the classifier. Its payload contains fifteen hundred stored bytes in a fixed order. These bytes are the observation available to the model, and changing their order can change the pattern. The metadata records length, time to live, protocol, and time delta, but stays outside neural input. The classifier receives only the transformed payload. During training, a label supplies the answer used to score its prediction afterward. Source identity and that answer never enter the forward calculation. The diagram separates the branches so you can point to exactly what informs a prediction.

On screen: 1,500 ordered stored bytes per packet · Metadata stays outside neural forward · Labels are targets, never model features

CSV row: Bytes + context + label

Payload: 1,500 stored bytes

Metadata: Ignored by neural input

Classifier: Receives payload only

Illustrative data path · supplied labels affect training loss, never model input.

Evidence: [project-guide](#), [packet-model](#), [packet-data](#)

00:21:41 — Rescale the bytes without changing their order

The adapter converts the validated bytes into floating point numbers, which can represent fractions. It divides each byte by one hundred twenty seven point five and subtracts one. Zero becomes negative one, and two hundred fifty five becomes positive one. This changes the numerical scale while preserving the byte order. A tensor is an array with named dimensions in our explanation: the batch, two dimensions of size one, and fifteen hundred byte positions. Stored zeros remain because the export does not establish which zeros represent padding. Removing them would silently change the observation supplied to this model.

On screen: Byte 0 becomes -1 ; byte 255 becomes $+1$ · Tensor: batch $\times$ 1 $\times$ 1 $\times$ 1,500 · Stored zeros remain in the input

```
batch = payload_tensor.shape[0]
byte_stream = (
    payload_tensor.to(dtype=torch.float32) / 127.5 - 1
).reshape(batch, 1, 1, 1500)
```

Evidence: [packet-model](#), [packet-report](#)

00:22:21 — Assemble 378 tokens from one packet

Start with fifteen hundred bytes from one packet. Grouping and embedding turn each four neighboring bytes into one learned vector, producing three hundred seventy five vectors. The assembled sequence adds three learned tokens, bringing its length to three hundred seventy eight. Each position contains two hundred fifty six numerical coordinates. Four Mamba blocks transform that sequence using the original encoder machinery. Finally, class logits provide two scores after the summary outputs are added and passed through the binary head. These are numerical processing stages, not words in a language model. The animation is a schematic of the computation, not a live execution.

On screen: 1,500 ordered bytes · 375 vectors from groups of 4 bytes · 3 learned tokens $\rightarrow$ 378 tokens $\times$ 256 values

Bytes: 1,500 positions

Embed: Groups of 4 375 vectors

Sequence: +3 → 378 × 256

Mamba: 4 native blocks

Output: 2 class logits

Packet adaptation · 375 byte vectors + 3 learned tokens; empty size and timing observations.

Evidence: [packet-model](#), [packet-report](#)

00:23:07 — Empty streams still have learned prefixes

The size stream contains no observations, and the interval stream also contains no observations. Interval means a gap between arrivals; these CSVs do not establish the flow timing needed for that input. Their two learned prefixes remain trainable model parameters, beginning identically for each packet. The native encoder scans in a causal order, meaning earlier positions cannot read later positions. These prefix outputs therefore cannot summarize bytes that appear afterward, although later byte positions can use the prefixes. Calling all three positions summaries of the packet would conceal this distinction. Empty streams are an explicit part of the adaptation.

On screen: Size observations: 0; interval observations: 0 · Two learned prefixes begin identically for each packet · Prefix outputs cannot summarize later bytes

Evidence: [packet-model](#), [packet-report](#), [initialization](#)

00:23:51 — Four blocks lead to two class scores

The sequence enters four native Mamba blocks. Each block updates a learned numerical state as it processes the ordered representation. The fusion step adds the three designated output vectors, and the binary head turns that combined vector into two logits. A head is the final scoring layer; a logit is a raw class score. One score represents benign and the other attack. Across the full packet classifier there are one million eight hundred fifty two thousand four hundred sixteen parameters, the adjustable numbers learned during training. These parameters are different from the changing byte values in each incoming observation.

On screen: 4 native Mamba blocks · Add the 3 output summary vectors · Binary head → benign and attack logits

Evidence: [packet-model](#), [packet-report](#)

00:24:31 — Transfer compatible weights into the packet model

Pretrained initialization starts with useful numerical settings from the authors' released checkpoint. The adapter copies fifty compatible model tensors and excludes thirty one tensors used for reconstructing hidden inputs. A tensor here is a stored array of parameters. The binary head starts fresh. Learned position values are explicitly selected from the original table, mapping three hundred seventy eight positions from four hundred forty three. Scratch initialization uses the same coordinates from fresh weights. This preserves a defined comparison, while the shorter byte sequence and empty numeric streams keep the packet experiment distinct from the paper's flow experiment.

On screen: Reuse compatible encoder state; start a fresh binary head · Learned positions remain trainable

Component — Packet treatment

Compatible trunk tensors — 50 copied

Reconstruction tensors — 31 excluded

Binary head — Fresh weights

Position table — 378 selected from 443; trainable

Position indices: [20, 41] + range(42, 417) + [442].

Evidence: [packet-model](#), [packet-report](#), [initialization](#)

00:25:14 — Explain the complete feature path

Now connect the pieces in ordinary language. One packet supplies ordered stored bytes; those bytes are rescaled and grouped into learned vectors. Three learned tokens join the sequence, four encoder blocks process it, and a binary head produces the final scores. The optional metadata has not joined that journey. The label helps training judge a prediction, but does not enter the forward calculation that produces it. This model performs numerical traffic classification without calling a chat service. Use the coming pause to explain both the information it receives and the information its input contract deliberately leaves outside.

On screen: Describe the stored bytes and their transformation · Identify learned tokens and empty streams · Separate model inputs from training answers

Evidence: [project-guide](#), [packet-model](#), [packet-report](#)

00:25:56 — Your turn · 10 seconds

Practice for 10 seconds: Explain how one packet becomes two scores, including what stays outside.

On screen: Explain how one packet becomes two scores, including what stays outside.

Evidence: [packet-model](#), [packet-report](#)

6. Understand how the packet model learns

00:26:06 — Preserve disagreements in the learning target

Training begins with supplied answers, including their disagreements. Imagine one identical payload recorded nine times as benign and once as attack. The group receives a soft target of ninety percent benign and ten percent attack. Soft means the answer preserves proportions instead of forcing one label to win. When that group is sampled, its total contribution to the learning loss is one group, divided according to those proportions. Repeated copies do not acquire extra total influence simply by being numerous.

On screen: Same payload: 9 benign labels + 1 attack label · Soft target: 90% benign / 10% attack · One sampled group contributes one total weight

Evidence: [packet-data](#), [packet-training](#), [packet-protocol](#)

00:26:41 — Compare six training arms

An experimental arm is one defined training comparison. Here the source choice is C I C alone, U N S W alone, or both together. Each choice runs with transferred pretrained weights and with fresh scratch weights, giving six arms. All use the same packet architecture and one model seed, zero. A seed fixes the recorded random choices for this run. Paired pretrained and scratch arms have identical fresh heads and sampled groups, helping isolate the initialization comparison. One seed still gives only one observed run per arm; it does not describe variation across many independently repeated training runs.

On screen: Three source choices × two initializations · One model seed: 0 · Paired arms share fresh heads and sampled groups

Training source — Transferred initialization — Fresh initialization

CIC only — `cic_pretrained` — `cic_scratch`

UNSW only — `unsw_pretrained` — `unsw_scratch`

CIC + UNSW — `joint_pretrained` — `joint_scratch`

Evidence: [packet-protocol](#), [packet-model](#), [packet-training](#)

00:27:23 — One update is one learning step

One update means one cycle through this diagram. Sample sixty four training groups, with replacement. The forward calculation produces two scores for each payload. The loss compares those scores with the supplied label proportions. Backward computation finds gradients, numbers that describe how weight changes could reduce that loss. The optimizer then changes the weights, completing the update. Joint batches contain thirty two groups from each source. Every arm completed one thousand updates, or sixty four thousand group presentations. Repeatedly presenting the same group is allowed. An update is not an epoch over the complete uploaded tables, and it does not modify the labels.

On screen: Sample a batch of 64 groups · Compare predictions with group targets · Update weights; complete 1,000 updates

Forward: Payload → two scores

Loss: Compare with soft targets

Backward: Calculate gradients

Update: Optimizer changes weights

Recorded protocol: 1,000 updates per arm · batch 64 · replacement sampling.

Evidence: [packet-training](#), [packet-protocol](#), [packet-results](#)

00:28:08 — Caps and replacement sampling define exposure

The selected training pool contains one hundred thousand C I C groups and twenty seven thousand three hundred eighty U N S W groups. Joint batches draw thirty two from each source, keeping the two sources equally represented within each batch. Sampling uses replacement, which means a group can appear again while another selected group never appears. Across one thousand updates, each model receives sixty four thousand group presentations. An epoch would mean a pass through a defined training dataset. These presentations are not full CSV epochs, and validating all original rows does not mean training on all of them.

On screen: Joint batches: 32 groups from each source · Sampling with replacement can repeat a group · 64,000 presentations per model, not full CSV epochs

Source — Selected training groups — Groups per joint batch

CIC — 100,000 — 32

UNSW — 27,380 — 32

Evidence: [packet-report](#), [packet-training](#), [packet-protocol](#)

00:28:50 — Validation chooses; frozen test groups measure

Validation runs every hundred updates, using groups separate from training. Its selection score is group weighted macro F one: a measure that combines precision and recall for each class and averages the two classes. Precision asks how often a predicted class is correct; recall asks how much of that class was found. Joint selection averages the two source scores. The earliest checkpoint with the best score wins any tie. All six selections freeze before the final test predictions. This order lets test groups measure the selected models without using their answers to choose which checkpoint looks most attractive.

On screen: Validation every 100 updates · Select earliest best group macro-F1 · Freeze all selections before final test inference

Evidence: [packet-protocol](#), [packet-training](#), [packet-results](#)

00:29:30 — Freeze the experiment and retain its receipts

A frozen protocol records what experiment will be run before its final results are known. It binds the data manifest, original source identities, implementation files, sampling rules, and training budget. Binding means recording hashes that identify exact file contents. A hash can reveal a changed file, although it cannot prove that an annotation is true. Checkpoints preserve the relevant protocol and data identities alongside their weights. Training receipts record achieved updates, selection history, and optimizer counters. Those counters establish that training continued even when an earlier checkpoint was selected.

On screen: Protocol binds data, code, rules, and budget · Checkpoints retain selection and input identities · Receipts establish achieved optimizer updates

Evidence: [packet-protocol](#), [packet-model](#), [packet-training](#)

00:30:12 — Distinguish training, selection, and inference

Before opening the code, explain three different uses of data. Training groups change model weights through repeated learning updates. Validation groups choose a saved checkpoint according to the rule fixed in advance. Inference applies selected weights to observations without another learning update; final test inference adds known answers only to calculate measurements. The client uses the joint pretrained selection, so both sources contributed supervised training examples. The separate split seed, seventeen, assigned payload groups to partitions; it is different from the model seed, zero. Use the pause to identify which stage changes weights, which chooses them, and which measures them.

On screen: Training groups change the weights · Validation groups choose saved weights · Inference applies a selected checkpoint

Evidence: [packet-data](#), [packet-protocol](#), [packet-training](#), [project-guide](#)

00:30:58 — Your turn · 10 seconds

Practice for 10 seconds: Explain which data changes weights, chooses a checkpoint, and measures it.

On screen: Explain which data changes weights, chooses a checkpoint, and measures it.

Evidence: [packet-protocol](#), [packet-training](#)

7. Read the code behind training and prediction

00:31:08 — Find the five implementation responsibilities

The files divide the workflow into responsibilities that are easier to inspect separately. The data module validates the supplied tables and prepares payload groups. The model module connects packet bytes to the pinned native encoder and manages compatible checkpoints. Shared study utilities calculate measurements and handle evidence records. The training program runs the frozen comparisons and records their selected weights. The prediction program applies a selected checkpoint to an unlabeled CSV.

On screen: Follow data preparation through training to prediction · Shared metrics keep result arithmetic consistent

File — Responsibility

packet_data.py — Validate and prepare groups

packet_model.py — Native adapter and checkpoints

packet_study.py — Metrics and evidence utilities

tools/train_packet_model.py — Run the frozen training study

tools/predict_packets.py — Predict an unlabeled packet CSV

Evidence: [packet-data](#), [packet-model](#), [packet-training](#), [packet-prediction](#), [packet-review](#)

00:31:40 — The forward call receives only payload tensors

A prepared batch becomes an unsigned byte tensor, meaning an array whose entries hold integers from zero through two hundred fifty five. Its dimensions are the batch size and fifteen hundred bytes. The forward call passes this payload tensor into the adapter and returns two logits per observation. The adapter handles normalization and constructs the empty numeric streams we already examined. The training answers exist in a different tensor and never enter this call.

On screen: Input: unsigned byte tensor, batch × 1,500 · Output: floating point logits, batch × 2 · Training targets stay outside this call

```
# payload: [batch, 1500], unsigned bytes
logits = packet_model.forward_payload(model, payload)['logits']
# logits: [batch, 2], floating point scores
```

Evidence: [packet-model](#), [packet-training](#)

00:32:09 — Loss, gradients, and an optimizer update

The learning loop first clears gradients left from the previous update. It obtains logits, then computes cross entropy against the soft targets. Cross entropy is the loss measuring how poorly the scores fit those target proportions. Backward calculation finds gradients: numerical sensitivities showing how parameter changes affect the loss. The optimizer uses them to update the model. The complete implementation also rejects invalid numerical values, limits gradient size, and checks that learning signals reach the byte embedding, every encoder block, and the head. Those checks establish an actual learning path; held out measurements still determine whether its predictions are useful.

On screen: Loss measures disagreement with soft targets · Gradients connect that loss to model parameters · The optimizer changes weights

```
optimizer.zero_grad(set_to_none=True)
logits = packet_model.forward_payload(model, payload)['logits']
loss = -(
    truth * torch.log_softmax(logits, dim=1)
).sum(dim=1).mean()
loss.backward()
optimizer.step()
```

Evidence: [packet-training](#), [packet-protocol](#)

00:32:55 — Load the selected checkpoint for inference

Read the joint pretrained selection from the frozen checkpoint record. That entry identifies the saved weights chosen by validation, including their file identity. The model loader strictly checks the packet architecture, expected parameter arrays, class mapping, and provenance, which records where the model came from. The prediction program then places the loaded model in evaluation mode and applies it without learning updates. This is why the released pretraining file cannot replace the selected packet checkpoint: the client needs the binary classifier produced by supervised training. Keeping the weights together with their identifying record makes a later prediction traceable.

On screen: Read Selected joint pretrained checkpoint · Strictly load weights and their metadata · Apply fixed weights to new packet bytes

Evidence: [packet-model](#), [packet-prediction](#), [packet-training](#), [client-setup](#)

00:33:39 — Validate the CSV before publishing completion

The prediction CSV accepts fifteen hundred payload columns, optionally followed by the four metadata columns. A label column is rejected, so create a separate unlabeled copy. Header order, row width, and payload byte validity are checked. A cap of one hundred twenty eight limits predicted rows while the program still scans the entire file. Predictions may be computed during that scan; successful completion requires validation to finish. A malformed later row prevents a successful completed result even if earlier predictions were calculated. The receipt reports validated and predicted counts separately, so a capped run cannot claim prediction coverage it lacks.

On screen: Exact unlabeled header and valid byte cells are required · A prediction cap still scans the whole CSV · The receipt distinguishes validated rows from predictions

Input layout — Accepted

1,500 ordered payload columns — Yes

Payload columns + ttl,total_len,protocol,t_delta — Yes; metadata stays outside forward

Any header containing label — No

Full input validation precedes a successful completion receipt, not the first prediction.

Evidence: [packet-prediction](#), [client-setup](#)

00:34:23 — Read class decisions and uncalibrated scores

Each prediction record includes the row identity, payload hash, two logits, class name, and probabilities. The larger logit chooses the class, with an exact tie assigned to benign. Softmax converts the logits into nonnegative numbers that sum to one; it does not supply independent evidence about the customer's network. These scores are uncalibrated, meaning their numerical size has not been established as a reliable frequency of real malicious events. The separate receipt ties outputs to input, checkpoint, and code identities.

On screen: logits: two finite raw scores · class_name: benign or attack · probabilities and receipt: scores plus run identity

Evidence: [packet-prediction](#), [project-guide](#)

00:34:58 — Check native execution on the prepared packets

The native packet gate checks whether the actual model operates in the prepared environment. Its recorded run on G B ten passed seventeen tests with none skipped, using two stored payloads from each source. It exercises compatible weight transfer, tensor shapes, gradients, an optimizer update, and strict checkpoint loading after saving. Synthetic targets supply a controlled learning signal for these checks. They do not measure benchmark accuracy or repeat the six arm training study.

On screen: 17 native GB10 tests passed; 0 skipped · Stored payloads with synthetic targets test functionality

```
python tools/check_packet_runtime.py \
--data runs/packet-data-new \
--upstream upstream/NetMambaPlus \
--initialization assets/checkpoints/fuse3_mamba.pth \
--output runs/packet-runtime-new
```

Evidence: [native-gate](#), [client-setup](#), [packet-model](#)

00:35:31 — Your turn · 10 seconds

Practice for 10 seconds: Explain what a successful capped prediction receipt proves about the input.

On screen: Explain what a successful capped prediction receipt proves about the input.

Evidence: [packet-prediction](#), [native-gate](#)

8. Explain the measured results and their limits

00:35:41 — The selected joint model's measured results

Start with the delivered joint pretrained model. It reached ninety seven point five six percent group balanced accuracy on C I C and ninety four point two nine percent on U N S W. The test sets contain twenty thousand and five thousand nine hundred thirty distinct payload groups, respectively, totaling twenty five thousand nine hundred thirty predictions. These groups came from the partitions reserved earlier. The selected weights were already frozen. This ties the headline to a defined model, unit, and evaluation set. Carry those conditions with the percentages whenever you explain what the study established to another person.

On screen: Joint pretrained checkpoint selected before final tests · 25,930 distinct test payload groups in total

Test source — Distinct groups — Group balanced accuracy

CIC — 20,000 — 97.56%

UNSW — 5,930 — 94.29%

All six native models were evaluated on these same source test groups.

Evidence: [packet-results](#), [packet-report](#), [project-guide](#)

00:36:22 — Class balance and duplicate weighting answer different things

Balanced accuracy averages benign recall and attack recall, giving the two classes equal importance. Recall means the fraction of a supplied class correctly recognized. Group weighting gives each distinct payload one total vote, divided across conflicting labels. Row weighting gives every original row a vote, so repeated payloads can change the outcome. With row weighting, the joint scores become ninety six point three one percent on C I C and eighty nine point two four on U N S W. The model has not changed between these calculations; the weights assigned to the evaluation observations have changed.

On screen: Balanced accuracy averages benign and attack recall · Group weight: one total vote per distinct payload · Row weight: one vote per original row

Joint pretrained test — Group balanced accuracy — Row balanced accuracy

CIC — 97.56% — 96.31%

UNSW — 94.29% — 89.24%

Evidence: [packet-report](#), [packet-results](#), [packet-review](#)

00:37:03 — Pretrained models show transfer tradeoffs

Read each row as one trained model and each column as a test source. The C I C pretrained model performs strongly on its own source but reaches only fifty three point seven three percent on U N S W. Transfer means applying learned weights to another source without supervised fitting on that source. Joint training changes this comparison: against the corresponding pretrained source models, it gains one point zero seven percentage points on U N S W and loses one point one one on C I C. These are observed tradeoffs in this run, not a universal benefit.

On screen: Each row is one model; each column is a test source · Joint versus source models: +1.07 points UNSW, -1.11 CIC

Pretrained training source — CIC test — UNSW test

CIC only — 98.67% — 53.73%

UNSW only — 71.52% — 93.22%

Joint — 97.56% — 94.29%

Every score in this table is group weighted balanced accuracy.

Evidence: [packet-results](#), [packet-report](#)

00:37:41 — All scratch arms trained; their outcomes were weak

Scratch models expose an important weak result in the completed study. The C I C only and U N S W only selections both score fifty percent balanced accuracy on both test sources. Their decisions stay in a single class: one class receives full recall and the other receives none, averaging to fifty percent. Both checkpoints were selected at update one hundred, although their training continued to one thousand. Joint scratch reaches fifty five point five seven percent on C I C and fifty percent on U N S W. Completed training therefore includes models that failed to discriminate usefully.

On screen: Every scratch arm completed 1,000 updates · Single-source selections made constant-class decisions

Scratch training source — Selected update — CIC test — UNSW test

CIC only — 100 — 50.00% — 50.00%

UNSW only — 100 — 50.00% — 50.00%

Joint — 1,000 — 55.57% — 50.00%

Test scores are group weighted balanced accuracy.

Evidence: [packet-results](#), [packet-training](#), [packet-report](#)

00:38:22 — Nine simple controls challenge neural superiority

Nine controls help interpret the neural results. Each source choice uses three methods: a majority classifier, which always predicts the class with more weighted training group target mass; a histogram model, which counts byte frequencies; and a metadata model, which uses recorded properties. Metadata stays outside the neural model's inputs. The U N S W only metadata control reaches ninety-nine point thirty-four percent group balanced accuracy. It exceeds every native model tested on that source. These recorded properties strongly distinguish this source's labels. The methods use different inputs, and their results do not establish that neural models are uniformly superior.

On screen: 3 source choices × majority, histogram, and metadata · UNSW metadata: 99.34% group balanced accuracy there · Different input features are part of the comparison

Evidence: [packet-controls](#), [packet-report](#)

00:39:08 — Class agreement and numerical agreement are distinct

The original full inference check reproduced every class across twenty five thousand nine hundred thirty predictions, while two rows failed the strict logit comparison. A separate fresh client run validated twenty thousand C I C rows and predicted one hundred twenty eight. All those classes agreed, but seventy six rows failed the strict numerical comparison. Both checks used relative tolerance one ten thousandth and absolute tolerance one millionth. A tolerance defines the allowed numerical difference. The fresh maximum difference is preserved exactly on screen. Agreement on a class does not make the underlying scores numerically identical.

On screen: Original full check and fresh capped check remain separate · Failures count compared prediction rows

Check scope — Validated / predicted — Classes agree — Strict logit failures

Original full check — 25,930 / 25,930 — 25,930 — 2 rows

Fresh client check — 20,000 / 128 — 128 — 76 rows

Fresh max $|\Delta \text{logit}| = 0.006159305572509766$; $\text{rtol}=1\text{e-}4$, $\text{atol}=1\text{e-}6$.

Evidence: [packet-results](#), [client-audit](#), [packet-prediction](#), [prediction-agreement](#)

00:39:51 — State what the evidence still cannot establish

Report these results with the conditions that make them interpretable. The study used one model seed, capped training pools, and a fixed update budget. The earlier data seen by the released pretrained weights is unknown. Exact payload grouping also cannot prove independence between network captures. The selected C I C test has no PortScan rows and only four distributed denial of service rows, so broad subtype claims would exceed its support. Finally, the paper's ninety seven point five zero percent flow accuracy measures a different task and metric. These packet results establish neither its reproduction nor an improvement over it.

On screen: One model seed, capped pools, fixed update budget · Earlier pretrained exposure remains unknown · CIC test: 0 PortScan rows; only 4 DDoS rows

Evidence: [packet-protocol](#), [packet-report](#), [packet-results](#)

00:40:34 — Your turn · 10 seconds

Practice for 10 seconds: State one measured success, one weak result, and two remaining limits.

On screen: State one measured success, one weak result, and two remaining limits.

Evidence: [packet-report](#), [packet-results](#), [client-audit](#)

9. Show the packet demo and explain a mistake

00:40:44 — Start with the right demonstration

Now demonstrate the system to someone else. Open the packet viewer from the project home page, or generate the same page from the client repository. The viewer displays predictions already produced by the selected joint model. It does not train a model or inspect your current network. Say that boundary before clicking anything. This makes the demonstration useful: you can show real model decisions and their limitations without pretending a browser is an operational security system. The next steps follow the same saved packet evidence used in the results chapter.

On screen: Recorded joint-model predictions · Both packet sources · No live network connection

Evidence: [project-guide](#), [packet-demo](#)

00:41:18 — Choose the source you are explaining

The source selector switches between the C I C and U N S W test groups. The C I C view contains twenty thousand distinct payload groups; the U N S W view contains five thousand nine hundred thirty. These are selected test groups, not the original total CSV row counts. The headline cards always describe the complete selected source test set. Table filters only choose which saved rows appear on screen. Explain which source is selected before reading any metric, because the model performs differently on the two sources.

On screen: CIC: 20,000 test groups · UNSW: 5,930 test groups · Headline metrics use the complete source

Evidence: [packet-demo](#), [packet-results](#), [packet-capture](#)

00:41:53 — One decision has several parts

Open one prediction's detail panel. The long hash identifies the exact payload group without displaying the original bytes. The model produced two logits, which are numerical scores before conversion into probabilities. The larger logit chooses benign or attack. The viewer also shows an uncalibrated probability for that choice. A large value means the model favors one class under its learned scoring rule. It does not prove that the packet is dangerous, and it is not a measured probability that an analyst will confirm an attack on a customer network.

On screen: Payload hash identifies the byte group · Two logits determine the class · Scores are uncalibrated

Evidence: [packet-demo](#), [packet-prediction](#)

00:42:31 — Keep the supplied labels visible

The same payload can appear in multiple rows with different supplied labels. The viewer therefore keeps the benign and attack row counts for each group. It does not silently choose a convenient single label. For example, a hypothetical group with nine benign labels and one attack label still contains both kinds of evidence. A benign prediction disagrees with the one attack-labeled row. That example explains the interface; it is not an additional measured result. The original labels have not been independently adjudicated, so disagreements can reveal model errors, label problems, or both.

On screen: Original benign and attack row counts · Conflicting labels remain visible · A group is not automatically one true label

Evidence: [packet-demo](#), [packet-data](#)

00:43:10 — Demonstrate an error without hiding it

Select the disagreement filter and inspect a result. The point is to show how a model can be wrong, including when its own score is high. Read the predicted class and the original label counts together. If you choose the conflicting-label filter, explain why disagreement is unavoidable for some rows sharing identical model inputs. Do not call every displayed disagreement a proven malicious packet that was missed. This is a labeled research dataset with a specific evaluation rule. A credible presentation explains that rule and shows an actual limitation alongside the positive results.

On screen: Actual saved error: Attack, 94.57% uncalibrated score · Supplied labels: 1 benign, 0 attack · A prediction is not an independently verified incident

Evidence: [packet-demo](#), [packet-report](#), [packet-capture](#)

00:43:50 — Fresh prediction is a separate execution step

Fresh inference happens in the Python packet predictor using the checked GPU environment and a trained joint checkpoint. It accepts a correctly formatted unlabeled CSV and writes predictions plus a receipt. The label column must be removed in a separate input copy because inference must not receive the answers. The new receipt distinguishes validated rows from predicted rows when a prediction cap is used. Opening the viewer and running the predictor demonstrate different parts of this workflow. The viewer explains retained results; the predictor performs a new model computation.

On screen: Supply an unlabeled packet CSV · Load the selected joint checkpoint · Retain predictions and the receipt

Evidence: [packet-prediction](#), [client-setup](#), [client-audit](#)

00:44:29 — What a working IDS would add

An operational intrusion detection system would connect additional components around this classifier. It would capture traffic, extract bytes with the same meaning as the training inputs, queue examples, run inference, apply a reviewed alert policy, and store events for investigation. Monitoring would need to report dropped packets and processing failures. A blocking system would need further response and recovery decisions. None of those components appears merely because a prediction says attack. The working demonstration here is a recorded packet viewer plus an actual CSV inference program. The missing operational components remain explicit engineering work.

On screen: Capture and compatible byte extraction · Inference queues and alert policy · Monitoring, review and response

Evidence: [project-guide](#), [hardware](#)

00:45:12 — Your turn: explain it aloud

Practice for 10 seconds: Explain what the browser demonstrates and what requires the GPU predictor.

On screen: Explain what the browser demonstrates and what requires the GPU predictor.

Evidence: [project-guide](#)

10. Set up, run and test the same packet workflow

00:45:22 — Start from the client repository

For a person who needs to run the delivered project, start from the client repository. Clone it, enter its root directory, and read the setup document there. The source repository remains useful for development and the longer explanations, but the client contains the selected execution files and their tests. Use a new output directory for each experiment. That preserves the previous run, its errors, and its identity records.

On screen: Clone netmambapplus-client · Read SETUP.md from its root · Keep output directories separate

```
git clone https://github.com/buffbeefalo/netmambapplus-client.git
cd netmambapplus-client
python -m unittest discover -s tests -v
python tools/verify_client.py
```

Evidence: [client-setup](#), [client-readme](#)

00:45:50 — Check the portable parts first

The first commands need Git and a supported Python interpreter, without a GPU or the private CSV files. Run the tests, verify the delivered inventory and evidence, and generate the packet viewer. On the recorded local Linux check, the client discovered one hundred eighty-five tests. One hundred forty-nine passed and thirty-six optional tests skipped. A skip is not a successful native GPU test. Windows had one additional skip in its hosted jobs. These counts belong to the recorded delivery revision. Later software revisions can change the suite, so retain the run output.

On screen: Python 3.10 or 3.12 · Tests and saved-evidence verification · Offline packet viewer

```
python -m unittest discover -s tests -v
python tools/verify_client.py
python tools/render_packet_demo.py \
  --output runs/packet-demo-new
```

Evidence: [client-audit](#), [client-setup](#)

00:46:28 — Prepare the native execution environment

Actual neural execution needs more than the portable test suite. The measured environment used Linux on A R M sixty-four, an NVIDIA G B ten, Python three point twelve, and Torch two point nine point one with CUDA thirteen. The compiler toolkit and custom operators also matter. A driver displaying a CUDA version does not mean the compiler is installed. Follow the exact package and environment commands in the setup document. Other physical GPUs need their own build, numerical and packet-model checks. Hosted Windows and macOS checks do not prove native neural execution there.

On screen: Measured: Linux ARM64 and GB10 · Torch plus CUDA toolkit and custom operators · Other GPUs need their own checks

Evidence: [client-setup](#), [support](#), [native-gate](#)

00:47:07 — Acquire only the required initialization

The setup obtains the pinned authors' source and checks its identity. The default asset fetch now acquires only the registered pretrained initialization, with its expected size and hash. It does not download a flow dataset. It also does not supply your two private CSVs or a finished binary packet classifier. You provide the CSVs locally and training produces the selected packet checkpoint. This explains why cloning the repository is enough to inspect evidence but not enough to run a trained application immediately. The asset notice records attribution and unresolved inherited terms.

On screen: Verify the pinned authors' source · Fetch pretrained weights separately · Supply both CSVs locally

```
python repro.py fetch
python tools/fetch_assets.py --output assets
python tools/build_cuda.py \
  --build-root runs/cuda-build-new
python tools/check_gpu_runtime.py \
  --output runs/cuda-ops-new.json
```

Evidence: [client-setup](#), [packet-assets](#), [client-notice](#)

00:47:48 — Prepare both inputs and run the native gate

Run packet preparation with both input paths and a fresh destination. It validates every row and byte, creates the global groups and partitions, and saves arrays with a manifest. For an exact repeat of the uploaded-file study, check the registered source identities in that manifest. The packet runtime gate then uses prepared bytes from both sources to exercise transfer, input shapes, gradients, an optimizer update and checkpoint reload. All seventeen cases passed on the recorded G B ten checks. The loss targets in that gate are synthetic, so its pass is functional evidence, not benchmark accuracy.

On screen: Validate and group both CSVs · Check registered source identities · Require native tests with no skips

Evidence: [client-setup](#), [native-gate](#), [packet-data](#)

00:48:28 — Freeze, train, select and predict

The setup gives the full commands to freeze a protocol, fit the controls, train the six comparison arms and evaluate the frozen results. Protocol freezing binds the data, implementation and declared budget. Read the completed receipts rather than assuming a requested number of updates finished. The selected joint pretrained checkpoint is listed in the frozen checkpoint file. Use that path for the client's prediction command. It belongs to this same packet workflow; the other arms are comparisons that help explain the chosen model. Preserve any failure report while investigating a new run.

On screen: Freeze the protocol before the study · Complete the six comparison arms · Use the joint pretrained checkpoint

Evidence: [client-setup](#), [packet-protocol](#), [packet-training](#), [packet-prediction](#)

00:49:07 — Read a failed check before retrying

If a check fails, identify whether the failure concerns input format, source identity, a missing operator, incomplete execution, or numerical agreement. Those are different problems with different fixes. Keep the failed report and use a new output path for a corrected execution. Do not loosen a scientific tolerance just to make a presentation green. The current record deliberately retains score-comparison failures even when predicted classes match. That makes the project more useful to the next engineer, who can see what was tested, what succeeded, and which remaining issue still requires investigation.

On screen: Find the failing stage and receipt · Correct the cause in a new run · Never turn a skip into a pass

Evidence: [client-setup](#), [client-audit](#), [packet-report](#)

00:49:47 — Your turn: explain it aloud

Practice for 10 seconds: Name the evidence needed before claiming a new machine runs the native model.

On screen: Name the evidence needed before claiming a new machine runs the native model.

Evidence: [project-guide](#)

11. Understand both repositories and their updates

00:49:57 — Two repos, one packet application

There are two repositories because their audiences need different amounts of material. The reproduction repository is the development and research source. It includes the full explanations, historical work, presentation builders and teaching materials. The client repository is a concise executable delivery with the packet code, selected dependencies, tests and evidence. They do not offer competing model workflows. Both lead to the same joint packet adaptation using the two CSVs. Tell a presenter to begin with the reproduction guide, and tell the engineer receiving the executable handoff to begin with the client setup.

On screen: Reproduction: develop, investigate and learn · Client: run the concise delivery · Shared code follows one workflow

Evidence: [project-guide](#), [client-readme](#), [client-maintenance](#)

00:50:37 — The model and data files have distinct jobs

The top-level packet data module validates inputs, maps labels, groups identical payloads and constructs prepared artifacts. The packet model module connects the supplied bytes to the original encoder, handles compatible pretrained transfer and defines the binary classifier. The packet study module supplies shared experiment and evidence functions. Keeping those responsibilities distinct makes the project easier to check. If an input row is rejected, investigate preparation or inference parsing. If checkpoint shapes disagree, investigate the model contract.

On screen: `packet_data.py` prepares inputs · `packet_model.py` adapts the encoder · `packet_study.py` supports the experiment

Evidence: [packet-data](#), [packet-model](#), [packet-study](#)

00:51:16 — Tools, configs and requirements support execution

The tools directory contains the programs for source and asset acquisition, native builds, runtime checks, protocol freezing, controls, training, prediction, evidence review and demo rendering. Configurations record settings and asset identities. Requirements record the measured dependency set. The shared reproduction helper remains an internal dependency for upstream identity and build operations, even in the client. Its historical orchestration does not create another supported client workflow. Use the documented packet setup rather than exploring old commands as if they were additional product choices. Each tool should have an identifiable input, output and checkable completion condition.

On screen: Tools run specific workflow stages · Configs record settings and identities · Requirements define the measured environment

Evidence: [client-setup](#), [client-maintenance](#), [walkthrough](#)

00:52:03 — Evidence and local runs are different objects

The public evidence directories contain saved predictions, metrics, protocols, receipts and hashes that support the published claims. Local run directories contain machine-specific execution outputs and separately acquired assets. Raw CSVs, the authors' checkout and trained weights are not automatically part of a public source clone. In the full reproduction repository, older flow experiments and calibration records remain historical research. They help explain earlier work but are not the present client setup. The file walkthrough explains individual paths and downloads so you can tell an executable component from a result, a source document or a generated artifact.

On screen: Public receipts and saved predictions · Local CSVs, prepared arrays and weights · Historical research stays labeled

Evidence: [project-guide](#), [client-notice](#), [walkthrough](#)

00:52:46 — The client manifest identifies the delivery

At the recorded packet-only delivery, the client contained one hundred twenty-four files, including its manifest. Seventy-five were packet-study evidence files. The manifest records the selected source revision, the origin of each delivered file, its size and its hash. A hash answers whether the bytes match a recorded identity. It does not prove that the scientific claim is true. The evidence reviewer adds arithmetic and consistency checks. Together these mechanisms let the recipient establish which software and results they received, while still reading the limits of the actual experiment.

On screen: 124 files at the recorded delivery · 75 packet-study evidence files · Exact source paths and SHA-256 hashes

Evidence: [client-maintenance](#), [client-audit](#)

00:53:25 — Updates flow from source to client

Development changes belong in the reproduction repository, including the templates used to generate client instructions. After successful source checks, the publication workflow selects the tested source revision and builds an explicit client export. It rejects unexpected client edits and checks the staged contents before publication. A separate publishing step updates the client, whose own platform checks then run. The packet-only update actually exercised managed removal of older flow files. If you manually change the client and it diverges, synchronization stops for review instead of silently overwriting the difference. This protects the handoff's identity.

On screen: Successful source CI gates publication · Export checks additions and removals · Client CI checks the delivered revision

Reproduction: Develop code + templates

Checks / export: Test selected source

Client: Publish shared workflow

Source updates produce a verified client export; teaching media stays in reproduction.

Evidence: [client-maintenance](#), [client-audit](#)

00:54:08 — Find the right download and its scope

This video, its captions, transcript, slide deck and handbook come from one reviewed lesson source. Their records bind the rendered assets to that source. The handbook includes the repository file guide for details that would be tedious to read aloud. The shorter packet presentation remains a concise summary of the same study. Teaching media belongs in the reproduction repository and is excluded from the client export. An explanation-only source edit may therefore leave the client's source marker unchanged. A selected code or template change follows the verified update process described in the previous scene.

On screen: Video, captions and transcript share a source · Slides and handbook follow the lesson · Client excludes teaching media

Evidence: [project-guide](#), [client-maintenance](#), [walkthrough](#)

00:54:47 — Your turn: explain it aloud

Practice for 10 seconds: Explain which repo you would give a presenter and which you would give an engineer.

On screen: Explain which repo you would give a presenter and which you would give an engineer.

Evidence: [project-guide](#)

12. Hardware roadmap, remaining work and your handoff

00:54:57 — Start with the complete processing path

Now consider a future deployment. Draw the entire path from observed network traffic to a usable security event. Capturing bytes, identifying the correct representation, buffering work, transferring data, running the classifier and recording an alert all take time and can fail. A model-only timing would not include those stages. This packet handoff does not establish operational throughput, end-to-end latency or an alert guarantee. The first hardware task is to define that complete path and its requirements. Only then can an engineering team decide which stages should run on which device.

On screen: Capture and extract compatible bytes · Queue, infer and produce an event · Measure the whole path

Evidence: [hardware](#), [project-guide](#)

00:55:35 — What a SmartNIC could contribute

A network interface card moves network data into and out of a system. A Smart N I C adds programmable processing near that network boundary. It could contribute capture, filtering, buffering or compatible preprocessing, depending on the specific hardware and runtime. Those are proposed responsibilities, not capabilities demonstrated by this repository. In particular, putting a model file on a network card does not establish correct packet extraction or reliable inference. The representation must match the packet bytes the model expects, and the complete system must report overload, dropped observations and processing failures.

On screen: Network-facing capture and buffering · Possible preprocessing and queue handling · No completed target deployment

Evidence: [hardware](#), [project-guide](#)

00:56:17 — What an NPU port would require

A neural processing unit accelerates computations supported by its own runtime. Net Mamba Plus currently relies on custom CUDA operations in the checked native implementation. A future port needs a chosen target, supported equivalents for the relevant operations, compatible layouts and an explicit precision policy. Saving parameter tensors is different from exporting an executable computation graph. Both may be necessary, and neither alone proves the port works. Begin by comparing the target's model outputs with the retained GPU reference. Preserve differences and determine whether they change decisions under the intended operating conditions.

On screen: A target runtime and supported operators · Equivalent tensor layouts and precision · Compare outputs against the GPU reference

Evidence: [hardware](#), [packet-model](#), [project-guide](#)

00:56:59 — Optimization needs a new measurement plan

Optimizations should answer a stated deployment need. Larger batches may improve throughput while increasing how long a packet waits. Lower precision can save memory or computation while changing scores and possibly classifications. Faster arithmetic does not automatically make the detector more useful. A target evaluation should include accuracy on suitable data, latency distributions, throughput, memory use and dropped work. The existing strict score-comparison failures already show why class agreement and identical numerical outputs must be reported separately. A port needs its own evidence instead of borrowing a successful check from another machine.

On screen: Batching changes latency and throughput · Lower precision can change predictions · Compare accuracy, failures and resource use

Evidence: [hardware](#), [client-audit](#), [packet-report](#)

00:57:44 — The major research gaps remain visible

The current results come from one model seed, capped groups and these supplied data sources. Pretrained exposure to related data is not fully known. Exact duplicate grouping does not prove independent captures, and sparse attack-subtype support limits some conclusions. The project has not established reliable confidence on customer traffic, calibrated packet probabilities, unknown-attack coverage, operational alert thresholds or live capture. It also has not run on an N P U or Smart N I C. These are concrete next tests and engineering tasks. They do not erase the completed training, but they constrain what it proves.

On screen: Independent customer traffic and more seeds · Unknown attacks and reliable confidence · Live extraction, alerts and target execution

Evidence: [packet-report](#), [hardware](#), [project-guide](#)

00:58:25 — Say precisely what has been built

Here is a defensible explanation. We adapted the original Net Mamba Plus encoder to packet payloads from both supplied CSV files. We validated the inputs, grouped exact duplicate payloads across sources, trained the declared comparison models and selected checkpoints using validation data. We measured the joint model on held-out packet groups and compared it with simpler controls. We executed saved-checkpoint inference and retained both class agreement and numerical differences. The client repository delivers the executable workflow; the reproduction repository explains the code, evidence and limitations. This is a tested research implementation, with further work required for deployment.

On screen: Both CSVs really enter native training · The joint classifier predicts packet labels · The records include weak results and failures

Evidence: [project-guide](#), [packet-report](#), [client-audit](#)

00:59:10 — Teach the complete story to the next person

For your final handoff, explain the project without reading a list of filenames. Begin with the two CSVs and the paper's idea. Follow one packet through preparation, the adapted encoder, training and inference. State what the joint results measure and mention the stronger metadata control on U N S W. Explain what each repository is for and where to find the setup and audit. Finish with the difference between the recorded demonstration and an operational intrusion detector. Take the upcoming pause to practice that story aloud. Use the transcript afterward to check anything you could not explain clearly.

On screen: Inputs, model, training and predictions · Both repos, tests and measured results · Limits and the next deployment experiment

Evidence: [project-guide](#), [packet-report](#), [client-maintenance](#)

00:59:50 — Your turn: explain it aloud

Practice for 10 seconds: Explain the complete CSV workflow, both repos, one measured result and one remaining limitation.

On screen: Explain the complete CSV workflow, both repos, one measured result and one remaining limitation.

Evidence: [project-guide](#)

Appendix: every file and every download

The following reviewed file guide is included so the entire handoff remains readable offline. File links point to the repository; downloaded model inputs and weights are described separately.

Every file in the NetMamba+ repository

This companion explains every current repository file, its inputs and outputs, and when to use it. The [packet study](#) supplies the single supported native NetMamba+ training and prediction workflow for both uploaded CSVs. Its preparation, frozen protocol, six completed native comparisons and saved test predictions are available as separate evidence. The original six-class flow measurements keep their original meaning.

The [current v5 video](#) explains both CSVs, the paper, the native packet adaptation, both repositories, actual tests, setup, predictions and remaining deployment work. Its [source](#), [downloads](#) and [individual frame entries](#) follow one lesson. The slide images match the video and the handbook includes this guide as searchable text.

The supported client workflow now uses both CSVs and the selected joint pretrained packet model. Earlier flow/calibration code and media in this full inventory are historical research. For a first reading before a client handoff, use [Understand and explain the client project](#). It explains both repository editions and the complete process, then shows how the client's manifest maps any delivered file back to the individual entries in this walkthrough.

The preserved historical [v4](#) course covers the original flow experiment and confidence calibration at immutable commit 4eba1da479238a10ff29097b2e0a27d98a79f063, whose complete tree contains **494 files**. Its [source](#), [checks](#) and [downloads](#) and [96-scene frame map](#) remain individually documented here. That video, PDF handbook and PowerPoint do not explain the later packet study. Historical verification checks the preserved edition; a separate current-guide check requires an individual entry for every tracked and nonignored file today. Neither check grants new video coverage or certifies every prose claim.

The original audited customer package at commit 17b4aaebcf9327ae9967ca45ddfaf16325766993 is historical. Its frozen ZIP, eight-page briefing, 16-slide deck and matching script retain their original contents and do not include later calibration or packet training. The current [customer index](#), [quickstart](#) and [acceptance map](#) explain the supported packet delivery; historical entries retain their earlier scope. The 54-test count and then-untested Windows/macOS statements in the archived ZIP belong to that older snapshot; current platform evidence is in the [support matrix](#).

V3 also stays unchanged: its source, frames, video/documents and receipts are verified against immutable Git snapshot 71290440e917105ffa094402d94e24d7e29a3549. The current verifier reads original Git blobs as data without executing old code; this shared guide can evolve. The [v3 section](#) retains its historical scene map. V2 remains separate history. There is one active video watch route, while the archived interactive /course/ route stays excluded from deployment.

“Tracked” means included in Git. Nonignored additions also need explanation and review. Downloaded upstream code, raw data, local weights, speech/build caches and Git internals remain outside this inventory; their roles appear near the end.

You can read this one file from top to bottom, use the links below, or search it for a filename. To keep a local copy, use GitHub's **Raw / Download raw file** action. A Markdown reader displays its formatting; an ordinary text editor can read the same file. Follow source links only when you want

the underlying code or evidence.

Jump to: [How it fits together](#) · [Vocabulary](#) · [Main code](#) · [GitHub automation](#) · [Settings](#) · [Dependencies](#) · [Tools](#) · [Tests](#) · [Foundational docs](#) · [Customer documents](#) · [Demo](#) · [Figures](#) · [Original evidence](#) · [Later audit](#) · [Council/research](#) · [V2 history](#) · [Current packet study](#) · [Historical v4 and calibration](#) · [V4 frame map](#) · [V3 history](#) · [Local files](#) · [Every release download](#).

First understand what the repository is

The current application receives one packet's 1,500 stored payload bytes, processes 378 tokens through the original four-block encoder and predicts benign/attack. Both supplied CSVs contribute selected groups to its joint training. The client repo receives the executable packet package; the reproduction repo also preserves research history and teaching media. The flow description below explains those archived files, not a second client setup.

Think of it as a small research laboratory with four parts:

1. **Execution code** gets the correct inputs into the authors' model and runs training or prediction.
2. **Checks** test the execution code, compare GPU calculations and verify the recorded evidence.
3. **Lab records** preserve what actually ran: settings, logs, model identities, predictions and failures.
4. **Teaching materials** turn those records into the slides, briefing, guide and browser demo.

The original neural model is **NetMamba+**, a classifier of network flows. A flow is a group of related packets. It reads byte content, packet sizes and time gaps, then assigns one of six dataset categories. Two categories are attacks; four describe IoT device/power activity. The current packet adaptation uses the same native encoder code with a two-class head and one packet's stored bytes. Neither route calls an AI chat service to make traffic predictions.

The uploaded CICIDS2017 and UNSW CSVs contain individual packet records. They lack established flow identity and ordering, so the original flow experiment uses the authors' separate CICIoT2022 release. The later packet study explicitly accepts that limitation: it validates all 1,500 stored byte slots and learns supplied benign/attack labels without reconstructing flows. The original model and flow tensor loader are fetched at a pinned commit; our scripts organize their execution and define the separate packet input contract.

In the tested flow preset, one flow contributes the first five packets with up to 320 stored bytes each, plus the first 20 packet sizes and 20 time gaps. The original loader pads/truncates and normalizes these values. The model turns them into 443 input units called tokens, processes them through four Mamba blocks and combines three summaries into six outputs. A Mamba block scans a sequence while updating an internal numerical state; that state is not an implemented live network-connection cache. The six categories are Flood, RTSP Brute Force, Audio, Other, Cameras and Home Automation under the release's recorded class names. The [packet workflow](#) instead has 378 tokens and no observed size or timing sequence.

Understand the preserved flow experiment

This diagram traces the original six-class flow experiment. Its files and recorded results remain separate from the packet workflow below.

flowchart TD

A[configs: source identity, settings, asset hashes] --> B[repro.py and fetch_assets.py]

```

acquire and validate]
  B --> C[build_cuda.py: build the pinned GPU dependencies]
  C --> D[check_gpu_runtime.py and check_model_runtime.py: operations and complete
classifier]
  D --> E[repro.py: invoke original training code]
  E --> F[Local selected classifier checkpoint]
  F --> G[evaluate.py: score labeled test flows]
  F --> H[predict.py: classify flows without labels]
  F --> I[replay.py: record labeled predictions and metrics]
  G --> J[review_experiments.py: inspect runs and collect results]
  I --> J
  J --> K[publish_evidence.py: copy reviewed records locally]
  K --> L[build_customer_package.py: documents and teaching guide]
  I --> M[Browser replay and recording]
  L --> N[verify_package.py and CPU tests]
  M --> N

```

The short pretraining check is a separate branch of this experiment. The three full fine-tuning runs started from the authors' released pretrained checkpoint, **not from the checkpoint produced by our short check**. That distinction matters when explaining what was reproduced.

Opening the finished demo only reads recorded predictions. It does not run these training steps again. Running the CPU tests also does not train a model. New GPU experiments use the [support matrix](#) for the current environment and the original source/assets and execution sequence described in the runbook. `build_gb10.py` retains the historical GB10 build route; `build_cuda.py` checks a selected compiler-supported NVIDIA target while keeping the pinned model unchanged.

Vocabulary for reading the files

Term	Meaning in this project
Python / .py	Executable program text. Run it with Python; inspect it to understand the implementation.
Markdown / .md	Readable document text with headings, links and tables. GitHub displays it as a formatted page.
JSON / .json	Structured data made of named fields, lists and values. It can describe settings, results or predictions; the extension alone does not tell you which.
YAML / .yaml	Structured settings used here to tell GitHub which automated jobs to run.
Manifest	A run's lab notebook: what was requested, which files and model were used, what environment ran it, and whether it completed.
Checkpoint	A saved state of a trained neural network. Native training checkpoints may also contain optimizer state and settings.
Model-only export	A local checkpoint containing the selected model tensors without the training optimizer and executable settings object. It is still a PyTorch model file.
Graph export	A different operation: capture the model's computations into a form another execution system might consume. The tested graph-export path failed. Successful model-only export does not establish graph-export support.
Provenance	The recorded origin and identity of a file or model, including which class index means which category.
Hash / SHA-256	A content fingerprint. Matching hashes establish matching file bytes; they do not prove scientific correctness or data rights.
Seed	A chosen starting value for random operations. The original flow study used seeds 0, 1 and 2 on one split. The packet protocol fixes split seed 17 and model/sampling seed 0 for six comparison arms. These are not independent datasets or repeated packet-study seeds.
Epoch / update	An epoch is a pass through the training loader. An optimizer update changes model parameters after a batch. The original flow runs each completed 120 epochs and 7,920 updates. The packet protocol requests 1,000 updates per arm with replacement sampling; that budget is not a full-file epoch or proof of completion.
Logits / scores	Raw model outputs: six for the original flow classifier, two for the packet classifier. Softmax produces normalized scores. The separate flow calibration study adds optional temperature scaling without changing the winning class. Packet scores are uncalibrated; neither route establishes customer-network attack probability.
Accuracy / F1	Accuracy is the fraction of correct predictions. F1 combines precision and recall. Weighted F1 accounts for each class's number of true examples; macro F1 gives each class equal weight.
Confusion matrix	A table of true versus predicted categories. Here rows are true classes and columns are predicted classes. It shows which mistakes produced an aggregate score.
Fixture / regression test	A controlled example used to test code behavior, such as rejecting a mismatched model. These CPU examples do not establish neural-model accuracy.
Runtime / environment	The actual Python, packages, compiler, driver and hardware used to execute the model.
CPU / GPU / GB10	The CPU runs general-purpose programs. A GPU performs many numerical operations in parallel. NVIDIA GB10 is the hardware profile used for the measured model execution here. The basic package tests need only a CPU.

Term	Meaning in this project
Torch / CUDA / Triton	PyTorch (imported as torch) supplies neural-network operations and training machinery. CUDA is NVIDIA's GPU computing platform. Triton compiles some specialized GPU calculations used by this software stack.
NIC / SmartNIC / DPU	A NIC connects a computer to a network. SmartNICs and data processing units can take on additional packet handling or computation. This project's proposed division of work is described in the hardware roadmap; it has not been deployed there.
NPU / IDS	An NPU is a neural processing unit designed to accelerate supported neural computations. An IDS is an intrusion detection system that observes traffic and raises security alerts. A traffic classifier is one component of a possible IDS, not the whole capture-to-alert system.

1. Files at the repository root

Course links: [main execution path](#), especially [the coordinator](#), and [root/settings/dependencies](#).

These are the main entry points and Git settings. Links in every inventory table lead to the actual file. A table's heading gives the folder; file names within that table are relative to it.

File	What it does and when you would open it
.gitattributes	Tells Git to preserve exact bytes for captured records and documents, and to treat PDFs, PowerPoints, images and video as binary files. This protects hash comparisons against automatic text conversions.
.gitignore	Lists local outputs that Git should normally exclude: downloaded source, datasets, checkpoints, run directories, virtual environments and secret-setting files. It reduces accidental publication; it is not a permission system.
README.md	The project's front door. Explains the measured result, links the customer material, gives basic commands and states the scientific and deployment limits. Read this before running the code.
repro.py	The main experiment coordinator. Its fetch, validate, pretrain, finetune and evaluate commands verify source/data, resolve native settings, create run manifests and launch the authors' code. It also supplies shared validation, hashing, output protection and provenance helpers to the other scripts.
evaluate.py	Reloads a saved classifier strictly and scores labeled test flows using the authors' loader and evaluation engine. It checks category meanings and writes the returned metrics. It does not train the classifier or select it using test accuracy.
predict.py	Reads native flows and a provenance-bound classifier, removes loader targets before forward, and returns original logits/scores/categories without unlabeled accuracy. Optional --calibration adds checked scaled scores; --abstain-threshold adds accept/defer for the unchanged category. Use the calibration guide for compatibility and policy limits.
replay.py	Runs labeled test inference, records each prediction and independently recomputes the metrics. It also contains the browser replay's HTML, CSS and JavaScript template. The generated page displays those saved results without executing the neural model in the browser.

The main pattern is **shared preparation, specialized execution**. repro.py checks identity and settings once; evaluation, prediction and replay reuse that preparation. The authors' loader remains responsible for tensor transformations, avoiding a second interpretation of packet bytes, padding and normalization.

2. .github/workflows/: automated GitHub jobs

Course links: [six portable CI combinations and tests](#), [workflows and evidence](#).

File	What it does and when you would open it
ci.yml	Runs six CPU jobs: Ubuntu 24.04, Windows 2022 and macOS 14 with Python 3.10/3.12. Runs tests, recomputes saved flow calibration and packet-study evidence, checks every current guide entry, verifies the pinned historical v3/v4 editions and checks page freshness. Full history supplies both immutable course trees. A green job does not repeat GPU training or prove a CPU model backend.
pages.yml	On a push to main or manual dispatch, runs tests and package verification, then publishes docs/customer/demo to GitHub Pages. The workflow stages that folder and excludes the retired course/ directory before publishing the replay, video, learning guide and meeting backups. It does not deploy a live IDS.

3. configs/: recipes and input identities

Course links: [the executable configuration](#) and [native input transformations](#).

File	What it does and when you would open it
assets.json	The acquisition inventory: expected download locations, byte sizes and hashes for three native flow splits and the released pretrained checkpoint, plus the exact class metadata to generate. It also records the unresolved inherited-asset rights. It contains instructions for acquisition, not the raw flows or weights.
ciciot2022.json	The main source-based recipe. Pins the authors' commit and core source hashes, six classes, byte/size/interval input settings, batch size 128, 120-epoch fine-tuning and learning-rate settings. Its provenance fields explain where settings came from and how they differ from the paper. Its longer pretraining preset is a recipe, not evidence that we executed it.
ciciot2022-pretrain-smoke.json	The short functional-pretraining recipe. Requests 100 steps on CICIOT2022 training data and frequent checkpoint saves. The original loop rounds to complete epochs, so the actual check completed two epochs and 132 updates. "Smoke" means a limited test that the training machinery works.

Changing a configuration changes an experiment. A configuration by itself does not prove that the requested run completed; find the corresponding manifest and logs.

4. requirements/: packages for different activities

Course links: [dependency groups](#) and [native model prerequisites](#).

File	What it does and when you would open it
gb10.txt	Pinned Python packages for the tested GB10 profile. The runbook installs the selected Torch/CUDA stack first and builds the custom extensions separately. This file alone is not a complete machine setup or universal environment lock.
browser.txt	Pins Playwright, the browser-automation library used to test and record the demo and guide. Chromium is installed separately. None of this is needed merely to open the finished HTML in a normal browser.

File	What it does and when you would open it
<code>presentation.txt</code>	Pins document/chart libraries such as python-pptx, ReportLab and Matplotlib. These build the editable deck, briefing and figures. External rendering tools, including LibreOffice and pdfinfo, are separate runbook requirements.

5. tools/: programs supporting the experiment

Course links: [acquire/build/measure tools](#), [document and video tools](#) and [GPU execution checks](#).

Some tools need a GPU, some need a real browser, and some run with ordinary Python. Start with the current support matrix, then use the runbook for the relevant experiment commands; reading the report does not require every dependency.

File	What it does and when you would open it
<code>benchmark.py</code>	Measures saved-classifier GPU execution for batches of 1, 16 and 128 flows, with 20 warmups and 100 synchronized timing samples each. Records mean, median, p95 and throughput, plus single-versus-batch prediction agreement. Capture, preparation and transfers are excluded.
<code>build_customer_package.py</code>	Reads reviewed measurements and document sources, fills result placeholders, builds charts, writes the briefing/results/script, creates the PowerPoint, renders PDFs and builds the guide/answer map. It checks actual PDF page counts. Rebuilding can change output hashes even when wording stays the same.
<code>build_gb10.py</code>	Builds the pinned Mamba fork and causal-convolution dependency for the tested GB10/CUDA profile. Applies recorded architecture and compiler-compatibility changes in separate build copies, then checks installed Mamba Python files against upstream. Build success is followed by numerical checks.
<code>build_course.py</code>	Renders the archived 30-minute interactive HTML course from its JSON source and saved evidence, including the real prediction, answer keys, styles and quiz controls. Uses only Python's standard library. Its --check mode verifies the immutable historical edition, including exact expected HTML and protected source/receipt bytes, without writing files or requiring current reference documents to stay frozen.
<code>build_learning_guide.py</code>	Generates the slide-by-slide HTML learning guide and Markdown question map from the same presentation source. It supplies the plain-language explanations, glossary and exact scripts so those outputs stay aligned with the deck. Uses the Python standard library.
<code>capture_runtime.py</code>	Records package versions, GPU, driver, compiler, source comparisons and compiled-extension hashes. It preserves dependency-check warnings and uses a bounded inventory rather than dumping the whole shell environment. Open its outputs to identify the machine/software conditions behind a run.
<code>check_gpu_runtime.py</code>	Compares optimized GPU operations with reference calculations, including forward outputs and backward gradients. Its seven checks cover causal convolution, selective scan, normalization and full Mamba blocks. Passing shows numerical agreement within the recorded tolerances for those cases.
<code>check_learning_guide.py</code>	Opens the guide in Chromium and checks all 16 titles/scripts, disclosures, internal anchors, keyboard skip navigation, errors and four viewport widths. Saves screenshots and a dated receipt. Its hosted mode also compares the live HTML bytes with the local reviewed guide.
<code>fetch_assets.py</code>	Downloads the research assets described in configs/assets.json, or generates the small metadata file, and verifies size/hash before making each target available. It preserves mismatched existing files and rejects bad downloads. It does not fetch the authors' Git checkout; repro.py fetch does that.

File	What it does and when you would open it
learning-guide.css	Controls the learning guide's typography, colors, spacing, navigation, responsive layout and print styling. The generator embeds this stylesheet into the finished HTML, allowing the page to work offline.
probe_export.py	First runs ordinary GPU inference, then tries one strict Torch graph-capture path with a single flow. Records either an exported graph or the actual exception. Our recorded attempt was unsupported at a custom causal-convolution operator; it did not test an NPU compiler.
profile_csvs.py	Uses Python and the DuckDB command-line tool to scan the uploaded packet CSVs, count rows/classes, summarize metadata and hash the files. It records missing flow context. It neither trains a packet classifier nor exhaustively validates every payload byte numerically.
publish_evidence.py	Copies a defined set of reviewed local run records and demo files into a new publication directory, preserving their bytes and writing an evidence index. Despite its name, it does not push to GitHub. It deliberately excludes raw data and model weights.
record_demo.py	Uses Chromium/Playwright to exercise the replay controls and filters, check desktop/mobile layout, and create screenshots plus a WebM video backup. It records actual browser behavior while displaying saved predictions.
render_replay.py	Rebuilds the replay display from an existing prediction JSON without running the GPU model. Recomputes the confusion matrix, preserves the prediction file byte for byte and records the new HTML's identity. Useful when changing presentation layout.
review_experiments.py	Inspects all three completed training runs: epoch histories, validation-selected checkpoints, optimizer counters, finite/changed weights and matching test results. Exercises a training batch on disposable model copies and makes tensor-identical model-only exports. Produces the combined results record; requires the runtime and local assets.
summarize_audit.py	Runs package verification and the actual CPU tests, then extracts compact facts from primary records for a reviewer or council. Its output is a dated evidence summary, not a new GPU experiment or automatic publication certificate.
verify_course.py	Defaults to the preserved 1,800-second interactive course at the immutable v4 snapshot, checking archived references, timing, questions, prediction, links and review status while rejecting current source/HTML/receipt drift. --current and explicit source/root calls retain strict supplied-data checks. Optional browser observations are separate; the retired route and pending human rehearsal stay explicit.
verify_package.py	Checks the delivered evidence and documents using ordinary Python: recomputes metrics from predictions, checks hashes and identities, compares fresh evaluations, checks guide/notes/question alignment and local links. Its checksum-refresh option is for reviewed intentional changes; refreshing hashes does not validate new claims.

An important distinction: a probe manifest can say succeeded because the probe completed and recorded its answer, while its metric file says `unsupported_in_tested_path`. Always read the specific result, not just the outer process status.

6. tests/: CPU regression tests

Course links: [what portable CI establishes](#) and [matching each check to its claim](#).

These use temporary examples and controlled substitutes for expensive components. Their purpose is to catch software mistakes and verify rejection behavior. The original audited suite contains 54 tests; 17 additional course tests brought the HTML-course snapshot to 71; seven video checks brought the first video snapshot to 78. Later portability, complete-model boundary and caption-anchor regressions are included in the current workflow. Use its commit-specific count; real GPU experiments have separate evidence.

File	What it does and when you would open it
test_assets.py	Tests asset acquisition: bad hashes never become published local targets, mismatched existing assets are preserved, and valid existing assets are reused without downloading again.
test_course.py	Preserves the original negative checks for altered results/predictions, budgets, topics, links, keys, stale HTML and false completion claims using archived-reference fixtures. Adds historical source/HTML/receipt drift, inventory, explicit-root source and during-check mutation tests; a real builder CLI regression verifies historical checking without rewriting the retired course. These checks validate software and preserved course evidence, not learning.
test_predict.py	CPU fixtures check native-flow adaptation, absent-label handling and unchanged original outputs for the same logits. Also reject incompatible/changed calibration artifacts and thresholds without calibration, and confirm targets never reach forward. Actual GPU behavior has separate records.
test_replay.py	Tests independent metrics, class support, weighted versus macro F1, malformed prediction inventories, stable softmax and HTML escaping. It helps prevent a misleading chart or executable text from entering the replay.
test_repro.py	The largest test collection. Exercises native-flow validation, source drift, report/output protection, concurrent run ownership, native parser settings, failure manifests, checkpoint provenance, strict loading and test-only evaluation access. Its mocked model interfaces are explicitly separate from numerical GPU tests.

7. docs/: foundational explanations

Course links: [the file guide](#), [paper reading](#) and [the native data contract](#).

File	What it does and when you would open it
audit.md	Historical sharing audit from 7 September, when the project had not completed GPU training. Preserves earlier defects, fixes, tests and an unratified council outcome. Read it as project history; the customer verification record gives the later results.
harness-reference.md	The precise reference for accepted native-flow fields, stage-specific data access, duplicate counting, strict evaluation and manifest meaning. Use this when implementing or troubleshooting an input file.
lesson.md	A teaching chapter explaining packets versus flows, tensor transformations, training stages, class provenance, experiment records and metric definitions. Includes four runnable CPU exercises.
repository-walkthrough.md	This current individual-file guide includes the packet study plus historical v4 and v3 scene maps. The preserved v4 handbook contains the guide revision from its 494-file edition. Every current file needs its own unique anchor and description; later guide text does not become part of old media.

8. docs/customer/: presentation and customer instructions

Course links: [choosing downloads](#), [measured results](#) and [hardware limits](#).

File	What it does and when you would open it
README.md	Current customer handoff index. Separates packet, flow/calibration and original archived presentation editions; links setup, seven answers, evidence and actual audit limits. The historical ZIP retains its original index.

File	What it does and when you would open it
NetMambaPlus-customer-briefing.pdf	The eight-page readable/printable briefing. Explains the work, inputs/outputs, training, actual results, demo, setup, original-repository differences and hardware roadmap.
NetMambaPlus-customer-slides.pdf	The 16-slide deck rendered into a fixed-layout PDF. Useful for presenting or sharing when PowerPoint rendering varies between computers.
NetMambaPlus-customer-slides.pptx	The editable 16-slide PowerPoint, including speaker notes. Use it for your presentation and read its notes to understand the intended explanation. The checked-in file is generated from the presentation source.
SHA256SUMS	A fingerprint list covering the customer and reviewed-research files, excluding the checksum file itself. The verifier uses it to detect changed or missing artifacts. It does not cover every source file in the repository.
acceptance.md	Current seven-question and requirements map for both model routes, calibration, tests, media and council outcomes. Distinguishes executed work from explained proposals and unimplemented deployment.
answers.md	Maps the seven requested customer questions to slide/script/guide numbers and briefing pages, then gives concise answers. Generated from the shared presentation source.
briefing-source.md	The editable briefing template. Contains prose, deliberate page breaks and measurement placeholders that the package builder fills. Edit this when changing the briefing's source explanation.
briefing.md	The generated briefing text after measured values are inserted. Read it on GitHub or compare it with the PDF. Direct edits can be overwritten by the builder.
course-source.json	Editable source for nine lessons, eight questions, answer explanations, the final teach-back rubric and exact teaching/practice times. Contains measured-value bindings, a real prediction row and original artifact fingerprints. It is teaching data, not model-training data.
course-verification.md	Explains how to open, download, rebuild and test the course. Records content review, timing assumptions, browser checks, publication receipts and the exact council decision. Keeps the human-paced full-route rehearsal visibly pending; automated checks do not establish mastery.
document-check.json	Records actual rendered page counts and hashes of the two PDFs and PowerPoint. Establishes document identity/inventory; it does not by itself establish that every explanation is correct.
hardware-roadmap.md	Explains a possible capture-to-alert IDS and the roles of a NIC, DPU, GPU and NPU. Identifies extraction, compiler, precision and validation work still needed. It is a roadmap, not proof of deployment.
presentation-source.json	Shared editable source for all 16 slides, their scripts, the seven questions, guide explanations and expected briefing page count. The generator substitutes measured values into this source to keep outputs consistent.
publication-checks.json	The historical public-download, hosted-browser and CI receipt for the earlier artifact commit. Inspect its commit and timestamps. The audited release has a separate release-attached verification receipt.
questions.md	Customer discussion questions with defensible answers. Useful for likely follow-up questions about credibility, limits and practical use; distinct from the numbered seven-question presentation map.
quickstart.md	Current setup/use/test guide for the one supported CSV packet workflow. Separates saved viewing/checks from fresh native execution. Explains full Git-history requirements, platform-specific limits, optional test skips and retained numerical failures.

File	What it does and when you would open it
client-project-guide.md	Complete single-workflow client explanation: the paper and both CSVs, joint packet model, training/inference, measured results and test limits, setup/demo, component responsibilities, mapping client files to this guide, differences from upstream, automatic updates and future deployment work. Includes a concise introduction and handoff checklist.
results.md	Generated readable tables of all three seed results, selected epochs, per-class errors, learning curves and model timings. Explains metric definitions and limits beside the numbers.
runbook.md	The detailed procedure to acquire source/assets, build the tested environment, run short pretraining and full fine-tuning, evaluate/predict, review exports and rebuild the presentation. Includes a meeting rehearsal sequence.
talk-track.md	The complete presenter script in slide order, plus question locations. Generated from the same script text placed in PowerPoint notes and the guide. Use it to rehearse what you will say.
upstream-comparison.md	Explains the paper's problem, the uploaded packet tables, the separate native flow dataset and exactly what this project adds around the authors' code. Start here if the source/data distinction is unclear.
verification.md	The detailed record of actual checks: source/data identity, runtime builds, numerical tests, training, inference, browser/document review and publication. Distinguishes original evidence, later rechecks and unresolved limitations.

9. docs/customer/demo/: files used for the browser demonstration

Course links: [the recorded replay](#), [one actual error](#) and [new inference](#).

File	What it does and when you would open it
index.html	The self-contained replay application. Displays seed-0 predictions, correct/incorrect results and attack-category filters. Open it offline or on Pages; playback speed controls display pace, not model execution speed.
guide.html	The standalone learning page following the 16 slides and exact speaker scripts. Includes setup routes, plain-language explanations and a glossary. Works offline; links to external resources still need internet access.
course/index.html	Archived interactive course, excluded from public deployment: nine lessons, eight scored questions, explanatory keys, the saved prediction error and a separate customer teach-back checklist. Works offline with no runtime AI service; answers also work without JavaScript. This later teaching artifact is outside the historical audited ZIP.
predictions.json	The browser demo's copy of the 1,041 seed-0 test predictions, with true labels, predicted classes, six logits/scores, metrics and source/model hashes. The HTML embeds its record, so no running prediction server is required.
browser-check.json	Receipt for the recorded browser rehearsal. Binds the page, predictions, model and video by hashes and records tested controls/layout. It is browser evidence, not an additional accuracy experiment.
recorded-demo.webm	A video of the working replay for meeting backup. It shows recorded classifier outputs and browser controls; it is not a live traffic capture.
replay-screenshot.png	Desktop screenshot from the browser rehearsal. Useful for previewing the demo and checking its reviewed layout.

File	What it does and when you would open it
replay-mobile.png	Mobile-width screenshot from the rehearsal. Shows how the same demo fits a narrow screen.

10. docs/customer/figures/: reusable charts

Course links: [three measured results](#) and [the confusion matrix](#).

Each chart is exported in three formats. PNG is a pixel image suitable for slides; SVG is scalable vector artwork suitable for the web/editing; PDF preserves a print-friendly vector figure. These are format variants of two charts, not six experiments.

File	What it does and when you would open it
learning-curves.png	Image version of the three runs' training/validation history. Embedded in the readable results and presentation materials.
learning-curves.svg	Scalable vector version of the learning-curves chart. Use when resizing or inspecting/editing vector artwork.
learning-curves.pdf	Standalone PDF version of the learning-curves chart for print or reuse in a document.
seed0-confusion.png	Image of the seed-0 confusion matrix. The six-by-six counts show which true categories were predicted as other categories.
seed0-confusion.svg	Scalable vector version of that same seed-0 confusion matrix. Useful for high-quality resizing.
seed0-confusion.pdf	Standalone PDF of that same matrix. It summarizes the demo model's 1,041 labeled test predictions.

How to read the experiment-record folders

The original evidence/ folder preserves the main experiment and first checks. The later audit-evidence/ folder preserves rechecks without overwriting those records. A later recheck on the same test flows strengthens execution repeatability; it does not create independent customer-traffic evidence.

Common names have consistent roles:

- **manifest.json** explains the run's identity, inputs, arguments, runtime and process status. Historical absolute machine paths identify what was used then; they are not paths you must reproduce exactly.
- **native.log** is verbose program output, including warnings. **log.txt** is usually one structured JSON record per epoch, despite its .txt extension.
- **train_stats.json** summarizes the training run. **test_stats.json** is the native trainer's final test result for the selected model.
- **metrics.json** contains the activity's result. For evaluation it has accuracy/F1; for timing it has latencies; for an export probe it has support/failure; for unlabeled prediction it contains outputs without accuracy.

- **predictions.json** retains row-level results so aggregate metrics can be independently recomputed.
- **model-export-provenance.json** binds a local model-only export to the original selected checkpoint and class meanings. The JSON is not the model weights.

11. docs/customer/evidence/: original aggregate and input records

Course links: [CSV profiles](#), [training stages and results and errors](#).

File	What it does and when you would open it
artifact-index.json	Inventory of 57 copied original evidence files, with byte sizes, hashes and original names. The index itself makes this folder contain 58 tracked files. Use it to verify that records were copied unchanged.
results.json	The main combined three-seed results record produced by experiment review. Contains each run's budget, selected epoch, metrics, checkpoint/export hashes, training curves, parameter checks and cross-seed aggregate statistics. This is the source for the presentation's measured numbers.
checkpoint-transfer.json	Compares the released pretrained model with the six-class classifier: parameter counts, missing classifier-head weights, extra reconstruction-decoder weights and shape mismatches. Explains why the pretrained encoder needs downstream fine-tuning before classification.
native-data-validation.json	Original validation of all 10,404 native flows: input hashes, split counts, six-category mapping and raw-input overlap. It discloses five train/validation and six train/test shared stored inputs.
uploaded-csv-profile.json	Historical complete-file metadata profiles of the two uploaded packet CSVs: sizes/hashes, row/column counts, label aggregates and missing flow context. These records predate the new payload validation and supervised packet fits; they are not evidence of those later runs. No raw packet table is embedded.
unlabeled-inference-agreement.json	Compares the original 128-flow unlabeled prediction check against the corresponding labeled replay outputs. All predicted classes agree; the recorded maximum logit difference is 0.001953125. This is execution agreement, not a new accuracy estimate.
browser-check.json	Preserved evidence copy of the successful replay browser rehearsal. The corresponding file also appears beside the demo so its identity record travels with the display.
replay-render-manifest.json	Records the corrected replay HTML's generation from unchanged saved predictions. Binds renderer, HTML and prediction hashes, making a display-only correction distinguishable from rerunning the classifier.

evidence/benchmark/

File	What it does and when you would open it
manifest.json	Identifies the original seed-0 model-timing invocation: settings, selected checkpoint, test inputs, runtime and completion. Read it to establish what was timed.
metrics.json	Contains every original latency sample for batches 1/16/128, summary statistics, mean-based throughput and batch/single prediction comparison. Its scope explicitly excludes packet capture and preparation.

evidence/export-probe/

File	What it does and when you would open it
manifest.json	Identifies the original graph-export experiment and model. A completed probe is not itself a successful export.
metrics.json	Records successful eager inference followed by the actual strict Torch graph-capture failure at <code>causal_conv1d_cuda.causal_conv1d_fwd</code> . This is the evidence for the limited portability blocker.

evidence/pretrain-functional/

These four files describe the original short reconstruction-training check. They do not describe the paper’s full Browser/Kitsune pretraining or the initialization history of the released checkpoint.

File	What it does and when you would open it
manifest.json	Records the short pretraining configuration, training-only input identity, original source, runtime, invocation and completed status.
native.log	Full console output from that short pretraining run, including progress, saves and warnings. Use it for the detailed execution trace.
log.txt	Two structured epoch records with overall reconstruction loss and separate byte, size and interval losses. Shows loss changing during the functional check.
train_stats.json	Compact run summary: batch size 128, two completed epochs and about 21.07 seconds elapsed. Update counts require the loop/log context, not just this summary.

evidence/runtime/

“Rehearsal” means a second isolated software installation on the same GB10 workstation. It is not a different hardware platform.

File	What it does and when you would open it
build-report.json	Successful native-extension build record. Lists upstream/causal-convolution pins, exact compatibility patches and matching installed Mamba Python sources. Establishes the built software’s identity.
numerical-check.json	Seven numerical GPU checks in the main environment, with tolerances and observed output/gradient differences. These synthetic checks support runtime correctness for the tested operations.
training-environment.json	Inventory of the main training environment: packages, GPU/driver/compiler, source identities, compiled-extension hashes and dependency warnings. Use when reconstructing the tested setup.
rehearsal-environment.json	Equivalent inventory for the second installation. Helps distinguish a repeated build from simply reusing the original environment.
rehearsal-numerical-check.json	Seven numerical checks repeated in that second environment. Preserves observed errors rather than relying only on matching version strings.
rehearsal-classifier-evaluation/manifest.json	Identifies seed-0 strict evaluation in the rebuilt environment, including the checkpoint and reacquired data hashes.
rehearsal-classifier-evaluation/metrics.json	The second-environment classifier’s measured test result. Its confusion matrix and aggregate metrics agree with the original seed-0 evaluation.

evidence/seed0/: the run used for the demo

Seed 0 completed 120 epochs and achieved **91.26%** test accuracy. Its chosen checkpoint was selected by validation performance; “seed 0” was the predeclared demo choice.

File	What it does and when you would open it
training/manifest.json	Seed-0 training’s configuration, source/data/checkpoint identities, runtime, command and resulting selected classifier. This is the run’s main provenance record.
training/native.log	Seed-0 trainer’s complete console output: progress, validation messages, checkpoint saves, final testing and warnings. Per-epoch “test samples” wording in upstream actually refers to its supplied validation loader.
training/log.txt	Seed-0 history with 120 JSON epoch records, including training loss/rate and validation metrics. Use it to trace learning and verify selection against the full history.
training/train_stats.json	Seed-0 training summary: completed epochs, batch size, elapsed time, best validation accuracy and chosen epoch. Chosen epochs are zero-based.
training/test_stats.json	Native fine-tuner’s final test metrics for its selected seed-0 classifier, including accuracy, weighted metrics and confusion matrix. This is separate from per-epoch validation.
eval/manifest.json	Records a fresh strict-load evaluation of that saved seed-0 classifier. Binds the same checkpoint and labeled test data to a separate invocation.
eval/metrics.json	Seed-0 strict evaluator’s returned metrics and class mapping. Compared against the trainer’s final test result to check saved-model reproducibility.
replay/manifest.json	Identifies the seed-0 inference pass that collected row-level outputs and generated the replay. Includes input/model identity and output references.
replay/metrics.json	Both native and independently computed seed-0 metrics, plus hashes of the prediction record and generated display. Establishes agreement between two metric calculations.
replay/predictions.json	All 1,041 labeled seed-0 predictions, logits and scores. You can reconstruct the 950 correct predictions and 91 mistakes from these rows. This is the original record copied into the demo.
model-export-provenance.json	Binds the local model-only seed-0 export to its selected training checkpoint, pretrained input, class order, source commit and selected epoch. Publishes identities without including the weights.

evidence/seed1/: the second declared training run

Seed 1 completed the same 120-epoch budget on the same split and achieved **84.05%** test accuracy. Its files preserve the less favorable result rather than hiding it behind the demo run.

File	What it does and when you would open it
training/manifest.json	Seed-1 training recipe, inputs, runtime, command, completion and selected-classifier identity. Use it to establish which run these results belong to.
training/native.log	Full native console trace for seed 1, including progress, validation, saves, final test evaluation and warnings.
training/log.txt	The 120 per-epoch seed-1 training/validation records. Provides the learning curve and evidence for its validation-selected epoch.
training/train_stats.json	Seed-1 budget, elapsed time and best validation result/epoch in a compact summary. It is not the final test score.
training/test_stats.json	Native final labeled-test metrics for seed 1’s chosen classifier. Contains the actual seed-1 test accuracy and class-confusion counts.

File	What it does and when you would open it
eval/manifest.json	Identity and completion record for the separate strict evaluation of the saved seed-1 checkpoint.
eval/metrics.json	Seed-1 metrics after strict checkpoint reload, with checkpoint/class provenance. Used to compare saved-model behavior with the trainer's final evaluation.
replay/manifest.json	Invocation and file/model identities for the seed-1 pass collecting full prediction records.
replay/metrics.json	Seed-1 native metrics alongside the independent reconstruction from row-level predictions, plus output hashes.
replay/predictions.json	All 1,041 labeled seed-1 predictions and six-output vectors. Supplies the primary rows for recomputing this run's metrics.
model-export-provenance.json	Identity and class-order record for the locally exported seed-1 model tensors, linked back to the native selected checkpoint.

evidence/seed2/: the third declared training run

Seed 2 completed the same budget and achieved **84.63%** test accuracy. Together the three runs yield **86.65% mean accuracy** and **4.00 percentage points sample standard deviation**. That standard deviation is not a confidence interval or a measure of performance on unseen customer networks.

File	What it does and when you would open it
training/manifest.json	Seed-2 training request, identities, runtime, completion and selected-classifier record.
training/native.log	Complete seed-2 native console output, including progress, validation, checkpoint selection, testing and warnings.
training/log.txt	The 120 seed-2 epoch records. Shows training and validation behavior across the full run.
training/train_stats.json	Seed-2 training budget/time and validation-selected epoch/accuracy. Read alongside its epoch history.
training/test_stats.json	Native final test metrics for the selected seed-2 classifier, including its confusion matrix.
eval/manifest.json	Separate strict evaluation's request, checkpoint/data identity and completion for seed 2.
eval/metrics.json	Seed-2 test metrics returned after reloading the saved classifier; compared with the original final test pass.
replay/manifest.json	Identifies the seed-2 full prediction-recording invocation and its model/data sources.
replay/metrics.json	Seed-2 native and independently reconstructed metric dictionaries, with hashes binding the row records and display.
replay/predictions.json	All 1,041 labeled seed-2 predictions and logits/scores. Enables independent recalculation of accuracy, F1 and confusion counts.
model-export-provenance.json	Records the local model-only seed-2 export's hash, original selected checkpoint, class meanings and upstream/pretraining identities.

evidence/unlabeled-prediction/

File	What it does and when you would open it
manifest.json	Records the original unlabeled seed-0 prediction invocation, source-input hash, checkpoint mapping and temporary native-loader adapter. No raw flow inputs are published here.
metrics.json	Despite its generic name, this contains the first 128 flows' predicted classes, logits and scores without true-label accuracy. Compare it using the separate agreement record.

12. docs/customer/audit-evidence/: later rechecks

Course links: [rechecking versus retraining](#) and [what the accuracy gap does not explain](#).

These 33 records plus their index are separate from the original 57 records. They include fresh execution and explicit failure probes; they do not represent three additional full 120-epoch training runs.

File	What it does and when you would open it
index.json	Inventory of the 33 later audit records and screenshots, with sizes, hashes and origins. Use it to distinguish fresh checks from unchanged original experiment evidence.
source-integrity.txt	Output of the later original-source verification. Confirms the pinned tracked checkout and explicit core hashes still match.
data-validation.json	Fresh validation of all 10,404 released native flows. Preserves split counts, mapping, hashes and overlap observations for comparison with the original validation.
csv-profile.json	Fresh full metadata scans of both uploaded CSVs. The resulting profiles match the original counts and aggregates; this remains profiling, not classifier training.
cpu-tests.txt	Captured output listing the 54 passing CPU tests during the audit. It is a dated test record; current CI indicates whether a later commit also passes.
dependency-install.json	Result of a third fresh Python-dependency installation rehearsal, including package/GPU observations. Native extensions were not rebuilt a third time in this check.
runtime-main.json	The seven numerical GPU checks repeated in the main environment during the later audit, with their tolerances and observed errors.
runtime-rehearsal.json	The same seven checks repeated again in the second built environment. It preserves a distinct runtime recheck record.
public-browser.json	Later browser check of the public replay, including controls/filters, 1,041 rows, 91 errors and viewport widths 320/390/768/1440. The 402 attack-category predictions are model outputs, not confirmed real-world incidents.
unlabeled-agreement.json	Compares all 1,041 later unlabeled seed-0 outputs with the original labeled prediction record. Classes and logits match exactly in this execution; binds the native/export checkpoint identities.
verifier-rejection-checks.json	Records disposable-package checks showing stale guide text and altered speaker script are rejected, even when checksum refresh is requested. These are supplementary failure probes, not extra tests added to the count of 54.
prediction-rejection-checks.json	Records rejection of shortened score vectors, wrong class names, reordered rows and unequal prediction collection lengths. Modified hashes were refreshed in disposable copies so this exercised semantic checks rather than only hash mismatches.

File	What it does and when you would open it
<code>page-count-rejection.json</code>	Records a deliberate wrong expected PDF page count being rejected. Demonstrates that the briefing-page map cannot silently drift just because files were regenerated.
<code>benchmark-arithmetic.json</code>	Independent arithmetic check of the original and repeated benchmark samples: means, medians, p95 and mean-based flows/second. It validates calculations, not network line-rate performance.
<code>primary-summary.json</code>	Compact dated extraction of tests, numerical results, seed metrics, document identities and limits. Prepared before final publication and some later supplementary checks; its pending-publication wording is historical.

audit-evidence/benchmark/

File	What it does and when you would open it
<code>manifest.json</code>	Identity and completion of the later model-timing repetition. Separates this timing run from the original headline measurements.
<code>metrics.json</code>	All later latency samples and summaries. Median batch 1/16/128 times were about 0.9043/4.3290/36.7392 milliseconds, still excluding capture, preparation and transfers.

audit-evidence/export-probe/

File	What it does and when you would open it
<code>manifest.json</code>	Records the later strict graph-export probe's inputs, model identity and completed invocation.
<code>metrics.json</code>	Records eager inference succeeding again and strict Torch export failing again at the tested custom operation. This does not establish that every possible export route would fail.

audit-evidence/export-seed0/, export-seed1/, export-seed2/

These folders evaluate the local **model-only checkpoint exports**. They are unrelated to the failed graph-capture export above.

File	What it does and when you would open it
<code>export-seed0/manifest.json</code>	Fresh strict-evaluation provenance for the seed-0 model-only export, including the export hash and class mapping.
<code>export-seed0/metrics.json</code>	Actual seed-0 export evaluation: its confusion matrix and metrics match the original selected classifier's result.
<code>export-seed1/manifest.json</code>	Fresh strict-evaluation provenance for the seed-1 model-only export. Establishes which exported weights were checked.
<code>export-seed1/metrics.json</code>	Actual seed-1 export evaluation, agreeing with that run's original metrics and confusion matrix.
<code>export-seed2/manifest.json</code>	Fresh strict-evaluation provenance for the seed-2 model-only export, with checkpoint/class identities.
<code>export-seed2/metrics.json</code>	Actual seed-2 export evaluation, agreeing with the original seed-2 saved-classifier result.

audit-evidence/guide/

File	What it does and when you would open it
browser-check.json	Receipt for the final local learning-guide rehearsal: 16 sections/scripts, seven questions, disclosure/anchor/keyboard checks and four widths. Its HTML hash binds it to the reviewed page.
guide-1440.png	Desktop-width screenshot from that guide rehearsal. Shows the reviewed page's visual presentation at 1,440 pixels.
guide-390.png	Narrow-screen screenshot of the guide at 390 pixels. Complements the automatic overflow checks with visible layout evidence.

audit-evidence/pretrain/

File	What it does and when you would open it
manifest.json	Identity, training-only inputs and completed status for the later short masked-pretraining repetition. It remains separate from the main three fine-tuning runs.
native.log	Full console output from that repeated two-epoch functional check, including progress, saving and warnings.
log.txt	Two structured epoch records from the repeated reconstruction-training check. Retains total and modality-specific losses.
train_stats.json	Compact summary of the repeated check: batch size 128, two epochs and approximately 17.91 seconds elapsed. This is a short functionality check, not full pretraining reproduction.

audit-evidence/unlabeled-export/

File	What it does and when you would open it
manifest.json	Provenance for predicting all 1,041 native flows without labels using the seed-0 model-only export. Records the source-input hash and temporary adapter behavior.
metrics.json	All 1,041 unlabeled predictions, with row indices, class names, six logits and six display scores. Supplies the actual outputs behind the full agreement check; it contains no ground-truth accuracy calculation.

13. docs/research/: reasoning, plans and council records

Course links: [evidence-backed claims](#) and [tests, documents and research records](#).

These are research and review records. Model-assisted review can identify issues and define acceptance criteria; it is not a substitute for actually running an experiment. Historical outcomes remain historical even after later work succeeds.

File	What it does and when you would open it
research-record.md	Curated account of the research: why the task moved from packet classification to flow reproduction, paper/source differences, checkpoint interpretation, installation failures/fixes, experiments and remaining gaps. This is the shared AI-assisted research narrative, not a private raw chat transcript.

File	What it does and when you would open it
execution-plan.md	Plan and commitments made for the experiments and customer package, including scope and acceptance work. A plan describes intended work; use manifests and verification for completed-work evidence.
council-decision.json	Immutable canonical council decision for the original experiment/customer-package plan. The package verifier checks its exact hash. Its scope predates the later numerical results.
council-receipt.json	Records the original plan council's ratification identity/order, pinned seats and scope. Establishes how that decision was accepted, not that training was already complete.
audit-council-initial-result.json	Full preserved result of the first later handoff review: ESCALATED / UNRATIFIED . Contains positions, reviews and evidence-pack gaps. It was not rewritten as a success after the narrower follow-up.
audit-primary-pack.json	Compact primary facts supplied to the focused follow-up council, including a hash of the underlying summary. A frozen intake snapshot, so its publication-pending statements reflect that time.
final-council-decision.json	Exact canonical CONSENSUS / RATIFIED decision for the bounded research-demo handoff criteria. Lists findings and required checks, while retaining limits on what the council itself saw and established.
final-council-receipt.json	Records both pinned seats' exact-hash acceptance of the final decision, with model/effort identity and scope. It is a ratification receipt, not a GitHub publication or GPU certification receipt.
final-council-review.md	Readable explanation of that final council decision, its exact hash, the earlier escalation and the local corrections/checks that followed. Links to the separate final public release verification.

14. Video foundation and preserved v2 files

Course links: [versioned teaching tools](#) and [current versus historical downloads](#).

The 30-minute lesson is preserved in the [historical course-video-v2 release](#) and the files listed here. Its source, media, captions, transcript, manifest and receipts describe that version. The [one active watch route](#) is a current navigation surface for the preserved v4 media; its HTML is not a frozen v2 receipt. The original v2 HTML remains available in Git history. These video releases do not replace the seven original audited customer attachments.

File	What it does and when you would open it
docs/customer/video-course-source.json	Preserved v2 narration and visible teaching for 60 scenes. Contains evidence-bound numbers, the nine original chapter budgets, practice durations and two audio-bound caption corrections. The later published v4 edition has its own preserved source file.
docs/customer/video-verification.md	Preserved v2 receipt and rebuild explanation: original downloads, scene times, council outcome and actual media/publication checks. Its pending human-listening status and recorded hashes remain about v2 after the active page changes.
docs/customer/demo/video/NetMambaPlus-30-minute-course.mp4	The actual narrated 30:00, 1080p H.264/AAC course. Download it for offline presentation; visible captions and countdowns are included in the picture.
docs/customer/demo/video/NetMambaPlus-30-minute-course.webm	Preserved browser encoding of the same v2 lesson for players without H.264/AAC support. It has its own exact file hash and the same lesson content as the v2 MP4.
docs/customer/demo/video/index.html	The active v5 watch page teaches the CSV packet workflow and both repositories, with playback, chapters, a reflowing transcript and matching downloads. It does not execute inference. Archived v3/v4 media remains in separate version directories.

File	What it does and when you would open it
<code>docs/customer/demo/video/poster.png</code>	Opening teaching image shown before playback. It is a course illustration, not a live network screenshot.
<code>docs/customer/demo/video/captions.vtt</code>	Timed WebVTT captions loaded by the browser's optional English text track; also a separate download. The encoded video already includes visible captions.
<code>docs/customer/demo/video/captions.srt</code>	The same timed caption text in SubRip format for desktop players and editing tools.
<code>docs/customer/demo/video/transcript.md</code>	Complete timestamped narration, visible teaching points, announced practices and evidence links. Read or download it without playing audio.
<code>docs/customer/demo/video/chapters.json</code>	Nine chapter titles and exact start/end seconds, from 00:00 through 30:00.
<code>docs/customer/demo/video/media-manifest.json</code>	Binds the video source, local voice/model identities, 60 scenes, caption cues, cited files and downloadable media hashes. This build manifest is distinct from the verification receipt.
<code>docs/research/video-council-decision.json</code>	Immutable exact-hash two-seat video review. Defines treatment and acceptance criteria; does not certify unseen final media or turn a pending human review into a pass.
<code>docs/research/video-quality-review.json</code>	Exact final quality-review result: ESCALATED / UNRATIFIED, with no ratified decision hash. Preserves both seats' positions and the outstanding human playback/caption gates; later deterministic fixes are recorded separately in video-verification.md.

Shared video generators and technical checks

File	What it does and when you would open it
<code>requirements/video.txt</code>	Pins the local Kokoro, ONNX, audio-array, WAV and Pillow packages used for the original rendering route. FFmpeg, fonts and matching local Kokoro model/voice files are separate prerequisites. V3's service client has its own dependency file; viewers install none of these packages.
<code>tests/test_course_video.py</code>	CPU regressions for schedules, image/reference containment, captions, codec metadata and isolated page paths. The real stored-media case checks pinned historical v3 source/references/assets/page. Historical v4 readiness has its own gate; synthetic fixtures do not prove decoded or human-reviewed playback.
<code>tools/narrate_course_video.py</code>	Shared source loading, fact substitution and hashing retain v4 defaults and historical reference roots. Explicit v5 sources declare an independent packet context so historical flow facts cannot enter the current lesson. The retained local Kokoro CLI explicitly defaults to v2 and checks its model/voice before writing audio/timing caches. Neither path authors narration.
<code>tools/build_course_video.py</code>	Combines selected source and measured narration with frames, captions, motion and recorded replay footage. Defaults to v4 source, runs/video-course/v4-neural and public v4 output; explicit paths support isolated previews. Checks source/reference/timing identities and writes media, narration and motion manifests. --webm-only derives the browser alternative.
<code>tools/build_video_page.py</code>	Builds the active page from v5 packet media and source-specific title/summary. Requires exact source and reference hashes; --check rejects stale HTML. Canonical v3/v4 inputs retain pinned historical reference checks, and historical versions require separate outputs.
<code>tools/verify_course_video.py</code>	Defaults to preserved v4 source/reference, artifact, scene/chapter and caption checks against its exact Git snapshot. --historical-v3 selects the separate v3 edition; --current checks supplied paths and current references strictly; --legacy retains v2. Optional decode/browser observations recheck protected bytes afterward. File identity is not human playback acceptance.

15. Current portability and full-video audit additions

Course links: [setup](#), [two GPU gates](#) and [platform limits](#).

The [support matrix](#) is the current setup route. The original presentation and quickstart describe the earlier audited experiment snapshot. Their older test counts and platform limits remain historically accurate for that revision, but do not describe the newer portability checks.

File	What it does and when you would open it
tools/build_cuda.py	Builds the two pinned CUDA extensions for a selected compiler-supported NVIDIA GPU on Linux x86-64 or ARM64. Refuses unsupported environments before installation, preserves source pins, records actual compiler flags and installed hashes, and retains failure reports. Additional GPU hardware is not automatically certified by this route.
tools/check_model_runtime.py	Runs the complete original classifier on six explicitly synthetic native-flow fixtures: forward pass, loss, gradients, one optimizer update, checkpoint save/strict reload and inference. Can bind installed extension hashes to a completed build report. Produces functionality evidence without claiming dataset accuracy.
tests/test_build_cuda.py	Tests CUDA preflight, source/architecture patch guards, actual compiler-flag extraction, output protection and build failure handling. These controlled CPU tests are separate from real GPU compilation.
tests/test_model_runtime.py	Tests malformed/incomplete build evidence, extension-hash mismatch, missing inputs, reused outputs and failure manifests for the complete-model checker. Actual GPU acceptance has a separate record.
tests/test_portability.py	Reproduces Windows-style encoding/newline failures with controlled fixtures and checks generated text/hash portability. The hosted workflow then runs the full suite on six actual OS/Python combinations.
docs/support-matrix.md	Current beginner-facing setup: view documents, check saved evidence or run new GPU experiments. Gives installation and test commands, measured platforms, unverified hardware and the reason a CPU model backend is still missing.
docs/portability-evidence.json	Public execution receipt for the generalized CUDA build, seven numerical cases, complete classifier update/reload/inference and all 1,041 real seed-0 predictions in the fresh environment. Keeps failed-first-build history and source/runtime identities.
docs/research/portability-council-review.json	Exact ESCALATED / UNRATIFIED council result, with the unresolved complete CPU integration issue. Directly authorized CUDA changes and subsequent tests do not turn this historical result into consensus.
docs/research/video-audit-v2.json	Historical v2 audit receipt: source coverage, measured corrections, complete audiovisual decode, all-scene frame samples, independent speech-recognition/caption review, browser checks and downloaded-release hashes. Its statuses bind v2 bytes; they do not certify v3. Human listening and uncertain speech-recognition anchors remain distinct from automated checks.

Current packet study: both uploaded CSVs

Start with the [packet study explanation](#) and [packet setup guide](#). All six native comparisons completed 1,000 updates each on GB10. Their frozen checkpoint choices, training records and twelve source-test prediction sets are available, alongside nine diagnostic controls and a separate presentation addendum. The offline native reviewer recomputes 155,580 group predictions representing 354,960 original-row predictions across the twelve tests. File existence, synthetic tests and timing probes alone do not establish these supervised results; the retained achieved records do.

Preparation validates every stored byte and enumerated source label. The SHA-256 of all 1,500 bytes identifies an exact-payload group globally, so the same payload cannot enter different partitions across the two sources. Split seed 17 assigns groups to training, validation or test; fixed hash order caps selected groups without consulting labels. Original row multiplicity and contradictory labels remain represented. This separates exact payloads, but cannot establish independent flows, captures or near-duplicates.

The adapter normalizes each packet's bytes and uses the original stride-four byte embedding, four native Mamba blocks and summary fusion. There are 375 byte tokens, one byte summary and two learned prefixes: 378 tokens of width 256. Size and timing streams have zero observations; their prefixes remain trainable. A learned native 443-position table supplies the same 378 selected coordinates in transferred and scratch models. A fresh binary head outputs benign/attack logits. Metadata, source identity and labels never enter neural forward; labels provide supervised targets outside it.

The frozen comparisons are CIC, UNSW and joint training, each with pretrained or scratch initialization. Each arm requests 1,000 updates of 64 sampled groups at seed 0, with validation every 100 updates. A group contributes its observed benign/attack proportions as the target. Joint batches draw 32 groups from each source. Paired arms use identical fresh heads and sampled groups. Validation group-weighted macro-F1 selects the earliest best checkpoint; joint selection averages the two source scores. All six checkpoint choices freeze before any final test inference, then each selected model is strictly reloaded and evaluated on both source tests.

flowchart TD

```
A[Both local original CSVs] --> B[packet_data.py: validate bytes, retain labels, group and split]
B --> C[Prepared arrays and manifest]
C --> D[freeze_packet_protocol.py: bind inputs, code and six-arm budget]
D --> E[train_packet_model.py and packet_model.py: native supervised updates]
E --> F[Validation selects all six checkpoints before final tests]
F --> G[Saved predictions, training logs and result receipts]
G --> H[review_packet_study.py: replay metrics and verify bindings]
H --> I[predict_packets.py: local checkpoint plus unlabeled packet CSV]
I --> J[packet_controls.py: fit train-only diagnostic models]
J --> K[Freeze all control models before their test inference]
K --> L[Separate packet report and presentation addendum]
```

Row-weighted metrics give each original CSV row one vote; group-weighted metrics give each exact payload one total vote split across its observed labels. Read both confusion matrices, accuracy, balanced accuracy, macro-F1, attack recall, false-positive rate, AUROC and average precision alongside original-label support. No selected PortScan support means no PortScan score; sparse classes need their denominators. Majority, byte-histogram and metadata linear controls diagnose how much simpler patterns explain. Their metadata features remain separate from the native packet model.

The saved-evidence reviewer checks artifact identities, achieved update logs, validation selection, paired sampling, test membership and recorded label totals, then recomputes predictions' metrics. Its optional scikit-learn comparison independently checks the arithmetic. Offline replay does not rescan the private raw CSVs, repeat GPU training or authenticate freely rewritten source records. New unlabeled inference has its own complete, capped or incomplete receipt and cannot report accuracy without truth.

Packet code, protocol and explanations

File	Purpose, inputs, outputs and use
packet_data.py	Reads both original CSVs, validates the exact byte/metadata/label schema, hashes observed inputs, preserves original and binary label counts, groups identical payloads globally and writes deterministic split/cap selections. Produces local NumPy arrays, metadata rows and a manifest. Use before freezing a study; old aggregate profiles cannot substitute for this scan.
packet_model.py	Implements the explicit packet adapter around pinned original NetMamba+ source: byte normalization, 378-token layout, learned native position mapping, summary fusion and a fresh two-class head. Transfers the compatible pretrained trunk or initializes scratch, and strictly saves/reloads packet checkpoints with source, input, class and study identities. Native execution needs the validated CUDA environment.
packet_study.py	Shared standard-library artifact checks and metric arithmetic. Reads per-group logits, binary counts and original-label counts to compute row/group confusion, ranking metrics and label slices while retaining conflicting labels. Used by training, controls and offline review; it does not fit a model.
configs/packet-study.json	Frozen measured-study protocol binding both original upload hashes, prepared-manifest digest, core implementation digests, six arms, learning rates, sampling, selection, resource limits and diagnostic-control settings. Requests 1,000 updates per arm and records the discarded timing-only probe. A completed run must match these bytes; this configuration alone proves no result.
tools/freeze_packet_protocol.py	Reads validated preparation records and writes a fresh protocol before fitting. Requires the registered original uploads for a measured study, declares all six paired comparisons and fingerprints the native execution code. Optional timing evidence informs the bounded update budget without using probe weights or scores as measured results.
tools/train_packet_model.py	Validates each prepared file, payload identity, split and protocol binding, then trains six native packet arms with group-proportion targets. Records gradients, changed parameters, optimizer counters, sampling and validation history; freezes every selected checkpoint before testing both sources. Strict reloads and final input/code checks precede a completed result; interrupted work retains an incomplete receipt.
tools/predict_packets.py	Runs a local selected packet checkpoint on a CSV without label fields: exactly 1,500 bytes, optionally the four declared metadata fields. Forwards bytes only, writes logits/scores/classes, and binds checkpoint, input and output hashes in a completion receipt. An explicit cap still validates and hashes the whole input; malformed or changed inputs leave an incomplete result.
tools/packet_controls.py	Fits nine separate train-only controls: majority, byte-histogram linear and group-metadata linear models for CIC, UNSW and joint data. Retains group soft targets, balances joint source mass, fits scaling only on training groups and freezes all model/code/data identities before control tests. Replays portable saved coefficients on the same selected test membership; NumPy/scikit-learn are required for fitting.
tools/review_packet_study.py	Verifies the complete public packet evidence inventory and six native result records using ordinary Python. Checks achieved updates, earliest-best validation selection, paired sampling, frozen membership and truth totals, then recomputes all twelve native source-test metric sets and invokes the control and unlabeled-inference reviewers. --independent compares scikit-learn arithmetic for both; neither mode repeats fitting or the private CSV scan.
tools/review_packet_controls.py	Offline reviewer for all nine frozen diagnostic control models and their eighteen source-test prediction sets. Checks prepared/protocol/code bindings, training-source weights, feature/scaler contracts, model inventories, selected test membership and original-label totals, then recomputes metrics. Reports convergence limits and optionally compares independent scikit-learn arithmetic; it does not refit controls or rescan private metadata.

File	Purpose, inputs, outputs and use
tools/review_packet_inference.py	Audits the eight public unlabeled-inference records without loading a model or raw CSV. Checks selected-checkpoint and frozen protocol/code bindings, compressed and original prediction hashes, sorted payload identities, class argmax and softmax, then recomputes class agreement and the fixed logit-tolerance failures. A separate optional check reconstructs the later capped128 regression JSONL and compares it with the original CIC prefix. Earlier default/override comparisons have summaries only; passing artifact checks must not erase their recorded numerical failures.
tools/build_packet_briefing.py	Reads completed packet evidence to build a separate eight-slide PDF and PowerPoint plus presenter script and slide-source JSON. Requires document libraries and a local font. These outputs are generated after measured results exist; they do not alter the preserved v4 lesson or prove its coverage of new files.
docs/customer/packet-model-study.md	Explains how both uploads supply native byte inputs and supervised labels, the learned position mapping, prepared group/support counts, six comparisons, controls and limits. Read its result tables together with completed receipts and explicit transfer/support limits, alongside the historical flow/calibration materials to keep the studies distinct.
docs/customer/packet-study-setup.md	Reproduction and use instructions for packet preparation, freezing, native training, controls, unlabeled checkpoint inference and saved-evidence review. Separates ordinary CPU contract checks from the measured GB10 runtime, requires fresh outputs and explains capped/incomplete predictions. Commands are instructions; their presence is not evidence they completed.
docs/superpowers/plans/2026-09-14-packet-model-integration.md	Implementation plan for connecting both packet CSVs to the native encoder, measuring paired training and controls, checking unlabeled inference, and publishing a separate addendum while preserving historical course bytes. Plans and candidate decisions are distinct from frozen protocol, achieved run evidence and council ratification.

Packet and current-guide regression tests

These controlled fixtures test code contracts and rejection behavior. Optional numerical tests need their declared packages and some native model tests need the local CUDA runtime. Skipped tests do not establish CPU model execution or dataset performance.

File	What the individual test module checks
tests/test_packet_data.py	Rejects invalid numeric bytes, schemas and unknown labels; checks exact-payload grouping, deterministic cross-source splits, hash-order caps, retained multiplicity/conflicts and metadata records using small CSV fixtures. Numerical preparation cases require NumPy.
tests/test_packet_model.py	Checks packet metadata and strict checkpoint contracts, rejects flow/position/source drift and preserves outputs. Optional native tests inspect trunk transfer, matching scratch position coordinates, gradients and save/reload behavior on synthetic packets; they do not measure dataset accuracy.
tests/test_packet_prepared_integrity.py	Uses small prepared fixtures to reject missing/extra files, descriptor substitutions, bad shapes/dtypes/counts, source/label mismatch, payload-hash drift, unsorted or duplicate identities, incorrect seeded splits and invalid caps. Verifies the training loader's input boundary with NumPy.
tests/test_packet_training.py	Tests frozen protocol validation without training: code/source bindings, complete required fingerprint inventory, safe arm/code paths, minimum measured update budget and finite positive optimizer/resource limits.
tests/test_packet_protocol_freeze.py	Checks that measured freezing requires the original registered source hashes, preserves an existing output and binds prepared/code identities, all six comparisons and declared budgets. Synthetic records establish rejection behavior, not a measured study.

File	What the individual test module checks
<code>tests/test_packet_study.py</code>	Checks row/group weighting with repeated or conflicting labels, class ties, score validation, original-label support and ranking metric definitions. Small analytic examples expose mistakes hidden by aggregate accuracy.
<code>tests/test_packet_evidence_review.py</code>	Synthetic evidence fixtures reject altered membership, truth totals, selected validation scores, pair sampling and incomplete achieved budgets even when related artifact hashes are refreshed. Optional independent scikit-learn comparisons use hand-calculated row/group metrics; no native training is fabricated.
<code>tests/test_packet_predict.py</code>	Exercises the no-label CLI with controlled model calls: strict schema/byte validation, metadata exclusion, checkpoint/study binding, full-file validation after a cap, changing inputs, malformed logits and complete/capped/incomplete receipts. Rejects output reuse and retains actual failures.
<code>tests/test_packet_inference_review.py</code>	Synthetic offline inference fixtures reject changed compressed or uncompressed hashes, wrong payload/order, false classes/probabilities, mismatched checkpoint bindings, loosened tolerances and hidden prior failure rows. Confirms that every class can agree while the strict numerical comparison still fails; no new GPU inference is performed.
<code>tests/test_packet_controls.py</code>	Checks stored-byte histograms, grouped metadata means/protocol fractions, soft targets and balanced joint source mass. Tiny numerical fits test train-only scaling, portable score replay, pre-test freeze and changed-model rejection; they are separate from measured controls.
<code>tools/verify_repository_guide.py</code>	Compares today's complete tracked and nonignored file inventory with explicit individual table entries in this guide. Requires one unique file anchor, one local first-cell file link and a description; rejects missing/extra paths and duplicate or unsafe entries. This structural check neither rewrites course coverage nor claims that a video or every explanation is current.
<code>tests/test_repository_guide.py</code>	Tests exact current inventory, individual-entry/anchor requirements, safe links and missing descriptions. Comments, fenced examples and indented code cannot impersonate real guide entries.
<code>tests/test_course_history_v4.py</code>	Tests exact immutable v4 bytes and protected inventory, archive/reference data handling, source/media/version path guards and preserved v3 protection. Negative fixtures reject added/changed protected artifacts and version substitution; historical Python is read as data and never executed.

Packet preparation, protocol and achieved-run evidence

File	What the individual record establishes and how to use it
docs/customer/evidence/packet-study/index.json	Complete retained packet-evidence file inventory with byte sizes and SHA-256 identities. The reviewer rejects extra, missing, duplicated or changed files. This index binds the exported bundle; it does not independently authenticate original private source truth.
docs/customer/evidence/packet-study/manifest.json	Complete original-CSV preparation record: observed and registered upload identities, source label order, payload/count/subtype/split/selection array hashes, all-row and selected support, cross-source duplicates and exact group-membership digests. Raw arrays remain local; use this record to identify the population behind each test.
docs/customer/evidence/packet-study/protocol.json	Exact copy of the measured protocol frozen before native fitting, binding both original uploads, prepared data, implementation, all six arms and the 1,000-update selection/sampling budget. Must match configs/packet-study.json and the run/result bindings.
docs/customer/evidence/packet-study/results.json	Completed six-arm native packet report with twelve source-test evaluations, selected checkpoint identities, training receipt links, runtime and elapsed time. Contains both row/group metrics and exposure labels; familiar-source performance and unsupervised source transfer have different meanings.
docs/customer/evidence/packet-study/runtime.json	Actual native study execution environment and command: GB10, Linux ARM64, Python, Torch/NumPy/CUDA versions, float32 with autocast/TF32 disabled, compiler path and protocol/preparation identities. Describes this measured machine, not every supported platform.
docs/customer/evidence/packet-study/frozen-checkpoints.json	All six validation-selected checkpoint paths, hashes, sizes, steps and scores, recorded together before final native test inference. Checkpoint files stay local; strict reload and exported prediction evidence retain their identities.
docs/customer/evidence/packet-study/initialization-audit.json	Primary-source/native-initialization audit binding the upstream files, checkpoint shapes and allowed transfer/omission sets. Establishes learned positional parameters, the exact 443-to-378 remap, compatible trunk transfer and fresh-head comparison. This mechanical audit accessed no study data and measured no accuracy.
docs/customer/evidence/packet-study/native-mechanics.json	Actual GB10 mechanical check using two real payloads from each source with synthetic targets. Records finite loss/gradients through the native adapter and exact checkpoint save/reload agreement, plus source and initialization provenance. This is functionality evidence, separate from the six measured supervised arms.
docs/customer/evidence/packet-study/native-mechanics-tests.txt	Whitespace-normalized captured output of 17 packet-model contract and native mechanical tests in the measured environment. Includes transfer/position checks, payload gradients and strict checkpoint roundtrip tests. The successful test log establishes those cases, not dataset accuracy or an additional training study.
docs/customer/evidence/packet-study/post-study-io-check.json	Actual GB10 regression after the predictor portability fix. Contains its capped receipt and all 128 predictions: the full 25,930-row input was validated, the first 128 CIC classes agreed, and their score comparison passed. This later check uses a newer predictor hash and does not replace the full experiment or erase its numerical failures.
docs/customer/evidence/packet-study/metadata-vocabulary-audit.json	Full original-row audit of named protocol tokens and invalid numeric metadata counts in both uploads. Grounds the controls parser's explicit protocol-name allowlist and recorded correction; metadata is still excluded from native packet forward.
docs/customer/evidence/packet-study/control-freeze-observation.json	Timestamped observation of the completed control-model freeze hash before any control test output existed, while native fitting was still running. Supplements the frozen control manifest; it is not a native checkpoint freeze or a control performance result.
docs/customer/evidence/packet-study/timing.json	Timing-only GB10 probe with real bytes, synthetic alternating targets, candidate update-budget estimates and the chosen 1,000 updates per arm. Probe weights were discarded and no scores were selected; measured training did not initialize from this probe.

File	What the individual record establishes and how to use it
<code>docs/customer/evidence/packet-study/timing-probe-source.txt</code>	Preserved source text for the local timing-only script named by the timing record. Read to inspect exactly what was timed; it is retained as evidence data and is not the measured six-arm trainer.

Every native packet arm: training and both source tests

File	What the individual record establishes and how to use it
docs/customer/evidence/packet-study/cic_pretrained/training.jsonl	CIC, transferred initialization: one observed loss, gradient norm and elapsed time per optimizer step, covering the achieved 1,000-update budget. Read with its training receipt; a selected checkpoint can come from an earlier validation step.
docs/customer/evidence/packet-study/cic_pretrained/training-receipt.json	CIC, transferred initialization: achieved optimizer counters, first-update gradient evidence, changed parameter hashes, sampled/distinct training groups, paired sampling digest, ten validation observations and the selected checkpoint. Binds the log and distinguishes completed updates from checkpoint selection.
docs/customer/evidence/packet-study/cic_pretrained/test-cic.jsonl.gz	CIC, transferred initialization: compressed per-group native logits, exact payload IDs and retained binary/original-label counts for 20,000 CIC payload groups representing 47,048 rows. This source participated in supervised fitting, with these selected test groups held out. The reviewer recomputes both weighting schemes and verifies membership/support.
docs/customer/evidence/packet-study/cic_pretrained/test-unsw.jsonl.gz	CIC, transferred initialization: compressed per-group native logits, exact payload IDs and retained binary/original-label counts for 5,930 UNSW payload groups representing 12,112 rows. The model had no supervised fitting on this source; this is its transfer test. The reviewer recomputes both weighting schemes and verifies membership/support.
docs/customer/evidence/packet-study/cic_scratch/training.jsonl	CIC, scratch initialization: one observed loss, gradient norm and elapsed time per optimizer step, covering the achieved 1,000-update budget. Read with its training receipt; a selected checkpoint can come from an earlier validation step.
docs/customer/evidence/packet-study/cic_scratch/training-receipt.json	CIC, scratch initialization: achieved optimizer counters, first-update gradient evidence, changed parameter hashes, sampled/distinct training groups, paired sampling digest, ten validation observations and the selected checkpoint. Binds the log and distinguishes completed updates from checkpoint selection.
docs/customer/evidence/packet-study/cic_scratch/test-cic.jsonl.gz	CIC, scratch initialization: compressed per-group native logits, exact payload IDs and retained binary/original-label counts for 20,000 CIC payload groups representing 47,048 rows. This source participated in supervised fitting, with these selected test groups held out. The reviewer recomputes both weighting schemes and verifies membership/support.
docs/customer/evidence/packet-study/cic_scratch/test-unsw.jsonl.gz	CIC, scratch initialization: compressed per-group native logits, exact payload IDs and retained binary/original-label counts for 5,930 UNSW payload groups representing 12,112 rows. The model had no supervised fitting on this source; this is its transfer test. The reviewer recomputes both weighting schemes and verifies membership/support.
docs/customer/evidence/packet-study/unsw_pretrained/training.jsonl	UNSW, transferred initialization: one observed loss, gradient norm and elapsed time per optimizer step, covering the achieved 1,000-update budget. Read with its training receipt; a selected checkpoint can come from an earlier validation step.
docs/customer/evidence/packet-study/unsw_pretrained/training-receipt.json	UNSW, transferred initialization: achieved optimizer counters, first-update gradient evidence, changed parameter hashes, sampled/distinct training groups, paired sampling digest, ten validation observations and the selected checkpoint. Binds the log and distinguishes completed updates from checkpoint selection.
docs/customer/evidence/packet-study/unsw_pretrained/test-cic.jsonl.gz	UNSW, transferred initialization: compressed per-group native logits, exact payload IDs and retained binary/original-label counts for 20,000 CIC payload groups representing 47,048 rows. The model had no supervised fitting on this source; this is its transfer test. The reviewer recomputes both weighting schemes and verifies membership/support.
docs/customer/evidence/packet-study/unsw_pretrained/test-unsw.jsonl.gz	UNSW, transferred initialization: compressed per-group native logits, exact payload IDs and retained binary/original-label counts for 5,930 UNSW payload groups representing 12,112 rows. This source participated in supervised fitting, with these selected test groups held out. The reviewer recomputes both weighting schemes and verifies membership/support.

File	What the individual record establishes and how to use it
docs/customer/evidence/packet-study/unsw_scratch/training.jsonl	UNSW, scratch initialization: one observed loss, gradient norm and elapsed time per optimizer step, covering the achieved 1,000-update budget. Read with its training receipt; a selected checkpoint can come from an earlier validation step.
docs/customer/evidence/packet-study/unsw_scratch/training-receipt.json	UNSW, scratch initialization: achieved optimizer counters, first-update gradient evidence, changed parameter hashes, sampled/distinct training groups, paired sampling digest, ten validation observations and the selected checkpoint. Binds the log and distinguishes completed updates from checkpoint selection.
docs/customer/evidence/packet-study/unsw_scratch/test-cic.jsonl.gz	UNSW, scratch initialization: compressed per-group native logits, exact payload IDs and retained binary/original-label counts for 20,000 CIC payload groups representing 47,048 rows. The model had no supervised fitting on this source; this is its transfer test. The reviewer recomputes both weighting schemes and verifies membership/support.
docs/customer/evidence/packet-study/unsw_scratch/test-unsw.jsonl.gz	UNSW, scratch initialization: compressed per-group native logits, exact payload IDs and retained binary/original-label counts for 5,930 UNSW payload groups representing 12,112 rows. This source participated in supervised fitting, with these selected test groups held out. The reviewer recomputes both weighting schemes and verifies membership/support.
docs/customer/evidence/packet-study/joint_pretrained/training.jsonl	CIC and UNSW jointly, transferred initialization: one observed loss, gradient norm and elapsed time per optimizer step, covering the achieved 1,000-update budget. Read with its training receipt; a selected checkpoint can come from an earlier validation step.
docs/customer/evidence/packet-study/joint_pretrained/training-receipt.json	CIC and UNSW jointly, transferred initialization: achieved optimizer counters, first-update gradient evidence, changed parameter hashes, sampled/distinct training groups, paired sampling digest, ten validation observations and the selected checkpoint. Binds the log and distinguishes completed updates from checkpoint selection.
docs/customer/evidence/packet-study/joint_pretrained/test-cic.jsonl.gz	CIC and UNSW jointly, transferred initialization: compressed per-group native logits, exact payload IDs and retained binary/original-label counts for 20,000 CIC payload groups representing 47,048 rows. This source participated in supervised fitting, with these selected test groups held out. The reviewer recomputes both weighting schemes and verifies membership/support.
docs/customer/evidence/packet-study/joint_pretrained/test-unsw.jsonl.gz	CIC and UNSW jointly, transferred initialization: compressed per-group native logits, exact payload IDs and retained binary/original-label counts for 5,930 UNSW payload groups representing 12,112 rows. This source participated in supervised fitting, with these selected test groups held out. The reviewer recomputes both weighting schemes and verifies membership/support.
docs/customer/evidence/packet-study/joint_scratch/training.jsonl	CIC and UNSW jointly, scratch initialization: one observed loss, gradient norm and elapsed time per optimizer step, covering the achieved 1,000-update budget. Read with its training receipt; a selected checkpoint can come from an earlier validation step.
docs/customer/evidence/packet-study/joint_scratch/training-receipt.json	CIC and UNSW jointly, scratch initialization: achieved optimizer counters, first-update gradient evidence, changed parameter hashes, sampled/distinct training groups, paired sampling digest, ten validation observations and the selected checkpoint. Binds the log and distinguishes completed updates from checkpoint selection.
docs/customer/evidence/packet-study/joint_scratch/test-cic.jsonl.gz	CIC and UNSW jointly, scratch initialization: compressed per-group native logits, exact payload IDs and retained binary/original-label counts for 20,000 CIC payload groups representing 47,048 rows. This source participated in supervised fitting, with these selected test groups held out. The reviewer recomputes both weighting schemes and verifies membership/support.
docs/customer/evidence/packet-study/joint_scratch/test-unsw.jsonl.gz	CIC and UNSW jointly, scratch initialization: compressed per-group native logits, exact payload IDs and retained binary/original-label counts for 5,930 UNSW payload groups representing 12,112 rows. This source participated in supervised fitting, with these selected test groups held out. The reviewer recomputes both weighting schemes and verifies membership/support.

Every diagnostic control model and test record

File	What the individual record establishes and how to use it
docs/customer/evidence/packet-study/controls/frozen-controls.json	Freeze manifest for all nine train-only controls, binding prepared/protocol/source/code identities, portable model file hashes, training groups, feature definitions and convergence observations before control inference. Metadata grouping and the named-protocol parser correction are explicit.
docs/customer/evidence/packet-study/controls/evaluation/results.json	Completed control report with all eighteen source-test metric sets, model/prediction hashes and convergence observations. Compare the same selected populations and row/group weights as the native models; metadata and byte-histogram controls are separate diagnostic inputs.
docs/customer/evidence/packet-study/controls/models/cic_majority.json	CIC: frozen constant group-majority model fitted only on selected training groups with their label proportions. Retains the selected constant class and weighted training label mass. Bound to the prepared data, protocol and control implementation; this is a small diagnostic model, not a native Mamba checkpoint.
docs/customer/evidence/packet-study/controls/evaluation/cic_majority-test-cic.jsonl.gz	CIC constant group-majority control: compressed logits, exact selected payload IDs and observed label counts on the CIC test split. Test source participated in this control's fitting. Metrics preserve the same row/group denominators as the native study.
docs/customer/evidence/packet-study/controls/evaluation/cic_majority-test-unsw.jsonl.gz	CIC constant group-majority control: compressed logits, exact selected payload IDs and observed label counts on the UNSW test split. Transfer evaluation on a source absent from this control's supervised fitting. Metrics preserve the same row/group denominators as the native study.
docs/customer/evidence/packet-study/controls/models/cic_byte_histogram_linear.json	CIC: frozen 256-bin stored-byte histogram linear model fitted only on selected training groups with their label proportions. Retains coefficients, intercept, training-only scaler, feature order, fitting settings and convergence observations for portable score replay. Bound to the prepared data, protocol and control implementation; this is a small diagnostic model, not a native Mamba checkpoint.
docs/customer/evidence/packet-study/controls/evaluation/cic_byte_histogram_linear-test-cic.jsonl.gz	CIC 256-bin stored-byte histogram linear control: compressed logits, exact selected payload IDs and observed label counts on the CIC test split. Test source participated in this control's fitting. Metrics preserve the same row/group denominators as the native study.
docs/customer/evidence/packet-study/controls/evaluation/cic_byte_histogram_linear-test-unsw.jsonl.gz	CIC 256-bin stored-byte histogram linear control: compressed logits, exact selected payload IDs and observed label counts on the UNSW test split. Transfer evaluation on a source absent from this control's supervised fitting. Metrics preserve the same row/group denominators as the native study.
docs/customer/evidence/packet-study/controls/models/cic_metadata_linear.json	CIC: frozen group-metadata linear model fitted only on selected training groups with their label proportions. Retains coefficients, intercept, training-only scaler, feature order, fitting settings and convergence observations for portable score replay. Bound to the prepared data, protocol and control implementation; this is a small diagnostic model, not a native Mamba checkpoint.
docs/customer/evidence/packet-study/controls/evaluation/cic_metadata_linear-test-cic.jsonl.gz	CIC group-metadata linear control: compressed logits, exact selected payload IDs and observed label counts on the CIC test split. Test source participated in this control's fitting. Metrics preserve the same row/group denominators as the native study.
docs/customer/evidence/packet-study/controls/evaluation/cic_metadata_linear-test-unsw.jsonl.gz	CIC group-metadata linear control: compressed logits, exact selected payload IDs and observed label counts on the UNSW test split. Transfer evaluation on a source absent from this control's supervised fitting. Metrics preserve the same row/group denominators as the native study.
docs/customer/evidence/packet-study/controls/models/unsw_majority.json	UNSW: frozen constant group-majority model fitted only on selected training groups with their label proportions. Retains the selected constant class and weighted training label mass. Bound to the prepared data, protocol and control implementation; this is a small diagnostic model, not a native Mamba checkpoint.

File	What the individual record establishes and how to use it
docs/customer/evidence/packet-study/controls/evaluation/unsw_majority-test-cic.jsonl.gz	UNSW constant group-majority control: compressed logits, exact selected payload IDs and observed label counts on the CIC test split. Transfer evaluation on a source absent from this control's supervised fitting. Metrics preserve the same row/group denominators as the native study.
docs/customer/evidence/packet-study/controls/evaluation/unsw_majority-test-unsw.jsonl.gz	UNSW constant group-majority control: compressed logits, exact selected payload IDs and observed label counts on the UNSW test split. Test source participated in this control's fitting. Metrics preserve the same row/group denominators as the native study.
docs/customer/evidence/packet-study/controls/models/unsw_byte_histogram_linear.json	UNSW: frozen 256-bin stored-byte histogram linear model fitted only on selected training groups with their label proportions. Retains coefficients, intercept, training-only scaler, feature order, fitting settings and convergence observations for portable score replay. Bound to the prepared data, protocol and control implementation; this is a small diagnostic model, not a native Mamba checkpoint.
docs/customer/evidence/packet-study/controls/evaluation/unsw_byte_histogram_linear-test-cic.jsonl.gz	UNSW 256-bin stored-byte histogram linear control: compressed logits, exact selected payload IDs and observed label counts on the CIC test split. Transfer evaluation on a source absent from this control's supervised fitting. Metrics preserve the same row/group denominators as the native study.
docs/customer/evidence/packet-study/controls/evaluation/unsw_byte_histogram_linear-test-unsw.jsonl.gz	UNSW 256-bin stored-byte histogram linear control: compressed logits, exact selected payload IDs and observed label counts on the UNSW test split. Test source participated in this control's fitting. Metrics preserve the same row/group denominators as the native study.
docs/customer/evidence/packet-study/controls/models/unsw_metadata_linear.json	UNSW: frozen group-metadata linear model fitted only on selected training groups with their label proportions. Retains coefficients, intercept, training-only scaler, feature order, fitting settings and convergence observations for portable score replay. Bound to the prepared data, protocol and control implementation; this is a small diagnostic model, not a native Mamba checkpoint.
docs/customer/evidence/packet-study/controls/evaluation/unsw_metadata_linear-test-cic.jsonl.gz	UNSW group-metadata linear control: compressed logits, exact selected payload IDs and observed label counts on the CIC test split. Transfer evaluation on a source absent from this control's supervised fitting. Metrics preserve the same row/group denominators as the native study.
docs/customer/evidence/packet-study/controls/evaluation/unsw_metadata_linear-test-unsw.jsonl.gz	UNSW group-metadata linear control: compressed logits, exact selected payload IDs and observed label counts on the UNSW test split. Test source participated in this control's fitting. Metrics preserve the same row/group denominators as the native study.
docs/customer/evidence/packet-study/controls/models/joint_majority.json	CIC and UNSW jointly: frozen constant group-majority model fitted only on selected training groups with their label proportions. Retains the selected constant class and weighted training label mass. Bound to the prepared data, protocol and control implementation; this is a small diagnostic model, not a native Mamba checkpoint.
docs/customer/evidence/packet-study/controls/evaluation/joint_majority-test-cic.jsonl.gz	CIC and UNSW jointly constant group-majority control: compressed logits, exact selected payload IDs and observed label counts on the CIC test split. Test source participated in this control's fitting. Metrics preserve the same row/group denominators as the native study.
docs/customer/evidence/packet-study/controls/evaluation/joint_majority-test-unsw.jsonl.gz	CIC and UNSW jointly constant group-majority control: compressed logits, exact selected payload IDs and observed label counts on the UNSW test split. Test source participated in this control's fitting. Metrics preserve the same row/group denominators as the native study.
docs/customer/evidence/packet-study/controls/models/joint_byte_histogram_linear.json	CIC and UNSW jointly: frozen 256-bin stored-byte histogram linear model fitted only on selected training groups with their label proportions. Retains coefficients, intercept, training-only scaler, feature order, fitting settings and convergence observations for portable score replay. Bound to the prepared data, protocol and control implementation; this is a small diagnostic model, not a native Mamba checkpoint.

File	What the individual record establishes and how to use it
docs/customer/evidence/packet-study/controls/evaluation/joint_byte_histogram_linear-test-cic.jsonl.gz	CIC and UNSW jointly 256-bin stored-byte histogram linear control: compressed logits, exact selected payload IDs and observed label counts on the CIC test split. Test source participated in this control's fitting. Metrics preserve the same row/group denominators as the native study.
docs/customer/evidence/packet-study/controls/evaluation/joint_byte_histogram_linear-test-unsw.jsonl.gz	CIC and UNSW jointly 256-bin stored-byte histogram linear control: compressed logits, exact selected payload IDs and observed label counts on the UNSW test split. Test source participated in this control's fitting. Metrics preserve the same row/group denominators as the native study.
docs/customer/evidence/packet-study/controls/models/joint_metadata_linear.json	CIC and UNSW jointly: frozen group-metadata linear model fitted only on selected training groups with their label proportions. Retains coefficients, intercept, training-only scaler, feature order, fitting settings and convergence observations for portable score replay. Bound to the prepared data, protocol and control implementation; this is a small diagnostic model, not a native Mamba checkpoint.
docs/customer/evidence/packet-study/controls/evaluation/joint_metadata_linear-test-cic.jsonl.gz	CIC and UNSW jointly group-metadata linear control: compressed logits, exact selected payload IDs and observed label counts on the CIC test split. Test source participated in this control's fitting. Metrics preserve the same row/group denominators as the native study.
docs/customer/evidence/packet-study/controls/evaluation/joint_metadata_linear-test-unsw.jsonl.gz	CIC and UNSW jointly group-metadata linear control: compressed logits, exact selected payload IDs and observed label counts on the UNSW test split. Test source participated in this control's fitting. Metrics preserve the same row/group denominators as the native study.

Actual unlabeled packet inference and retained numerical differences

These records check a strictly reloaded joint-pretrained checkpoint on the selected test payloads with labels omitted from the input. They are a repeatability check on the same held-out groups. All 25,930 predicted classes agree with the measured evaluation; the fixed strict logit comparison still fails for two CIC rows under matched source batches and FP32 settings. The earlier default and override-only failures remain visible as summaries.

File	What the individual inference record establishes and how to use it
docs/customer/evidence/packet-study/unlabeled/agreement.json	Comparison of actual unlabeled inference with the saved joint-pretrained model evaluation: all 25,930 classes agree. Retains matched FP32/source-batch settings, prediction/receipt hashes and the two CIC rows that fail the fixed raw-logit tolerance; <code>strict_logit_tolerance_passed</code> remains false. Class agreement is separate from bit-exact or tolerance-passing numerical replay.
docs/customer/evidence/packet-study/unlabeled/cic-predictions.jsonl.gz	Compressed actual unlabeled predictions for 20,000 selected CIC payload groups from the joint-pretrained checkpoint. Each row retains input order, payload hash, two logits, class and uncalibrated probabilities. The offline reviewer compares these with the corresponding measured group evaluation; two rows retain strict logit-tolerance failures despite complete class agreement.
docs/customer/evidence/packet-study/unlabeled/cic-receipt.json	Completed CIC inference receipt for all 20,000 byte-only rows, with whole-input and original uncompressed-output identities, selected checkpoint/study/code bindings and batch size. No cap or label fields were used. Read beside the compressed predictions and agreement record; completion alone does not establish numerical equivalence.
docs/customer/evidence/packet-study/unlabeled/unsw-predictions.jsonl.gz	Compressed actual unlabeled predictions for 5,930 selected UNSW payload groups from the joint-pretrained checkpoint. Preserves row/payload identities, logits, classes and uncalibrated probabilities. All classes and the recorded strict logit comparison agree with the matching measured evaluation, although floating-point outputs are not bit-identical.

File	What the individual inference record establishes and how to use it
docs/customer/evidence/packet-study/unlabeled/unsw-receipt.json	Completed UNSW inference receipt covering all 5,930 byte-only rows, with whole-input/original-output hashes, checkpoint metadata, implementation identities and batch size. Binds this separate-source execution to the frozen joint-pretrained model; the public bundle does not include its private raw input CSV or neural weights.
docs/customer/evidence/packet-study/unlabeled/default-receipt.json	Completed first standalone inference receipt for the combined 25,930-row unlabeled input under the default execution settings. Binds the input, selected joint-pretrained checkpoint and original local prediction output. Only this receipt and its comparison summary are retained publicly for that initial run; the later separate-source predictions have their own receipts.
docs/customer/evidence/packet-study/unlabeled/default-comparison.json	Retained initial default-inference comparison: 25,930 matching classes, 39 failed rows at the fixed strict logit tolerance and maximum absolute logit difference about 0.001894. Explains the precision-setting difference from training. Earlier raw predictions are not public, so offline review checks this summary's internal consistency rather than replaying those differences.
docs/customer/evidence/packet-study/unlabeled/override-only-comparison.json	Retained precision diagnostic using NVIDIA_TF32_OVERRIDE=0 with the combined-source input. All 25,930 classes still agree, while eight rows fail the unchanged strict logit comparison. Combined input changes source-boundary batch composition; this summary remains separate from the later matched source-batch records and does not claim a successful strict numerical replay.

Separate packet presentation and review history

File	What the individual record establishes and how to use it
docs/customer/packet-addendum/NetMambaPlus-packet-addendum.pdf	Eight-slide fixed-layout packet study addendum built from completed evidence: byte/label integration, groups, native training, selected results, controls, inference and limits. Present beside the preserved v4 course; this PDF contains the later packet measurements.
docs/customer/packet-addendum/NetMambaPlus-packet-addendum.pptx	Editable eight-slide PowerPoint for the same separate packet addendum, with speaker notes. Generated from packet evidence and the builder; it does not replace the historical v4 or original sixteen-slide flow deck.
docs/customer/packet-addendum/packet-addendum-script.md	Presenter notes in slide order for the eight-slide packet addendum. Explains the actual adaptation, achieved measurements and limits; read alongside the generated PDF/PowerPoint.
docs/customer/packet-addendum/packet-addendum-source.json	Generated structured content, tables and notes resolved from the packet evidence for all eight addendum slides. Inspect to trace the visible wording/numbers to the builder and retained results; it is teaching data, not model input.
docs/customer/packet-addendum/verification.json	Hash-bound content inspection of the eight-slide packet PDF, PowerPoint, structured slide source and presenter script. Records matching titles/result cells/notes, inclusion of the stronger metadata control and inference numerical limits, plus review of rendered PDF pages. Its scope excludes PowerPoint application rendering and a new narrated video.
docs/research/packet-connection-council.json	First packet-connection council result: ESCALATED / UNRATIFIED, no mutual decision hash. Retains positions and the blocker about independently verifying current file coverage without claiming old video coverage of new fits. Later directly authorized work leaves this historical outcome unchanged.
docs/research/packet-model-council.json	Native packet-model council result: ESCALATED / UNRATIFIED, no mutual decision hash. Retains missing primary positional/initialization evidence and the required matching scratch mapping. Subsequent source audit and measured work are separate evidence, not retroactive consensus.
tests/test_packet_controls_review.py	Synthetic offline control-review fixtures reject inconsistent model/scaler/feature contracts, frozen identities, missing control evaluations, altered membership or truth totals and mismatched recorded metrics. These tests check saved-evidence integrity without fitting real controls or running a GPU.

Run `python tools/verify_repository_guide.py` for current file coverage, `python tools/review_packet_study.py` for a completed public packet bundle, and the v3/v4 verification commands for their exact published editions. Current guide entries do not make new packet files part of an old lesson. Further evidence or addendum files need their own individual entries when created.

16. Published v4 course and confidence calibration

V4 preserves one 60-minute lesson with 12 chapters, 96 stable scene IDs and 200 seconds of declared practice. Codex-authored text and Andrew synthetic speech describe the original flow experiment and its confidence calibration. The published edition is fixed at commit `4eba1da479238a10ff29097b2e0a27d98a79f063`, including its complete 494-file tree and then-current guide. Its source and measured narration allocation drive the historical scene map below. Its records bind actual encoded duration, layout, speech/caption observations and downloads; they do not cover the later packet study.

The flow calibration feature fits one positive temperature per frozen classifier on 1,040 validation flows, then adds calibrated scores and optional accept/defer to native prediction. It changes no weights or winning class. Across the three already-seen 1,041-flow test sets, mean NLL changed from **0.4416 to 0.4172**, Brier from **0.2035 to 0.2012**, and 15-bin ECE from **7.03% to 3.82%**; mean accuracy stayed **86.65%**. Lower is better for these confidence measurements, but they are different metrics, not an operational guarantee.

At the fixed 0.90 threshold, seed 0 accepted **620/1,041** flows with **9/620** errors before scaling, versus **893/1,041** with **37/893** errors afterward. More accepted rows included more mistakes. Validation reuse, five training/validation duplicates, unknown pretraining exposure and an already-examined test set make this a retrospective study. See the calibration guide for all seeds and the unchanged row-635 mistake.

The production path was source → identity-checked speech caches → timed video/frames → matching documents → technical/content checks → coverage/publication. The v3 and v4 verification CLIs now default to their exact published Git snapshots, reading archived reference bytes as data without executing old code and rejecting changes to protected artifacts. V4 requires its original 13 scientific claims, ten calibration claims and complete 494-file historical inventory. Explicit `--current` coverage verification still compares a course against today's tree and will reject new files the course does not cover. The independent current-guide gate checks today's file entries without relaxing either historical protection or course readiness.

Calibration code, documentation and recorded inputs/outputs

File	Purpose, inputs, outputs and use
calibration.py	Pure standard-library fitting, calibrated softmax, NLL/Brier/ECE and acceptance arithmetic. Inputs are finite logits, fitting/measurement labels and a model/runtime binding; outputs are a positive temperature, scores/metrics or explicit validation errors. Shared by fitting, prediction and CPU recomputation.
tools/calibrate.py	Reads metadata.json and data-valid.json plus matching recorded evidence, then runs the original CUDA loader/classifier with targets removed before forward. Writes calibration.json and an execution manifest with complete validation observations to a fresh directory. Never opens actual train/test splits; --dry-run checks inputs without fitting.
tools/review_calibration.py	Reads all three saved fitting/prediction records, fixed protocol and hash-bound historical labels. Repeats validation fits and recomputes paired test metrics/counts on CPU. --check compares results.json without writing; normal mode creates a new summary. It does not rerun CUDA or create an independent holdout.
tools/verify_video_course_v4.py	Defaults to the exact published v4 Git tree: all 494 historical files, archived guide/reference bytes, 23 required claims and unchanged protected source/media/audit identities. --current explicitly enforces coverage against today's inventory; --content does not grant readiness. Human full-watch/all-caption acceptance stays pending.
tests/test_calibration.py	Analytic and rejection fixtures for the temperature optimum, normalization, preserved argmax, metric definitions, empty accepted-set error and malformed/incompatible artifacts. Native-interface fixtures check validation-only data and fitting boundaries; these are temporary tests, not research samples.
tests/test_calibration_review.py	Temporary three-seed records test the CPU reporter's paired arithmetic, validation refit, source/checkpoint/runtime bindings, chronology and score/decision consistency. Altered records are rejected even with refreshed hashes. No GPU execution or public audit is created by the tests.
tests/test_video_versions.py	Tests v4 defaults, selected-source drift, full nonignored inventory and immutable historical v3 blobs. V4 fixtures reject missing calibration claims, wrong evidence/pointers/units/rounding/visible values and seed identities; v3 retains its original 13 requirements.
docs/customer/confidence-calibration.md	Current beginner method/results/use guide: logits, temperature, validation reuse, optional output fields, all-seed measurements and denominators. Explains the fixed-threshold error tradeoff, row 635 and remaining customer-network/live-IDS/hardware limits. Read beside the v4 calibration scenes.
docs/customer/video-course-v4-source.json	Preserved Codex-authored 96-scene source with exact narration, examples, citations, practice, voice and one-hour budgets. Covers the original flow experiment and calibration. Speech, video, captions and documents bind these historical bytes; the later packet study is separate.
docs/research/calibration-code-review.json	Recorded source/study review with inspected hashes, corrected reporter findings, actual tests and remaining limits. Separates code review/CPU arithmetic from new GPU execution and council ratification. Match its source identities before applying its conclusions.
docs/research/improvement-council-review.json	Public ESCALATED / UNRATIFIED council history, bounded evidence gaps and separately authorized direct follow-up. Preserves authority and outcome without claiming consensus or numerical certification.
docs/superpowers/plans/2026-09-13-confidence-and-course.md	Plan for calibration math, prediction integration, GB10 measurements and versioned media/publication. Records fixed fitting/threshold choices, dataset limits and preservation requirements. Execution/review records establish completion; plan boxes alone do not.
docs/customer/evidence/calibration/protocol.json	Protocol frozen before fitting and scaled test evaluation: three checkpoints, validation-only fitting, temperature range, fixed 0.90 policy, metrics and retrospective limits. Later manifests bind it so policy choices cannot silently change after results.

File	Purpose, inputs, outputs and use
docs/customer/evidence/calibration/results.json	Actual all-seed paired temperatures, NLL/Brier/15-bin ECE, accuracy, acceptance/errors and aggregate statistics. Binds source records, chronology, class agreement and FP16 tolerance. Recompute with <code>review_calibration.py --check</code> ; v4 claims use exact pointers here.
docs/customer/evidence/calibration/default-path-check.json	Actual updated-predictor GB10 run with calibration disabled on all 1,041 flows. Records original output fields and unchanged categories alongside small FP16 logit differences. Use its command/source hashes to distinguish compatibility from bitwise equality.
docs/customer/evidence/calibration/gb10-baseline-audit.json	Fresh baseline source/runtime audit, seven CUDA reference comparisons, complete-model optimizer/save/reload check and seed-0 inference. Preserves the failed bitwise expectation and observed tolerance. This is functionality evidence, not another full training run or latency gain.
docs/customer/evidence/calibration/seed0/calibration.json	Seed 0: Fitted positive temperature and checkpoint/class/config/upstream/runtime binding. Produced from this classifier's validation observations; supplied to <code>predict.py --calibration</code> . It is a small postprocessing artifact, not new model weights or a safety certificate.
docs/customer/evidence/calibration/seed0/fit-manifest.json	Seed 0: Fitting command, source/runtime and validation-file identities, saved validation logits/labels, batch coverage and artifact hash. The CPU reviewer repeats the fit from these observations; test labels do not enter the optimization.
docs/customer/evidence/calibration/seed0/fit.log	Seed 0: Console output from validation fitting. Use with <code>fit-manifest.json</code> to trace messages and warnings; the structured manifest supplies the input, timing and output identities.
docs/customer/evidence/calibration/seed0/prediction-manifest.json	Seed 0: Fresh calibrated inference command binding the original classifier, native flows, runtime, fitted artifact and 0.90 policy. Identifies the per-row output and completion; independent arithmetic checks the metrics.
docs/customer/evidence/calibration/seed0/prediction.log	Seed 0: Console output from actual calibrated prediction. Supports troubleshooting and execution trace; read <code>prediction-manifest.json</code> and <code>predictions.json</code> for structured provenance and outputs.
docs/customer/evidence/calibration/seed0/predictions.json	Seed 0: All 1,041 fresh outputs: raw logits/scores, unchanged category, calibrated scores and accept/defer. The reviewer pairs these with separately recorded labels to compute errors. The original browser replay remains unchanged.
docs/customer/evidence/calibration/seed1/calibration.json	Seed 1: Fitted positive temperature and checkpoint/class/config/upstream/runtime binding. Produced from this classifier's validation observations; supplied to <code>predict.py --calibration</code> . It is a small postprocessing artifact, not new model weights or a safety certificate.
docs/customer/evidence/calibration/seed1/fit-manifest.json	Seed 1: Fitting command, source/runtime and validation-file identities, saved validation logits/labels, batch coverage and artifact hash. The CPU reviewer repeats the fit from these observations; test labels do not enter the optimization.
docs/customer/evidence/calibration/seed1/fit.log	Seed 1: Console output from validation fitting. Use with <code>fit-manifest.json</code> to trace messages and warnings; the structured manifest supplies the input, timing and output identities.
docs/customer/evidence/calibration/seed1/prediction-manifest.json	Seed 1: Fresh calibrated inference command binding the original classifier, native flows, runtime, fitted artifact and 0.90 policy. Identifies the per-row output and completion; independent arithmetic checks the metrics.
docs/customer/evidence/calibration/seed1/prediction.log	Seed 1: Console output from actual calibrated prediction. Supports troubleshooting and execution trace; read <code>prediction-manifest.json</code> and <code>predictions.json</code> for structured provenance and outputs.
docs/customer/evidence/calibration/seed1/predictions.json	Seed 1: All 1,041 fresh outputs: raw logits/scores, unchanged category, calibrated scores and accept/defer. The reviewer pairs these with separately recorded labels to compute errors. The original browser replay remains unchanged.

File	Purpose, inputs, outputs and use
docs/customer/evidence/calibration/seed2/calibration.json	Seed 2: Fitted positive temperature and checkpoint/class/config/upstream/runtime binding. Produced from this classifier's validation observations; supplied to predict.py --calibration. It is a small postprocessing artifact, not new model weights or a safety certificate.
docs/customer/evidence/calibration/seed2/fit-manifest.json	Seed 2: Fitting command, source/runtime and validation-file identities, saved validation logits/labels, batch coverage and artifact hash. The CPU reviewer repeats the fit from these observations; test labels do not enter the optimization.
docs/customer/evidence/calibration/seed2/fit.log	Seed 2: Console output from validation fitting. Use with fit-manifest.json to trace messages and warnings; the structured manifest supplies the input, timing and output identities.
docs/customer/evidence/calibration/seed2/prediction-manifest.json	Seed 2: Fresh calibrated inference command binding the original classifier, native flows, runtime, fitted artifact and 0.90 policy. Identifies the per-row output and completion; independent arithmetic checks the metrics.
docs/customer/evidence/calibration/seed2/prediction.log	Seed 2: Console output from actual calibrated prediction. Supports troubleshooting and execution trace; read prediction-manifest.json and predictions.json for structured provenance and outputs.
docs/customer/evidence/calibration/seed2/predictions.json	Seed 2: All 1,041 fresh outputs: raw logits/scores, unchanged category, calibrated scores and accept/defer. The reviewer pairs these with separately recorded labels to compute errors. The original browser replay remains unchanged.

Published v4 coverage and audit records

These preserved records describe the published v4 edition. Consult each actual status, method and artifact hash; later guide edits do not expand their scope. Human full-watch/listening and all-caption acceptance remain pending.

File	What the record establishes and how to use it
docs/customer/video-course-v4-coverage.json	Published source-bound inventory and content/visual map for the complete historical 494-file tree and its guide: individual anchors, paper/CSV teaching, clusters, worked answers and 23 quantities. Binds actual v4 media and technical receipts; it does not cover later packet files.
docs/customer/video-verification-v4.md	Preserved v4 build/review/publication receipt with exact artifacts, commands, observations and limits. Separates computational checks/Codex review from pending human watch/listening and all-caption acceptance. Its seven-question map belongs to that edition.
docs/research/video-audit-v4.json	Preserved bundle binding script/frame review and encoded audio/caption/browser observations to published v4 bytes. Retains uncertainties and scope; cannot borrow v3 acceptance, certify later packet additions or invent a human-listening pass.
docs/research/video-v4/decode.json	Complete actual v4 audiovisual decode, measured stream duration and encoded-audio checks, bound to the MP4 and decoder. Planned seconds or encoder exit alone cannot establish this record.
docs/research/video-v4/browser.json	Observed playback, captions, chapter seeking, viewport layout and download equality for exact page/media bytes. Identifies staged/public URL and browser; does not execute the classifier or establish human listening.
docs/research/video-v4/captions.json	Source/video-bound caption and encoded-speech observations, timing, recognition uncertainty and confirmed defects. Machine checks remain distinct from complete human caption/listening acceptance.
docs/research/video-v4/claims.json	Published v4 visible/spoken quantity review against original flow evidence and calibration results. Binds source/encoded examples with correct population, units and limits; it contains no later packet measurements.

File	What the record establishes and how to use it
docs/research/video-v4/rendered_samples.json	Actual encoded samples for technical terms, identifiers, numbers and chapter transitions, with times/hashes/findings. Supplements all 96 base-frame reviews; selected frames do not equal a full human watch.

Every published v4 download and production record

Use the [active watch route](#) or the [versioned v4 release](#). All format alternatives describe the same preserved flow/calibration lesson. Its records establish the completed technical checks and retain pending human acceptance; later packet results require a separate addendum.

File	Purpose, inputs, outputs and use
docs/customer/demo/video/v4/NetMambaPlus-one-hour-course.mp4	Complete published v4 H.264/AAC lesson assembled from speech, frames, captions, motion and recorded flow replay footage. Use for offline viewing/presentation; historical decode and review bind these preserved bytes.
docs/customer/demo/video/v4/NetMambaPlus-one-hour-course.webm	VP9/Opus browser alternative for the same lesson, with its own hash and codec identity. This is another encoding, not another course or experiment.
docs/customer/demo/video/v4/poster.png	Opening scene image used before playback. Produced from the v4 teaching source, not a live traffic capture.
docs/customer/demo/video/v4/captions.vtt	WebVTT text/times for the optional browser English track and download. Derived from the same narration schedule as burned-in captions; the caption receipt records measured alignment and uncertainties.
docs/customer/demo/video/v4/captions.srt	Equivalent SubRip subtitles for desktop players/editing. Same v4 words and timing, not an alternative script.
docs/customer/demo/video/v4/transcript.md	Complete timed narration, visible content, practices, worked answers and evidence links. Read/search without audio; content must match the source and timeline.
docs/customer/demo/video/v4/chapters.json	Twelve IDs/titles and start/end times from measured narration allocation. Drives navigation and is checked against source/media; it is not independent duration evidence.
docs/customer/demo/video/v4/media-manifest.json	Production identity record for source/version/voice, scenes, cues, references and output sizes/hashes. Feeds page/documents/verifiers; a build record alone does not certify quality.
docs/customer/demo/video/v4/narration-manifest.json	Public identities/timing for all spoken passages and final audio, including samples, voice/client and unavailable cloud-model revision. The MP4 contains public speech; raw caches remain local.
docs/customer/demo/video/v4/motion-manifest.json	Timed geometry/meaning of drawings, emphasis, reveals and recorded clip. Lets reviewers distinguish illustrative animation from actual predictions.
docs/customer/demo/video/v4/NetMambaPlus-course-slides.pptx	One image slide per v4 scene, with editable narration/evidence notes. Use for presenting/adapting the explanation. Artwork is not editable diagram shapes; the original 16-slide deck is separate history.
docs/customer/demo/video/v4/NetMambaPlus-course-slides.pdf	Fixed-layout PDF of the same 96 scene images with scene navigation. Use to present/print the checked layout without PowerPoint rendering differences.
docs/customer/demo/video/v4/NetMambaPlus-course-handbook.pdf	Searchable v4 narration, examples, code/tables and references plus the historical 494-file guide appendix. Read for deeper detail; source/walkthrough/document hashes identify the included revision, which predates the packet study.
docs/customer/demo/video/v4/companion-manifest.json	Source/walkthrough/frame hashes, slide count and output identities for the PPTX and two PDFs. Produced by the document builder; actual page/layout checks remain separate.

File	Purpose, inputs, outputs and use
docs/customer/demo/video/v4/manifest.json	Final validation summary connecting build, companions, coverage and public audit records. Separate from production media-manifest.json; statuses require actual receipts and retain pending human review.

Published v4 chapter, scene and frame map

Each PNG consumes the visible fields of its named source scene and supplies the base image for the MP4 and matching document slide. Motion, recorded footage and captions are added during encoding, so a base PNG is not every encoded frame. These 96 entries use the finalized measured-narration timeline; times are rounded to milliseconds. The final media manifest and encoded checks must agree with it.

Chapter 1: Start here: what you built · 00:00.000-04:41.000

Scene and base image	What the individual visual teaches
docs/customer/demo/video/v4/frames/orientation-01.png · 00:00.000-00:36.533	A research lab you can explain. Paper → research proposal; Repository → executable experiment; Evidence → what actually happened.
docs/customer/demo/video/v4/frames/orientation-02.png · 00:36.533-01:17.200	The result to remember. Three full fine-tuning runs on compatible flows; Mean test accuracy: 86.65%; Paper result: 97.50% — not reproduced.
docs/customer/demo/video/v4/frames/orientation-03.png · 01:17.200-01:55.367	Packet, header and payload. Packet: a small unit sent across a network; Header: delivery and protocol information; Payload: carried content, possibly encrypted.
docs/customer/demo/video/v4/frames/orientation-04.png · 01:55.367-02:34.100	A flow needs a grouping rule. Related packets must be grouped; Their order and timing matter; Adjacent CSV rows do not prove a connection.
docs/customer/demo/video/v4/frames/orientation-05.png · 02:34.100-03:14.500	Classifier, detector and response. Classifier: select a category; IDS: observe traffic and manage alerts; Response: investigate or enforce a policy.
docs/customer/demo/video/v4/frames/orientation-06.png · 03:14.500-03:54.900	How to read an evidence claim. Implemented: code exists; Executed: a recorded command ran; Validated: a named check passed.
docs/customer/demo/video/v4/frames/orientation-07.png · 03:54.900-04:31.000	Your presentation has an evidence trail. Watch this course to learn the story; Use the transcript to explain it again; Use the file guide to find each underlying record.
docs/customer/demo/video/v4/frames/orientation-08.png · 04:31.000-04:41.000	Your turn · 10 seconds. Practice: Does a working classifier prove a complete live IDS?. Declared silent practice, not additional model execution.

Chapter 2: The PDF: ideas, experiments and boundaries · 04:41.000-09:55.000

Scene and base image	What the individual visual teaches
docs/customer/demo/video/v4/frames/paper-01.png · 04:41.000-05:23.867	What the attached PDF is. arXiv 2601.21792v1 · January 2026; Traffic classification research; A framework with more than one model variant.
docs/customer/demo/video/v4/frames/paper-02.png · 05:23.867-06:10.233	Three problems motivate the paper. Computation cost of long sequences; Information lost or biased during extraction; Rare categories hidden by class imbalance.
docs/customer/demo/video/v4/frames/paper-03.png · 06:10.233-06:49.733	What multimodal means here. Bytes: content patterns; Sizes: transmission shape; Intervals: transmission rhythm.
docs/customer/demo/video/v4/frames/paper-04.png · 06:49.733-07:30.567	Mamba and the alternative backbone. Mamba: input-dependent state updates; NetTrans: a separate attention-based alternative; Our run uses the Mamba classifier.
docs/customer/demo/video/v4/frames/paper-05.png · 07:30.567-08:12.333	Read the paper in this order. Framework and representation: sections IV-V; Model and learning details: section VI; System and measurements: sections VII-VIII.
docs/customer/demo/video/v4/frames/paper-06.png · 08:12.333-08:54.967	A table is not our result. Paper Table IV: 97.50% for CICIoT2022; Paper also studies other tasks and settings; Our three-run mean: 86.65%.
docs/customer/demo/video/v4/frames/paper-07.png · 08:54.967-09:45.000	Two details that prevent overclaiming. Paper byte sequence: 401 tokens; Our multimodal sequence: 443 tokens; Paper online throughput is a separate NetMamba prototype.
docs/customer/demo/video/v4/frames/paper-08.png · 09:45.000-09:55.000	Your turn · 10 seconds. Practice: Name one paper claim that has not been reproduced here.. Declared silent practice, not additional model execution.

Chapter 3: The first CSV: CICIDS2017 packet records · 09:55.000–14:53.000

Scene and base image	What the individual visual teaches
docs/customer/demo/video/v4/frames/cic-csv-01.png · 09:55.000–10:35.633	A CSV is a table, not a running database. Rows: individual records; Columns: named fields; This file: 1,410,255 rows and 1,505 columns.
docs/customer/demo/video/v4/frames/cic-csv-02.png · 10:35.633–11:16.867	One row contains five kinds of metadata. Compares Field group, Recorded meaning across payload_byte_1 ... 1500, ttl, total_len, protocol / t_delta, label.
docs/customer/demo/video/v4/frames/cic-csv-03.png · 11:16.867–12:00.933	Benign and attack labels are imbalanced. Compares Recorded label, Rows across BENIGN, DoS Hulk, DDoS, DoS GoldenEye, DoS slowloris.
docs/customer/demo/video/v4/frames/cic-csv-04.png · 12:00.933–12:40.667	Why the small classes matter. Compares Recorded label, Rows across SSH-Patator, FTP-Patator, Bot, PortScan, Web Attack – Sql Injection.
docs/customer/demo/video/v4/frames/cic-csv-05.png · 12:40.667–13:23.200	Odd values are questions to investigate. Some t_delta values are negative; Minimum observed: –0.000018; Some total_len values exceed 1,500.
docs/customer/demo/video/v4/frames/cic-csv-06.png · 13:23.200–14:03.367	What the profiling actually checked. Full-file row counts and metadata aggregates; Label and protocol distributions; Payload values were not exhaustively validated.
docs/customer/demo/video/v4/frames/cic-csv-07.png · 14:03.367–14:43.000	The missing information changes the task. No established addresses, ports or flow ID; No established order within each flow; Payload-only rows cannot establish header semantics.
docs/customer/demo/video/v4/frames/cic-csv-08.png · 14:43.000–14:53.000	Your turn · 10 seconds. Practice: Why is matching a tensor shape insufficient to establish compatible flow data?. Declared silent practice, not additional model execution.

Chapter 4: The second CSV and the choice of flow data · 14:53.000–19:55.000

Scene and base image	What the individual visual teaches
docs/customer/demo/video/v4/frames/unsw-csv-01.png · 14:53.000–15:32.200	The UNSW extract has the same broad schema. 79,881 rows; 1,500 payload columns + 5 metadata fields; 10 label names.
docs/customer/demo/video/v4/frames/unsw-csv-02.png · 15:32.200–16:13.100	Read the UNSW label distribution. Compares Recorded label, Rows across normal, generic, exploits, fuzzers, reconnaissance.
docs/customer/demo/video/v4/frames/unsw-csv-03.png · 16:13.100–16:53.900	Rare labels and protocol variety. Compares Recorded label, Rows across dos, backdoor, analysis, shellcode, worms.
docs/customer/demo/video/v4/frames/unsw-csv-04.png · 16:53.900–17:37.533	Related subject, different learning problems. Compares Input collection, Unit / category count across Uploaded CICIDS2017, Uploaded UNSW, Authors' CICIoT2022 release.
docs/customer/demo/video/v4/frames/unsw-csv-05.png · 17:37.533–18:19.567	Why the actual experiment used other data. Train: 8,323 flows · Validation: 1,040 flows; Test: 1,041 flows · 6 native classes; Record the departure from the uploaded CSVs.
docs/customer/demo/video/v4/frames/unsw-csv-06.png · 18:19.567–19:04.167	Compatible data still has limitations. 5 identical stored inputs across train / validation; 6 across train / test; Capture independence and pretraining exposure remain unknown.
docs/customer/demo/video/v4/frames/unsw-csv-07.png · 19:04.167–19:45.000	The defensible connection between the files. The paper defines a flow representation; The CSVs reveal an input-contract mismatch; The repository tests a compatible native route.
docs/customer/demo/video/v4/frames/unsw-csv-08.png · 19:45.000–19:55.000	Your turn · 10 seconds. Practice: Which data was actually used for training, and why were the uploaded CSVs not used?. Declared silent practice, not additional model execution.

Chapter 5: Follow one native flow into the model · 19:55.000-24:46.000

Scene and base image	What the individual visual teaches
docs/customer/demo/video/v4/features/features-01.png · 19:55.000-20:32.400	The actual input is a list of flow objects. Shows the source example/command: [{ . data: packet byte strings; sizes and intervals: numeric strings; label and name: known training category
docs/customer/demo/video/v4/features/features-02.png · 20:32.400-21:13.067	Bytes: five packets, 320 stored bytes each. $5 \times 320 = 1,600$ stored byte positions; Short inputs are padded; long inputs are truncated; Paper header/payload extraction is not independently verified.
docs/customer/demo/video/v4/features/features-03.png · 21:13.067-21:50.700	Normalization changes the numerical scale. Shows the source example/command: $\text{normalized_byte} = (\text{byte} / 255 - 0.5) / 0.5$. $\text{byte} / 255 \rightarrow$ range 0 to 1; Subtract 0.5, divide by 0.5 $\rightarrow$ range -1 to 1; Tensor shape: batch $\times 1 \times 1 \times 1,600$
docs/customer/demo/video/v4/features/features-04.png · 21:50.700-22:30.200	Sizes and intervals use their own rules. Compares Example input, Transformed value across Observed gap $x = 0$, Observed gap $x = 1$, Padded interval, Size larger than 1,500.
docs/customer/demo/video/v4/features/features-05.png · 22:30.200-23:05.900	The 443-token calculation. 400 byte tokens; 20 size + 20 interval tokens; 3 summary tokens $\rightarrow$ 443 total tokens.
docs/customer/demo/video/v4/features/features-06.png · 23:05.900-23:57.333	Four blocks produce six raw outputs. 4 Mamba blocks · width 256; Combine the 3 summary vectors; Classifier head $\rightarrow$ 6 logits.
docs/customer/demo/video/v4/features/features-07.png · 23:57.333-24:36.000	Explain the input contract back. Meaning and origin must match; Dimensions and transforms must match; Class meanings must match the checkpoint.
docs/customer/demo/video/v4/features/features-08.png · 24:36.000-24:46.000	Your turn · 10 seconds. Practice: Describe the three feature views and the six-number output.. Declared silent practice, not additional model execution.

Chapter 6: How pretraining, training and inference work · 24:46.000-29:43.000

Scene and base image	What the individual visual teaches
docs/customer/demo/video/v4/frames/learning-01.png · 24:46.000-25:25.600	Three activities, three different outputs. Pretraining → general representation weights; Fine-tuning → task-specific classifier; Inference → predictions for new inputs.
docs/customer/demo/video/v4/frames/learning-02.png · 25:25.600-26:05.767	Pretraining hides part of the input. Mask byte content and selected feature values; Reconstruct the missing information; Compare reconstruction with the original values.
docs/customer/demo/video/v4/frames/learning-03.png · 26:05.767-26:44.033	What our pretraining check actually did. 2 completed epochs · 132 updates; Loss: 9.51953 → 6.12702; Separate from the checkpoint used for full fine-tuning.
docs/customer/demo/video/v4/frames/learning-04.png · 26:44.033-27:25.667	A labeled batch changes the classifier. Forward pass → predictions; Loss → mismatch with known labels; Backward pass + optimizer → updated parameters.
docs/customer/demo/video/v4/frames/learning-05.png · 27:25.667-28:08.333	Three splits prevent one obvious shortcut. Training split: update parameters; Validation split: choose the checkpoint; Test split: measure the selected classifier.
docs/customer/demo/video/v4/frames/learning-06.png · 28:08.333-28:51.533	The completed training protocol. Seeds 0, 1 and 2; 120 epochs and 7,920 updates per run; Batch 128; effective learning rate 0.001.
docs/customer/demo/video/v4/frames/learning-07.png · 28:51.533-29:33.000	A successful reload has a narrow meaning. Strict loading checks all expected tensors; Model state and class meanings must stay together; New accuracy requires known labels.
docs/customer/demo/video/v4/frames/learning-08.png · 29:33.000-29:43.000	Your turn · 10 seconds. Practice: What does each of pretraining, fine-tuning and inference produce?. Declared silent practice, not additional model execution.

Chapter 7: The main code: follow a real execution path · 29:43.000–34:26.000

Scene and base image	What the individual visual teaches
docs/customer/demo/video/v4/frames/code-01.png · 29:43.000–30:25.567	Our code surrounds the authors' code. Acquire and verify the original source; Invoke the original loader and model; Record execution and check its outputs.
docs/customer/demo/video/v4/frames/code-02.png · 30:25.567–31:04.833	repro.py is the experiment coordinator. Shows the source example/command: python repro.py fetch. fetch and validate; pretrain and finetune; Fresh output directories and run manifests
docs/customer/demo/video/v4/frames/code-03.png · 31:04.833–31:40.800	evaluate.py answers: how accurate is this model?. Labeled native test flows; Strict saved-classifier loading; Native evaluation metrics.
docs/customer/demo/video/v4/frames/code-04.png · 31:40.800–32:20.300	Predict a class, then calibrate its confidence. predict.py keeps raw logits and class names; calibration.py adjusts scores with one temperature; tools/calibrate.py fits on validation labels only.
docs/customer/demo/video/v4/frames/code-05.png · 32:20.300–32:58.700	replay.py connects execution to explanation. Run labeled inference; Save per-flow results and independent metrics; Generate a browser page from those records.
docs/customer/demo/video/v4/frames/code-06.png · 32:58.700–33:42.500	Configurations and assets define the recipe. configs: settings, hashes and class mapping; assets: downloaded data and weights, kept locally; upstream/NetMambaPlus: pinned source, kept locally.
docs/customer/demo/video/v4/frames/code-07.png · 33:42.500–34:16.000	Read a run in this order. Manifest: what was requested and what completed; Log: what execution reported; Metrics and predictions: what was measured.
docs/customer/demo/video/v4/frames/code-08.png · 34:16.000–34:26.000	Your turn · 10 seconds. Practice: Which scripts handle unlabeled prediction and labeled evaluation? Where is run provenance recorded?. Declared silent practice, not additional model execution.

Chapter 8: Read the actual results, including the mistakes · 34:26.000-39:21.000

Scene and base image	What the individual visual teaches
docs/customer/demo/video/v4/frames/results-01.png · 34:26.000-35:05.667	Three runs, one fixed test set. Seed 0: 91.26% · 950 / 1,041 correct; Seed 1: 84.05% · 875 / 1,041 correct; Seed 2: 84.63% · 881 / 1,041 correct.
docs/customer/demo/video/v4/frames/results-02.png · 35:05.667-35:43.567	Mean and spread answer different questions. Mean: 86.65%; Sample standard deviation: 4.00 percentage points; Not a confidence interval or an independent holdout study.
docs/customer/demo/video/v4/frames/results-03.png · 35:43.567-36:22.867	Accuracy, precision, recall and F1. Accuracy: correct predictions / all predictions; Precision: how often a predicted category is right; Recall: how much of a true category was found.
docs/customer/demo/video/v4/frames/results-04.png · 36:22.867-37:02.033	Read a confusion matrix. Rows: known category; Columns: predicted category; Off-diagonal cells are mistakes.
docs/customer/demo/video/v4/frames/results-05.png · 37:02.033-37:46.667	Fresh checks ran on the actual GB10. 7 CUDA operator comparisons passed; Complete model: real optimizer step and strict reload; 1,041 classes match; tiny FP16 logit differences remain.
docs/customer/demo/video/v4/frames/results-06.png · 37:46.667-38:27.333	The paper gap is still an open question. Paper: different runtime and training conditions; Released checkpoint history is incomplete; No experiment has isolated the gap's cause.
docs/customer/demo/video/v4/frames/results-07.png · 38:27.333-39:11.000	Measured confidence improves; accuracy is unchanged. Compares Confidence metric, Raw mean, Calibrated mean across NLL (natural logs), Brier score, ECE (%).
docs/customer/demo/video/v4/frames/results-08.png · 39:11.000-39:21.000	Your turn · 10 seconds. Practice: Explain the mean and spread without claiming customer-network validation.. Declared silent practice, not additional model execution.

Chapter 9: Demonstrate one prediction, then reveal an error · 39:21.000-44:00.000

Scene and base image	What the individual visual teaches
docs/customer/demo/video/v4/frames/demo-01.png · 39:21.000-39:56.600	Open the recorded replay. Runs without a GPU or Python; Displays saved seed-0 predictions; Opening it does not run new inference.
docs/customer/demo/video/v4/frames/demo-02.png · 39:56.600-40:36.367	Choose the demo model before inspecting outcomes. Seed 0 is the declared demonstration model; All three seeds remain visible in the results; A convenient demo is not a new evaluation protocol.
docs/customer/demo/video/v4/frames/demo-03.png · 40:36.367-41:10.767	Inspect JSON row 635. Browser display row: 0636; Highest score: RTSP Brute Force; Display score: 84.43%, not a calibrated attack probability.
docs/customer/demo/video/v4/frames/demo-04.png · 41:10.767-41:46.533	Reveal the known label. True category: Other; Predicted category: RTSP Brute Force; This confident-looking prediction is wrong.
docs/customer/demo/video/v4/frames/demo-05.png · 41:46.533-42:27.433	Use the optional calibrated inference path. Shows the source example/command: python predict.py \. Fit calibration.json from validation data first; Keep original class, logits and raw scores; Add calibrated scores and accept/defer decisions
docs/customer/demo/video/v4/frames/demo-06.png · 42:27.433-43:14.133	Accept or defer is a review policy. Raw scores: 620 accepted flows; 9 wrong flows; Calibrated: 893 accepted flows; 37 wrong flows; Fewer deferrals, more accepted errors.
docs/customer/demo/video/v4/frames/demo-07.png · 43:14.133-43:50.000	Practice the demonstration handoff. Show the replay and one actual error; State what runs locally and what is recorded; Separate a category score from an alert policy.
docs/customer/demo/video/v4/frames/demo-08.png · 43:50.000-44:00.000	Your turn · 10 seconds. Practice: Why was the 84.43% display score insufficient to trust this prediction?. Declared silent practice, not additional model execution.

Chapter 10: Set it up, use it and test the right layer · 44:00.000-48:45.000

Scene and base image	What the individual visual teaches
docs/customer/demo/video/v4/frames/setup-01.png · 44:00.000-44:38.067	Choose one of three setup routes. Present: browser or document viewer; Recheck: Git and Python; Execute the model: checked Linux CUDA environment.
docs/customer/demo/video/v4/frames/setup-02.png · 44:38.067-45:15.700	Clone and run the portable checks. Shows the source example/command: <code>git clone https://github.com/buffbeefalo/netmambaplus-reproduction.git</code> . Clone the current repository; Run the unit suite and package checks; These checks do not download or train the model
docs/customer/demo/video/v4/frames/setup-03.png · 45:15.700-45:55.000	What cross-platform CI establishes. Linux x86-64, Windows x86-64, macOS ARM64; Python 3.10 and 3.12: six combinations; Hosted jobs run portable checks, not GPU training.
docs/customer/demo/video/v4/frames/setup-04.png · 45:55.000-46:37.300	The model environment has native dependencies. Python 3.12 · Torch 2.9.1 + CUDA 13.0; Pinned Mamba and causal-convolution extensions; Compiler, driver and GPU architecture must agree.
docs/customer/demo/video/v4/frames/setup-05.png · 46:37.300-47:18.733	Pass two gates before a new experiment. Shows the source example/command: <code>python tools/check_gpu_runtime.py \</code> . Numerical check: forward and backward comparisons; Complete-model check: update, save, reload, infer; Keep the build report with the runtime results
docs/customer/demo/video/v4/frames/setup-06.png · 47:18.733-48:00.100	Know what is still platform-specific. GB10: measured training and inference; Other NVIDIA GPUs: build route, machine-specific validation needed; CPU, Apple MPS, NPU: no validated model backend.
docs/customer/demo/video/v4/frames/setup-07.png · 48:00.100-48:35.000	Match the test to the claim. Document checks → package integrity; GPU gates → execution correctness within their scope; Labeled holdout → measured predictive performance.
docs/customer/demo/video/v4/frames/setup-08.png · 48:35.000-48:45.000	Your turn · 10 seconds. Practice: Which setup is enough to present, and which setup is needed for new model predictions?. Declared silent practice, not additional model execution.

Chapter 11: Every repository area and every download · 48:45.000-53:50.000

Scene and base image	What the individual visual teaches
docs/customer/demo/video/v4/frames/repository-01.png · 48:45.000-49:25.800	Use the file guide as the complete index. One entry for every tracked file; Separate explanations for local downloads and run outputs; Links take you from an explanation to its evidence.
docs/customer/demo/video/v4/frames/repository-02.png · 49:25.800-50:07.167	Root files, configuration and dependencies. README and four entry points: start and execute; configs: experiment recipes and identities; requirements: packages grouped by activity.
docs/customer/demo/video/v4/frames/repository-03.png · 50:07.167-50:54.033	Tools: acquire, build, measure and calibrate. Acquire assets and profile the two CSVs; Build CUDA, test operators and check the full model; Fit calibration; measure confidence and review coverage.
docs/customer/demo/video/v4/frames/repository-04.png · 50:54.033-51:35.367	Tools: documents, replay and this video. Package and guide builders produce teaching artifacts; Replay rendering and recording preserve demonstrations; Narration, video and page builders share one lesson source.
docs/customer/demo/video/v4/frames/repository-05.png · 51:35.367-52:22.333	Tests, workflows, documents and evidence. tests: controlled regression cases; workflows: portable CI and Pages publication; docs: explanations, evidence, audits and research records.
docs/customer/demo/video/v4/frames/repository-06.png · 52:22.333-53:06.267	Choose the right download. Current course: one MP4, transcript, captions and companion slides; Historical customer bundle: fixed original experiment package; Current support matrix: latest tested setup boundaries.
docs/customer/demo/video/v4/frames/repository-07.png · 53:06.267-53:40.000	A practical reading order. README → course → file guide; Runbook and support matrix before executing commands; Results → manifests → logs when checking a claim.
docs/customer/demo/video/v4/frames/repository-08.png · 53:40.000-53:50.000	Your turn · 10 seconds. Practice: Where would you verify a platform claim and an actual training result?. Declared silent practice, not additional model execution.

Chapter 12: Hardware roadmap, missing work and your handoff - 53:50.000-60:00.000

Scene and base image	What the individual visual teaches
docs/customer/demo/video/v4/frames/hardware-01.png · 53:50.000-54:28.367	Keep the measured timing boundary visible. Batch 1 median: 0.881265 ms; Measured on GB10 after warmup; Excludes capture, flow waiting, preprocessing and transfers.
docs/customer/demo/video/v4/frames/hardware-02.png · 54:28.367-55:04.833	A possible SmartNIC and NPU division. NIC / SmartNIC: packet handling and flow assembly; Host: orchestration, buffering and policy; NPU: supported neural computation after a verified port.
docs/customer/demo/video/v4/frames/hardware-03.png · 55:04.833-55:48.233	Weights are not a portable computation graph. Model-only export: saved parameter tensors; Graph export: represent the computation for another runtime; Tested strict graph capture failed at a custom CUDA operator.
docs/customer/demo/video/v4/frames/hardware-04.png · 55:48.233-56:28.433	A defensible hardware validation process. Check full-model output and gradient parity where relevant; Measure quantization accuracy on labeled data; Measure throughput, tail latency, buffering and drops.
docs/customer/demo/video/v4/frames/hardware-05.png · 56:28.433-57:12.800	What remains missing. Matching paper accuracy and full pretraining; Independent calibration and customer-network validation; Live extraction, unknown attacks and hardware deployment.
docs/customer/demo/video/v4/frames/hardware-07.png · 57:12.800-57:51.467	Teach the project to someone else. What data went in, and why?; What was executed and measured?; What remains before customer deployment?.
docs/customer/demo/video/v4/frames/hardware-08.png · 57:51.467-59:21.467	Your turn · 90 seconds. Practice: Explain the data choice, tested result, repository and remaining deployment work.. Declared silent practice, not additional model execution.
docs/customer/demo/video/v4/frames/hardware-06.png · 59:21.467-60:00.000	Your concise explanation. We built a checked execution and evidence layer; The original model trained on compatible native flows; Measured results and limits are both available.

17. Historical v3 course and shared production tools

The preserved v3 source describes the earlier one-hour, 12-chapter, 96-scene lesson. Its original source, media and reviews remain immutable. Shared tools below describe current behavior; new authoring and evidence belong to v4. The [scene map](#) includes stable IDs and the narration-backed schedule. The generated chapters.json, transcript and verification receipt bind those locations to the encoded media.

Codex authors and directs the lesson. The selected Microsoft en-US-AndrewMultilingualNeural voice renders that text through edge-tts 7.2.7. This is a network speech service, not local Kokoro synthesis, a Codex voice, or a local language model writing narration. The service does not expose an immutable model revision. Retained audio, word events, client/settings records and file hashes establish which rendering was used; they cannot promise that a later request returns identical sound.

The production chain has distinct inputs and outputs:

1. The versioned source supplies narration, visible examples, evidence references, practice instructions and production settings. The paper/data companion supplies deeper explanations; this walkthrough supplies individual-file detail.

2. The neural narrator sends the reviewed text for speech rendering, validates returned word boundaries against the original text, and writes reusable audio/timing caches. It neither researches nor rewrites the script.
3. The video builder combines those measured caches with teaching frames, diagrams, captions and a bounded recorded-replay excerpt. It writes the encoded lesson and separate build manifests. Animated packets and highlights explain a concept; they are not live model outputs.
4. The document builder uses the same scene frames and narration for slide images, editable speaker notes and a searchable handbook. Its --work-dir must point to the frame directory from the matching video build.
5. Technical checks, content/visual reviews and publication observations produce separate records. Coverage links those records to the exact source, companion and media. Only the checked current version is activated at the one watch route.

Source, supporting explanations and production code

File	Purpose, inputs, outputs and relationship
docs/customer/video-course-v3-source.json	Immutable earlier 96-scene v3 source: exact narration, visible fields, citations, voice, practice and animation settings. Historical verification reads the pinned release snapshot; calibration teaching and new revisions belong in video-course-v4-source.json.
docs/customer/paper-and-data-explained.md	Deeper companion for the attached paper and both packet CSVs. Explains all major paper sections, each CSV field and label distribution, missing flow context, native data, loader transformations, training, results and source/paper discrepancies. It links the paper, pinned upstream code and recorded evidence. Read it alongside the paper/CSV/features chapters rather than expecting filenames alone to teach those topics.
docs/customer/video-figures/recorded-replay-controls.png	Captured view of the saved-prediction replay controls, used by scene demo-01. Its source field carries a SHA-256 so the image cannot silently change during rendering. It illustrates a recorded browser interface; it is not packet capture or new inference.
requirements/video-neural.txt	Pins the edge-tts client used for Andrew speech in v3/v4. Input is reviewed text and voice settings; the service and FFmpeg are separate requirements. This does not obtain or pin the provider model weights.
requirements/video-documents.txt	Pins python-pptx, ReportLab and Pillow for course slides/PDFs and images. Builders consume matching source, frames and this guide; they do not train the classifier or synthesize speech. Fonts and FFmpeg are separate prerequisites.
tools/narrate_video_neural.py	Reads --source (v4 by default), voice settings and exact text. Sends one bounded service call per uncached scene, validates complete returned word alignment, converts MP3 to 24 kHz mono WAV and writes events, scene identities and narration.json. Exact text/voice/client/conversion/file checks protect reused caches; source drift fails. --only-scene writes a labeled partial manifest; no automatic request retry occurs.
tools/video_motion.py	Delegates the new packet diagrams to packet_video_motion.py while preserving earlier flow diagrams. Given timed scenes and narration cues, emits bounded ASS drawing events for moving packets/tokens, progress, progressive outlines and measured-result bar reveals. Optional focus phrases locate a measured narration passage. The video builder combines this illustrative motion with captions; the module performs no network classification.
tools/build_video_documents.py	Requires --source and its matching media manifest, reviewed references and frames. Writes the same frame artwork into a PowerPoint with editable narration notes, a slide PDF and a searchable handbook with this guide. Rejects source/reference/frame drift and records output hashes. Slide artwork is an image; actual page/layout checks are separate.
tools/verify_video_course_v3.py	Shares strict coverage/claim/media checks and immutable snapshot handling with v4. Its default CLI reads all 341 files at commit 71290440e917105ffa094402d94e24d7e29a3549 as data, rejecting protected v3 drift and retaining 13 claims. V4 selects its own 494-file edition and ten additional calibration claims. --current preserves current-tree checks; no historical code executes.
tests/test_video_neural.py	Offline fixtures check punctuation/acronym alignment, caption limits, invalid word times, cache/audio identity, request failures and explicit-source selection/drift. No CI service call occurs; subjective pronunciation quality requires a different review.
tests/test_video_motion.py	Checks narration-anchored emphasis after tempo conversion, event bounds, disabled legacy motion, recorded-clip hashes/geometry/duration and handbook link/anchor handling. Its identity-only clip fixture is explicitly not playable-media evidence.
tests/test_course_video_v3.py	Acceptance fixtures reject incomplete inventory, missing paper/CSV explanations, stale review/audio/media identities, wrong scientific units or attribution, caption defects and absent required checks. Positive fixtures cover declared practice and equivalent quantity formatting. These tests keep content-only checks distinct from publication readiness; real decode and browser checks have separate receipts.

File	Purpose, inputs, outputs and relationship
docs/research/video-v3-council-decision.json	Exact consensus decision defining the expanded course, evidence protection, coverage and acceptance contract. It requires a separate v3 source/media/receipt and preserves the older escalated video review. A ratified plan is not evidence that the final video, documents or release have passed their checks.

Where the paper and CSV detail lives

The paper chapter introduces the problem, representations, model and experimental boundaries. For individual sections, follow the [paper reading map](#), which includes ablations, few-shot/OOD experiments and the authors' online prototype. These were not all reproduced here. The [optional technical notes](#) explain details such as the paper's 401-token table versus this classifier's 443 tokens and the interval-formula notation discrepancy.

For the CSV chapters, read the [complete field explanation and label counts](#) with the hashed metadata profile. One packet row has byte slots and metadata; it does not establish a flow's membership or order. The [native-flow and model walkthrough](#) then explains the separately acquired data that actually entered the experiment. This companion treatment is deeper than a brief on-screen table or a link in the video.

Public coverage and review records

These files form the final evidence chain. A pending record is allowed while generation or review continues, and its presence is not a passing result. Read each record's method, status, scope and reviewed hashes. Completed checks must be backed by actual observations; unresolved work remains visible.

File	What the completed record must establish
docs/customer/video-course-v3-coverage.json	The actual inventory snapshot and every file's companion anchor/teaching scene; substantive paper/CSV coverage; reviewed teaching-cluster minima; and source-bound script, worked-answer and visual reviews. Scene IDs are stable; final chapter locations must come from measured media.
docs/customer/video-verification-v3.md	The separate human-readable v3 receipt: actual build/check commands, media and document identities, completed reviews, unresolved defects and publication observations. It must retain pending human full-watch/all-caption acceptance rather than borrow v2 status.
docs/research/video-audit-v3.json	The public v3 audit bundle linking the standalone checks and content/visual review records. It binds those observations to the final versions and preserves the older v2 escalation as history.
docs/research/video-v3/decode.json	Actual complete audiovisual decode, stream/runtime and encoded-audio measurements. Scheduled seconds and a successful encoder exit alone do not establish this check.
docs/research/video-v3/browser.json	Browser playback, chapter seeking, captions, responsive layout and download checks against exact page/media bytes. Must state whether the tested URL was staged or publicly deployed.
docs/research/video-v3/captions.json	Canonical text coverage, timing/provenance, ordering/bounds and readable rendered-caption review against the final audio. Structural checks and sampled inspection must not be described as human acceptance of every word.
docs/research/video-v3/claims.json	Scientific quantities and explanations compared with the authoritative paper, profiles and experiment records, using explicit units and rounding. Correct digits without correct attribution or meaning are insufficient.

File	What the completed record must establish
docs/research/video-v3/rendered_samples.json	Which rendered technical terms, identifiers, numbers and chapter transitions were inspected, the method used, the exact artifacts and any defects. Sampling has a narrower scope than a complete human watch-through.
docs/customer/demo/video/v3/manifest.json	Final validation summary linking the measured build, companion, coverage and public audit records. It is separate from media-manifest.json, which describes production outputs. Neither summary may manufacture completed checks from a pending placeholder.

Preserved v3 outputs and their relationships

The following are preserved v3 files, checked against their pinned release snapshot. Their original statuses do not certify v4. The coverage record checks every actual path, including additions or renamed files. All these outputs belong to the same lesson; format alternatives and the short source capture do not create another promoted course.

Repository path	Output and relationship
docs/customer/demo/video/v3/NetMambaPlus-one-hour-course.mp4	The complete H.264/AAC narrated lesson with captions and instructional motion. Actual duration, decode, audio and content checks are required before activation.
docs/customer/demo/video/v3/NetMambaPlus-one-hour-course.webm	VP9/Opus browser encoding derived from that checked MP4 when a fallback is needed. It represents the same lesson and has its own byte identity.
docs/customer/demo/video/v3/poster.png	The opening scene image used while the active player is loading or paused before playback.
docs/customer/demo/video/v3/captions.vtt	Final WebVTT captions referenced through the active page's same-origin v3/ path.
docs/customer/demo/video/v3/captions.srt	Equivalent final caption text/timing for desktop subtitle players and editing software.
docs/customer/demo/video/v3/transcript.md	Complete timed narration, practice instructions and worked answers, visible teaching points, source links and production attribution.
docs/customer/demo/video/v3/chapters.json	Chapter IDs/titles and measured start/end locations for navigation, checked against the source and media.
docs/customer/demo/video/v3/media-manifest.json	Build inventory of source/version/voice settings, timing, scenes, captions, evidence references and artifact sizes/hashes. Feeds page generation and technical checks; it is not the final validation receipt.
docs/customer/demo/video/v3/narration-manifest.json	Published narration identity and timing records copied from the successful rendering cache. Identifies the synthetic voice/client and unavailable provider-model revision without redistributing the provider's model.
docs/customer/demo/video/v3/motion-manifest.json	Bounded scene-level drawing/clip events and their instructional meanings, so animation can be reviewed separately from the scientific predictions.
docs/customer/demo/video/v3/NetMambaPlus-course-slides.pptx	One slide image per scene with editable narration/evidence notes. This is the expanded course's deck, separate from the original 16-slide customer presentation.
docs/customer/demo/video/v3/NetMambaPlus-course-slides.pdf	Fixed-layout PDF of those same scene images, with scene bookmarks for presenting or printing.
docs/customer/demo/video/v3/NetMambaPlus-course-handbook.pdf	Searchable narration, examples and references plus this reviewed individual-file appendix. The handbook supplies detail that cannot all be spoken in an hour.
docs/customer/demo/video/v3/companion-manifest.json	Source/walkthrough/frame identities, slide count and hashes of the three newly generated companion documents. Actual page/render checks remain separate.

Repository path	Output and relationship
docs/customer/video-figures/record-ed-controls.webm	Original short replay-controls recording, used as bounded supporting footage in v3/v4 demo-01. Hash and placement are checked. It displays saved original predictions, not new calibration, live capture or another standalone lesson.

Chapter, scene and individual frame map

Each PNG under `docs/customer/demo/video/v3/frames/` is named after its stable scene ID. It is the rendered base teaching image consumed by the video and matching companion slide. Timed motion, recorded footage, captions and countdowns can be added during encoding, so one PNG does not represent every later video frame. The rows below explain each base visual and the part of the lesson it belongs to. These 96 historical PNGs remain bound to the original v3 frame hashes and visual review. The locations below come from the final measured-narration schedule; scene times are rounded to the nearest millisecond. The media checks verify the encoded lesson against that schedule. The final chapter intentionally places its concise answer after the teach-back and 90-second practice, while retaining scene ID hardware-06.

Chapter 1: Start here: what you built · 00:00–04:43

File and explanation links: [project structure](#) and [vocabulary](#).

Scene and base frame	What this individual visual teaches
orientation-01.png · 00:00.000–00:36.800	A research lab you can explain. Paper → research proposal.
orientation-02.png · 00:36.800–01:17.767	The result to remember. Three full fine-tuning runs on compatible flows.
orientation-03.png · 01:17.767–01:56.200	Packet, header and payload. Packet: a small unit sent across a network.
orientation-04.png · 01:56.200–02:35.233	A flow needs a grouping rule. Related packets must be grouped.
orientation-05.png · 02:35.233–03:15.933	Classifier, detector and response. Classifier: select a category.
orientation-06.png · 03:15.933–03:56.633	How to read an evidence claim. Implemented: code exists.
orientation-07.png · 03:56.633–04:33.000	Your presentation has an evidence trail. Watch this course to learn the story.
orientation-08.png · 04:33.000–04:43.000	Your turn · 10 seconds. Does a working classifier prove a complete live IDS? A declared practice interval; it does not add a new model result.

Chapter 2: The PDF: ideas, experiments and boundaries · 04:43–09:59

File and explanation links: [the paper reading map](#).

Scene and base frame	What this individual visual teaches
paper-01.png · 04:43.000–05:25.967	What the attached PDF is. arXiv 2601.21792v1 · January 2026.
paper-02.png · 05:25.967–06:12.667	Three problems motivate the paper. Computation cost of long sequences.

Scene and base frame	What this individual visual teaches
paper-03.png · 06:12.667–06:52.467	What multimodal means here. Bytes: content patterns.
paper-04.png · 06:52.467–07:33.600	Mamba and the alternative backbone. Mamba: input-dependent state updates.
paper-05.png · 07:33.600–08:15.667	Read the paper in this order. Framework and representation: sections IV–V.
paper-06.png · 08:15.667–08:58.600	A table is not our result. Paper Table IV: 97.50% for CICIoT2022.
paper-07.png · 08:58.600–09:49.000	Two details that prevent overclaiming. Paper byte sequence: 401 tokens.
paper-08.png · 09:49.000–09:59.000	Your turn · 10 seconds. Name one paper claim that has not been reproduced here. A declared practice interval; it does not add a new model result.

Chapter 3: The first CSV: CICIDS2017 packet records · 09:59–15:00

File and explanation links: [CSV fields and distributions](#).

Scene and base frame	What this individual visual teaches
cic-csv-01.png · 09:59.000–10:40.033	A CSV is a table, not a running database. Rows: individual records.
cic-csv-02.png · 10:40.033–11:21.700	One row contains five kinds of metadata. payload_byte_1 ... payload_byte_1500.
cic-csv-03.png · 11:21.700–12:06.233	Benign and attack labels are imbalanced. BENIGN: 362,108 rows.
cic-csv-04.png · 12:06.233–12:46.367	Why the small classes matter. PortScan: 830 rows.
cic-csv-05.png · 12:46.367–13:29.333	Odd values are questions to investigate. Some t_delta values are negative.
cic-csv-06.png · 13:29.333–14:09.933	What the profiling actually checked. Full-file row counts and metadata aggregates.
cic-csv-07.png · 14:09.933–14:50.000	The missing information changes the task. No established addresses, ports or flow ID.
cic-csv-08.png · 14:50.000–15:00.000	Your turn · 10 seconds. Why is matching a tensor shape insufficient to establish compatible flow data? A declared practice interval; it does not add a new model result.

Chapter 4: The second CSV and the choice of flow data · 15:00–20:06

File and explanation links: [CSV fields and distributions](#) and [input records](#).

Scene and base frame	What this individual visual teaches
unsw-csv-01.png · 15:00.000–15:39.733	The UNSW extract has the same broad schema. 79,881 rows.
unsw-csv-02.png · 15:39.733–16:21.167	Read the UNSW label distribution. normal: 21,000.
unsw-csv-03.png · 16:21.167–17:02.533	Rare labels and protocol variety. dos: 3,397 · backdoor: 1,239.

Scene and base frame	What this individual visual teaches
unsw-csv-04.png · 17:02.533–17:46.767	Related subject, different learning problems. CICIDS2017 CSV: 15 packet labels.
unsw-csv-05.png · 17:46.767–18:29.367	Why the actual experiment used other data. Train: 8,323 flows · Validation: 1,040 flows.
unsw-csv-06.png · 18:29.367–19:14.600	Compatible data still has limitations. 5 identical stored inputs across train / validation.
unsw-csv-07.png · 19:14.600–19:56.000	The defensible connection between the files. The paper defines a flow representation.
unsw-csv-08.png · 19:56.000–20:06.000	Your turn · 10 seconds. Which data was actually used for training, and why were the uploaded CSVs not used? A declared practice interval; it does not add a new model result.

Chapter 5: Follow one native flow into the model · 20:06–25:00

File and explanation links: [the native dataset](#) and [configuration files](#).

Scene and base frame	What this individual visual teaches
features-01.png · 20:06.000–20:43.767	The actual input is a list of flow objects. data: packet byte strings.
features-02.png · 20:43.767–21:24.867	Bytes: five packets, 320 stored bytes each. $5 \times 320 = 1,600$ stored byte positions.
features-03.png · 21:24.867–22:02.867	Normalization changes the numerical scale. byte / 255 → range 0 to 1.
features-04.png · 22:02.867–22:42.767	Sizes and intervals use their own rules. First 20 sizes: clipped to 0 ... 1,500.
features-05.png · 22:42.767–23:19.033	The 443-token calculation. 400 byte tokens.
features-06.png · 23:19.033–24:10.967	Four blocks produce six raw outputs. 4 blocks · width 256 · 1,870,080 parameters.
features-07.png · 24:10.967–24:50.000	Explain the input contract back. Meaning and origin must match.
features-08.png · 24:50.000–25:00.000	Your turn · 10 seconds. Describe the three feature views and the six-number output. A declared practice interval; it does not add a new model result.

Chapter 6: How pretraining, training and inference work · 25:00–30:00

File and explanation links: [training evidence](#) and [main code](#).

Scene and base frame	What this individual visual teaches
learning-01.png · 25:00.000–25:40.000	Three activities, three different outputs. Pretraining → general representation weights.
learning-02.png · 25:40.000–26:20.600	Pretraining hides part of the input. Mask byte content and selected feature values.
learning-03.png · 26:20.600–26:59.267	What our pretraining check actually did. 2 completed epochs · 132 updates.

Scene and base frame	What this individual visual teaches
learning-04.png · 26:59.267–27:41.333	A labeled batch changes the classifier. Forward pass → predictions.
learning-05.png · 27:41.333–28:24.467	Three splits prevent one obvious shortcut. Training split: update parameters.
learning-06.png · 28:24.467–29:08.100	The completed training protocol. Seeds 0, 1 and 2.
learning-07.png · 29:08.100–29:50.000	A successful reload has a narrow meaning. Strict loading checks all expected tensors.
learning-08.png · 29:50.000–30:00.000	Your turn · 10 seconds. What does each of pretraining, fine-tuning and inference produce? A declared practice interval; it does not add a new model result.

Chapter 7: The main code: follow a real execution path · 30:00–34:46

File and explanation links: [the individual entry points](#) and [supporting tools](#).

Scene and base frame	What this individual visual teaches
code-01.png · 30:00.000–30:43.000	Our code surrounds the authors' code. Acquire and verify the original source.
code-02.png · 30:43.000–31:22.667	repro.py is the experiment coordinator. fetch and validate.
code-03.png · 31:22.667–31:59.000	evaluate.py answers: how accurate is this model? Labeled native test flows.
code-04.png · 31:59.000–32:39.133	predict.py answers: which category fits this flow? Unlabeled native flow input.
code-05.png · 32:39.133–33:17.933	replay.py connects execution to explanation. Run labeled inference.
code-06.png · 33:17.933–34:02.167	Configurations and assets define the recipe. configs: settings, hashes and class mapping.
code-07.png · 34:02.167–34:36.000	Read a run in this order. Manifest: what was requested and what completed.
code-08.png · 34:36.000–34:46.000	Your turn · 10 seconds. Which scripts handle unlabeled prediction and labeled evaluation? Where is run provenance recorded? A declared practice interval; it does not add a new model result.

Chapter 8: Read the actual results, including the mistakes · 34:46–39:33

File and explanation links: [published results](#), [figures](#) and [later rechecks](#).

Scene and base frame	What this individual visual teaches
results-01.png · 34:46.000–35:26.067	Three runs, one fixed test set. Seed 0: 91.26% · 950 / 1,041 correct.
results-02.png · 35:26.067–36:04.367	Mean and spread answer different questions. Mean: 86.65%.
results-03.png · 36:04.367–36:44.067	Accuracy, precision, recall and F1. Accuracy: correct predictions / all predictions.
results-04.png · 36:44.067–37:23.633	Read a confusion matrix. Rows: known category.
results-05.png · 37:23.633–38:04.267	Rechecking a result is different from retraining. Native evaluation, strict reload and replay agree.

Scene and base frame	What this individual visual teaches
results-06.png · 38:04.267–38:45.333	The paper gap is still an open question. Paper: different runtime and training conditions.
results-07.png · 38:45.333–39:23.000	Choose wording the evidence can support. “Three measured training runs completed.”.
results-08.png · 39:23.000–39:33.000	Your turn · 10 seconds. Explain the mean and spread without claiming customer-network validation. A declared practice interval; it does not add a new model result.

Chapter 9: Demonstrate one prediction, then reveal an error · 39:33–44:01

File and explanation links: [browser demonstration files](#).

Scene and base frame	What this individual visual teaches
demo-01.png · 39:33.000–40:08.867	Open the recorded replay. Runs without a GPU or Python.
demo-02.png · 40:08.867–40:48.933	Choose the demo model before inspecting outcomes. Seed 0 is the declared demonstration model.
demo-03.png · 40:48.933–41:23.600	Inspect JSON row 635. Browser display row: 0636.
demo-04.png · 41:23.600–41:59.200	Reveal the known label. True category: Other.
demo-05.png · 41:59.200–42:34.533	New inference starts outside the browser. Supply already assembled native flows.
demo-06.png · 42:34.533–43:14.867	A simple next IDS prototype. Capture and group traffic.
demo-07.png · 43:14.867–43:51.000	Practice the demonstration handoff. Show the replay and one actual error.
demo-08.png · 43:51.000–44:01.000	Your turn · 10 seconds. Why was the 84.43% display score insufficient to trust this prediction? A declared practice interval; it does not add a new model result.

Chapter 10: Set it up, use it and test the right layer · 44:01–48:49

File and explanation links: [current platform/setup instructions](#), [dependencies](#) and [CI](#).

Scene and base frame	What this individual visual teaches
setup-01.png · 44:01.000–44:39.500	Choose one of three setup routes. Present: browser or document viewer.
setup-02.png · 44:39.500–45:17.567	Clone and run the portable checks. Clone the current repository.
setup-03.png · 45:17.567–45:57.300	What cross-platform CI establishes. Linux x86-64, Windows x86-64, macOS ARM64.
setup-04.png · 45:57.300–46:40.067	The model environment has native dependencies. Python 3.12 · Torch 2.9.1 + CUDA 13.0.
setup-05.png · 46:40.067–47:21.933	Pass two gates before a new experiment. Numerical check: forward and backward comparisons.

Scene and base frame	What this individual visual teaches
setup-06.png · 47:21.933–48:03.733	Know what is still platform-specific. GB10: measured training and inference.
setup-07.png · 48:03.733–48:39.000	Match the test to the claim. Document checks → package integrity.
setup-08.png · 48:39.000–48:49.000	Your turn · 10 seconds. Which setup is enough to present, and which setup is needed for new model predictions? A declared practice interval; it does not add a new model result.

Chapter 11: Every repository area and every download · 48:49–53:53

File and explanation links: [all file entries](#), [expected v3 downloads](#) and [historical release attachments](#).

Scene and base frame	What this individual visual teaches
repository-01.png · 48:49.000–49:30.167	Use the file guide as the complete index. One entry for every tracked file.
repository-02.png · 49:30.167–50:11.900	Root files, configuration and dependencies. README and four entry points: start and execute.
repository-03.png · 50:11.900–50:55.500	Tools: acquire, build, measure and explain. <code>fetch_assets</code> / <code>profile_csvs</code> / <code>capture_runtime</code> .
repository-04.png · 50:55.500–51:37.233	Tools: documents, replay and this video. Package and guide builders produce teaching artifacts.
repository-05.png · 51:37.233–52:24.633	Tests, workflows, documents and evidence. tests: controlled regression cases.
repository-06.png · 52:24.633–53:08.967	Choose the right download. Current course: one MP4, transcript, captions and companion slides.
repository-07.png · 53:08.967–53:43.000	A practical reading order. README → course → file guide.
repository-08.png · 53:43.000–53:53.000	Your turn · 10 seconds. Where would you verify a platform claim and an actual training result? A declared practice interval; it does not add a new model result.

Chapter 12: Hardware roadmap, missing work and your handoff · 53:53–60:00

File and explanation links: [the hardware roadmap](#) and [current platform evidence](#).

Scene and base frame	What this individual visual teaches
hardware-01.png · 53:53.000–54:31.800	Keep the measured timing boundary visible. Batch 1 median: 0.881265 ms.
hardware-02.png · 54:31.800–55:08.667	A possible SmartNIC and NPU division. NIC / SmartNIC: packet handling and flow assembly.
hardware-03.png · 55:08.667–55:52.567	Weights are not a portable computation graph. Model-only export: saved parameter tensors.
hardware-04.png · 55:52.567–56:33.200	A defensible hardware validation process. Check full-model output and gradient parity where relevant.

Scene and base frame	What this individual visual teaches
hardware-05.png · 56:33.200–57:11.933	What remains missing. Matching paper accuracy and full pretraining.
hardware-07.png · 57:11.933–57:51.033	Teach the project to someone else. What data went in, and why?
hardware-08.png · 57:51.033–59:21.033	Your turn · 90 seconds. Explain the data choice, tested result, repository and remaining deployment work. A declared practice interval; it does not add a new model result.
hardware-06.png · 59:21.033–60:00.000	Your concise explanation. We built a checked execution and evidence layer.

Current accuracy cleanup and presentation checks

The cleanup reviews current instructions without changing frozen training records or media. The original package, retired HTML course and v3/v4 video editions keep their historical scopes. These additions document fresh checks and catch future packet-presentation drift.

File	Purpose and limits
tools/verify_packet_briefing.py	Portable comparison of completed packet evidence with generated source, actual PowerPoint slide order/content, attached notes and matching script. Checks reviewed PDF/PPT/source/script file identities. Does not render documents, retrain or certify visual/video quality.
tests/test_packet_briefing.py	Ten regression cases using real copied documents: rejects altered measurements, wrong scores, changed notes/scripts, reordered slides, misattached notes, modified PDF bytes and an incomplete receipt, even after rehashing where applicable.
docs/research/accuracy-cleanup.md	Current cleanup findings, applied corrections, fresh checks, archived ZIP inspection and remaining limitations. Separates council advice from independently executed verification.
docs/research/accuracy-cleanup-evidence.json	Machine-readable cleanup evidence: raw-source and prepared-input integrity, independent packet metric recomputation, actual downloaded ZIP inspection, document identities and execution receipts. These are checks of preserved experiments, not new training.
docs/research/accuracy-cleanup-council.json	Exact result of council run 7b5313e7-d426-4af7-a5ca-35db5cb14288: ESCALATED / UNRATIFIED, no decision hash. Preserves both seats' positions and unresolved evidence requests; later direct checks do not change this outcome.

Files you may see locally that are not tracked in this repo

The inventory above gives individual entries for current repository files. The structural validator checks those entries; it does not certify the prose or count planned outputs as completed evidence. Setup and execution also create local files outside the published inventory:

- **upstream/NetMambaPlus/**: the separately fetched authors' Git checkout. The original model, training engine, argument parsers and tensor loader live there. The repository verifies 103 tracked upstream files; it does not mirror them. See the pinned source links in the comparison and lesson for those internals.

- **assets/data/ciciot2022/**: data-train.json, data-valid.json, data-test.json and metadata.json. These are the actual native flows and class mapping obtained through asset acquisition. Their counts are 8,323 / 1,040 / 1,041 flows.
- **assets/checkpoints/fuse3_mamba.pth**: the authors' pretrained reconstruction model used as the starting point for downstream fine-tuning. It is not already the trained six-class IDS classifier.
- **assets/acquisition.json**: locally generated receipt recording acquisition statuses, expected hashes and inventory identity.
- **runs/<new-name>/**: a new experiment's local manifest, logs, metrics and possibly checkpoints/predictions/HTML. A training directory can contain checkpoint-best.pth; the runbook requires fresh output directories to avoid mixing runs.
- **Local model exports**: seed0-classifier.pth, seed1-classifier.pth, seed2-classifier.pth and their provenance JSON. These are generated by experiment review. Only their provenance and derived prediction evidence are published in the Git tree.
- **Packet preparation directory**: manifest.json plus each source's payload.npy, hashes.npy, counts.npy, subtypes.npy, split.npy, selected.npy and metadata.csv. Contains the validated original bytes and all row labels, with deterministic group selections. These private arrays stay local; public preparation records retain their hashes, counts and membership summaries.
- **Packet training/checkpoint directories**: each arm's logs and selected checkpoint-step-*.pth files, plus frozen checkpoint identities and results. These are binary packet classifiers, separate from the original six-class flow exports. Published evidence retains model identities and derived predictions; the neural weights remain local.
- **Packet inference output directory**: predictions.jsonl plus its input/model/output-bound receipt. Check whether it says complete, capped or incomplete before using the rows. No supplied label means no accuracy measurement. Diagnostic control fit directories instead contain small coefficient/scaler JSON models.
- **runs/video-course/**: local speech caches, source snapshots, rendered frames, codec logs, transcription diagnostics and disposable failure-test media. V2 Kokoro, v3 neural-service and v4 neural-service caches are separate. A copied v3 scene is reusable only when its exact text, voice, conversion and retained file identities match. The latter retain service MP3s, converted WAVs, word-event JSON and per-scene identities; cache files are not model training inputs. Current media belong under docs/customer/demo/video/v4/; v3 stays in v3/, and the one active page stays above both.
- **Virtual environments and build directories**: installed Python packages, compiler outputs and disposable copies of native dependency source. They are environment-specific and excluded from Git.
- **.git/ and __pycache__/**: Git's internal history/index and Python's generated bytecode caches. Neither is application source you need to edit.
- **Original uploaded PDF/CSVs**: inputs supplied outside the Git tree. The original evidence retains historical aggregate CSV profiles. The new packet preparation separately scans both actual CSVs for native supervised training; neither the PDF nor those rows establish flow ordering. The uploads themselves remain local.

The runbook can use a different asset/cache directory if passed explicitly. Historical paths inside manifests describe the original execution machine; use your own valid paths when repeating the commands.

Release attachments are separate from tracked files

The [audited GitHub release](#) has downloadable artifacts outside the Git tree:

Download	What it contains; how to use it
netmambaplus-customer-package.zip	The complete audited customer package and source snapshot at its recorded commit. Unzip it, then open the guide/replay or follow the quickstart. It includes the documents, recorded replay backup, code and evidence, with an added SOURCE-COMMIT.txt identifying the snapshot. Raw research assets and trained weights are separate. Later videos, calibration, portability updates, the packet study and this guide belong to newer GitHub source and separate releases, outside this older frozen ZIP.
NetMambaPlus-customer-briefing.pdf	Eight-page explanatory briefing. Open it in a PDF reader to read or print the customer story without downloading the entire repository. Same reviewed PDF as the tracked customer document.
NetMambaPlus-customer-slides.pdf	Fixed-layout PDF of all 16 slides. Present from this when you want the reviewed layout without depending on a PowerPoint installation.
NetMambaPlus-customer-slides.pptx	Editable 16-slide PowerPoint with the presenter script in its notes. Open with compatible presentation software; the slide PDF is the checked fixed-layout alternative.
SHA256SUMS-release.txt	Plain text containing hashes of the ZIP and three document downloads. Compare against your downloaded bytes to detect incomplete or changed files. Different from the broader docs/customer/SHA256SUMS inside the package.
release-inventory.json	Machine-readable package commit and the four principal downloads' names, byte sizes and hashes. Useful when matching a file to an exact release. It does not inventory every repository file.
release-verification.json	Final commit-specific CI/Pages, anonymous-download, extracted-ZIP test and hosted-guide checks. It records what was checked after publication. Its scope is public delivery, not independent GPU certification.

GitHub also automatically offers **Source code (zip)** and **Source code (tar.gz)** on a release. These are two archive formats for the same tagged Git source tree. ZIP is widely convenient on desktop computers; .tar.gz is common on Linux. They include the tracked documents/evidence at that tag, but do not contain separately attached release receipts or the customer ZIP's added SOURCE-COMMIT.txt. They do not acquire the external training assets. The **Code → Download ZIP** action on the repository's main page instead downloads the current branch, including this newer walkthrough.

The [older `customer-2026-09-15` release](#) remains available as history. It has six named attachments: the customer ZIP, briefing PDF, slide PDF, PowerPoint, release checksums and release-verification JSON. Those names have the same roles explained above, but contain the earlier six-page/13-slide package and its own receipt. That release has no separate release-inventory.json. Use the audited release for the expanded eight-page/16-slide customer package.

The web guide, replay, recording, images and individual source/evidence files can also be opened or downloaded directly from GitHub. Their purposes are all covered in their individual inventory entries

above. The hosted guide and replay are ordinary browser pages; they do not require installing the GPU dependencies.

During setup, there are three further groups of downloads. `repro.py` fetch gets the pinned authors' Git source; `fetch_assets.py` obtains the three native data splits and pretrained checkpoint while generating the exact small metadata file; the runbook's package-install and build commands acquire Python libraries, browser/runtime tools and the pinned causal-convolution source. The `configs/`, `requirements/`, `tools/` and `local-files` sections explain each group and the checks applied. None of these is silently bundled into the customer ZIP.

Keeping a release attached to its recorded commit lets you distinguish "the package that was checked" from later documentation additions. Always match a receipt to the release/commit it names.

Which file should you open for a particular question?

Your next question	Start here
"What should I say to the customer?"	<code>docs/customer/talk-track.md</code> , then the slide-aligned <code>demo/guide.html</code> .
"How do I get it running?"	<code>docs/customer/quickstart.md</code> and <code>docs/customer/packet-study-setup.md</code> for the single packet workflow; <code>docs/support-matrix.md</code> distinguishes current setup from historical compatibility records. The ZIP contains its own historical instructions.
"What was actually measured?"	<code>docs/customer/results.md</code> , then <code>evidence/results.json</code> and the relevant seed predictions.
"How do I know the model was really trained?"	A seed's training manifest, 120-row epoch log, selected-checkpoint identity, review results and separate evaluation record. The weights themselves remain local.
"What does the model read?"	<code>docs/harness-reference.md</code> , <code>configs/ciciot2022.json</code> and the original loader linked by <code>docs/lesson.md</code> .
"How do both uploaded CSVs train the new packet model?"	<code>docs/customer/packet-model-study.md</code> , <code>packet_data.py</code> , <code>packet_model.py</code> and <code>configs/packet-study.json</code> . The six-class flow route above remains separate.
"Which packet results and controls actually completed?"	<code>docs/customer/evidence/packet-study/results.json</code> , the individual training receipts and <code>controls/evaluation/results.json</code> . Run <code>tools/review_packet_study.py</code> and <code>tools/review_packet_controls.py</code> to check saved records; inspect both row/group weights and source exposure.
"How do I classify a new packet CSV without labels?"	<code>docs/customer/packet-study-setup.md</code> and <code>tools/predict_packets.py</code> , using a local selected packet checkpoint. Read the complete/capped/incomplete receipt before using predictions.
"Does the hour-long video cover the new packet experiment?"	It covers the preserved flow/calibration edition. Use the separate packet report and eight-slide addendum for new packet measurements, and this current guide for every file.
"Where are the model weights and neural layers?"	The separately acquired local checkpoints and pinned upstream checkout, not the presentation or prediction JSON.
"Why isn't this the paper's exact score?"	<code>docs/customer/upstream-comparison.md</code> and <code>docs/research/research-record.md</code> . The cause of the gap has not been isolated.
"Can I put it on a SmartNIC now?"	<code>docs/customer/hardware-roadmap.md</code> and the export-probe result. The deployment is still future work.
"Was every requested item addressed?"	<code>docs/customer/acceptance.md</code> maps the current requirements and seven questions. <code>answers.md</code> follows only the historical 16-slide deck.

A short self-check

Before presenting, try explaining these three distinctions aloud:

1. **Settings versus evidence:** the config requests a training run; the manifest, epoch logs, checkpoint and evaluations show what actually happened.
2. **Prediction versus replay:** predict.py executes a saved six-class classifier on compatible flows; tools/predict_packets.py executes a separate binary packet checkpoint on unlabeled byte rows. The HTML replay displays outputs already measured.
3. **A successful check versus a stronger claim:** passing CPU tests verifies selected software behavior; it does not establish the paper's 97.50%, independent customer-traffic performance, live intrusion prevention or NPU/SmartNIC deployment.

The original flow result remains three completed source-based fine-tuning runs with **86.65% mean test accuracy**. The packet study has a separate input contract, frozen protocol and six completed native comparison records, with markedly different familiar-source and transfer outcomes. This file map keeps each claim attached to its own records and limits.

Client edition maintenance files

File	Purpose
<code>.github/workflows/client-sync.yml</code>	Publishes checked client snapshots after verified source main; isolates its write key and rejects stale or diverged copies.
<code>client/NOTICE.md.in</code>	Source template preserving paper/upstream attribution and inherited-asset boundaries.
<code>client/README.md.in</code>	Source template for the concise client landing page, experiment boundaries and entry commands.
<code>client/RESULTS.md.in</code>	Source template for the measured flow and packet results and their scientific limitations.
<code>client/SETUP.md.in</code>	Source template for client GPU setup, both training routes, unlabeled prediction and offline verification.
<code>client/ci.yml.in</code>	Read-only portable verification workflow exported to the client repository.
<code>client/gitattributes.in</code>	Prevents Git checkout newline conversion for all delivered client files.
<code>configs/client-manifest.json</code>	Explicitly selects unchanged runtime, test and evidence files plus the compact client templates.
<code>docs/research/client-edition.md</code>	Explains the current packet-only client selection, retained internal dependencies, publication, maintenance, update direction and exact verification scopes.
<code>docs/superpowers/plans/2026-09-14-client-edition.md</code>	Implementation and acceptance plan for the public client edition and automatic updates.
<code>tests/test_client_export.py</code>	Tests real Git export boundaries, deterministic output, path safety, drift, rollback and managed deletion.
<code>tests/test_client_publication.py</code>	Tests the archive publisher against a real local bare remote, including competing pushes and tampering.
<code>tests/test_client_sync_gate.py</code>	Tests canonical successful-main eligibility and rejection of stale, failed, PR and wrong-repository events.
<code>tests/test_client_verification.py</code>	Tests delivered-file inventory checks plus rejection of reindexed false packet metrics and incomplete training, missing/modified files, unsafe paths and source identity.
<code>tools/check_client_export.py</code>	Rejects exported flow entry points/evidence and runs packet tests, CLI help, scientific verification and recorded packet-demo generation in a fresh temporary directory.
<code>tools/client_sync_gate.py</code>	Allows publication only for successful canonical main push verification at the current source revision.
<code>tools/export_client.py</code>	Builds deterministic client snapshots from explicitly selected immutable Git blobs.
<code>tools/publish_client.py</code>	Packages checked managed files and publishes a hash-verified archive with a normal fast-forward Git push.
<code>tools/sync_client.py</code>	Checks the previous source snapshot and stages updates while rejecting client drift and source rollback.
<code>tools/verify_client.py</code>	Verifies client file identities and all packet study/control/inference evidence without presentation assets or Git history.
<code>docs/research/client-edition-council.json</code>	Preserves the non-mutual client architecture council outcome, exact blockers, positions and model identities.
<code>docs/research/client-edition-validation.json</code>	Binds actual client CPU/platform and GB10 verification to its source/client revisions, comparisons, retained failures and raw receipts.

File	Purpose
docs/research/client-checks/packet-native-initial.log	Retains the initial test invocation failure caused by selecting the wrong local payload-cache directory.
docs/research/client-checks/packet-native.log	Records all 17 packet-model tests passing from the client checkout with native CUDA cases enabled.
docs/research/client-checks/native-ops.json	Records seven fresh native forward/backward operator comparisons from the client checkout.
docs/research/client-checks/flow-model.json	Records the client flow model optimizer step, gradients, strict reload and inference on synthetic fixtures.
docs/research/client-checks/flow-predictions.json	Fresh client flow predictions for all 1,041 retained test inputs, without an invented unlabeled accuracy field.
docs/research/client-checks/flow-inference-manifest.json	Binds the fresh client flow prediction command to its input, code and checkpoint identities.
docs/research/client-checks/packet-predictions.jsonl	Fresh client binary predictions for the first 128 CIC held-out payload groups.
docs/research/client-checks/packet-inference-receipt.json	Records full validation of 20,000 input rows and explicitly capped 128-row native prediction.

Single CSV workflow additions

The active [client guide](#) explains the packet delivery. All historical science/media bindings keep their earlier meaning.

File	Purpose
configs/packet-assets.json	Pins the sole default acquisition for the supported packet workflow: the authors' pretrained initialization, with size/hash/location and asset terms.
docs/research/client-edition-initial.md	Historical two-route client-delivery report from source 19f849c, including the original 218-file edition and first live automatic update. Its counts are not the new packet client's counts.
docs/research/packet-only-council.json	Preserves the single-workflow council's ABORTED state, Claude HTTP 529 failure and non-mutual Astra position. No consensus or provider retry is claimed.
docs/research/packet-only-handoff.md	Applied-change and execution audit for the single CSV workflow, including native packet checks, demo/doc review and source/client publication scope.
docs/research/packet-checks/native-receipt.json	Actual prepared-directory GB10 gate receipt: all 17 native packet cases executed without skips, binding data, source, initialization, test program and produced reports.
docs/research/packet-checks/native-tests.log	Raw successful 17-case packet test log from the new native gate; covers portable contracts plus actual CUDA transfer, gradients and checkpoint behavior.
docs/research/packet-checks/native-model-probe.json	Actual native gradient/update/checkpoint probe using two stored payloads from each source with synthetic targets. Functional evidence, not new benchmark accuracy.
docs/research/packet-checks/acquisition.json	Actual default asset-fetch receipt showing that only the pinned pretrained initialization was downloaded and hash-verified.
docs/research/packet-checks/browser.json	Chromium offline packet-demo checks for source metrics, filtering, details, pagination, empty search, mobile width, absent external requests and JavaScript errors.
docs/research/packet-checks/client-export.json	Actual packet-only export and publication evidence: portable test counts, 17-case native receipt/log/probe, all 128 fresh predictions and their 76 strict score-comparison failures, successful platform/publication runs, and anonymous download checks.

File	Purpose
docs/superpowers/plans/2026-09-14-packet-only-handoff.md	Implementation and acceptance checklist for making the CSV packet model the sole client workflow while preserving historical research and automatic updates.
tests/test_packet_demo.py	Checks contradictory-label counts, group versus row weighting, ties, invalid/duplicate records, safe JSON embedding and output preservation for the packet viewer.
tests/test_packet_runtime.py	Checks that skipped, empty or failed native suites cannot pass the packet runtime gate and existing output evidence is preserved.
tools/render_packet_demo.py	Verifies retained packet evidence and renders the full joint-model CIC/UNSW groups into an offline HTML viewer with filters, original label counts, declared metrics and a source-bound receipt.
tools/check_packet_runtime.py	Verifies the prepared directory, runs the 17-case packet suite with native execution enabled on bytes from both sources, and records achieved tests/probe/log identities; synthetic targets make this a functional gate.

Current v5: both CSVs and both repositories

Watch the [single current video](#) · [Versioned release](#). The release provides the MP4, browser WebM, captions, transcript, chapter map, PowerPoint, slide PDF, handbook and media/document manifests. These are copies of the versioned files explained below. Raw CSVs and model weights remain external.

File	Purpose and how to use it
docs/customer/demo/video/v5/NetMambaPlus-CSV-workflow-course.mp4	Single continuous current H.264/AAC video with neural narration, captions, diagrams and recorded packet-demo interaction.
docs/customer/demo/video/v5/NetMambaPlus-CSV-workflow-course.webm	VP9/Opus encoding of the same lesson for browsers lacking H.264/AAC support.
docs/customer/demo/video/v5/NetMambaPlus-course-handbook.pdf	Searchable lesson script, on-screen examples and full individual-file guide appendix.
docs/customer/demo/video/v5/NetMambaPlus-course-slides.pdf	PDF rendering of the same scene images, suitable for offline presentation.
docs/customer/demo/video/v5/NetMambaPlus-course-slides.pptx	One slide per video scene, with the identical diagram image and editable narration in speaker notes.
docs/customer/demo/video/v5/captions.srt	SubRip subtitles mapped from speech-service word boundaries through final audio timing.
docs/customer/demo/video/v5/captions.vtt	WebVTT copy of the video subtitles for the browser player.
docs/customer/demo/video/v5/chapters.json	Measured chapter start/end positions used by the player and companion documents.
docs/customer/demo/video/v5/companion-manifest.json	Companion document hashes, source identity, slide count and embedded file-guide identity.
docs/customer/demo/video/v5/frames/cic-csv-01.png	Video and slide scene: A text table of packet examples. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/cic-csv-02.png	Video and slide scene: Read one row in three parts. Narration and primary references appear in the transcript.

File	Purpose and how to use it
docs/customer/demo/video/v5/frames/cic-csv-03.png	Video and slide scene: Byte slots contain numbers, including stored zeros. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/cic-csv-04.png	Video and slide scene: Four recorded properties, with specific limits. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/cic-csv-05.png	Video and slide scene: Keep the answer outside the model input. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/cic-csv-06.png	Video and slide scene: Many rows repeat the same stored payload. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/cic-csv-07.png	Video and slide scene: Check what the selected test set contains. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/cic-csv-pause.png	Video and slide scene: Explain it back. Labeled practice prompt.
docs/customer/demo/video/v5/frames/code-01.png	Video and slide scene: Find the five implementation responsibilities. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/code-02.png	Video and slide scene: The forward call receives only payload tensors. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/code-03.png	Video and slide scene: Loss, gradients, and an optimizer update. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/code-04.png	Video and slide scene: Load the selected checkpoint for inference. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/code-05.png	Video and slide scene: Validate the CSV before publishing completion. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/code-06.png	Video and slide scene: Read class decisions and uncalibrated scores. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/code-07.png	Video and slide scene: Check native execution on the prepared packets. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/code-08.png	Video and slide scene: Your turn · 10 seconds. Labeled practice prompt.
docs/customer/demo/video/v5/frames/demo-01.png	Video and slide scene: Start with the right demonstration. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/demo-02.png	Video and slide scene: Choose the source you are explaining. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/demo-03.png	Video and slide scene: One decision has several parts. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/demo-04.png	Video and slide scene: Keep the supplied labels visible. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/demo-05.png	Video and slide scene: Demonstrate an error without hiding it. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/demo-06.png	Video and slide scene: Fresh prediction is a separate execution step. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/demo-07.png	Video and slide scene: What a working IDS would add. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/demo-practice.png	Video and slide scene: Your turn: explain it aloud. Labeled practice prompt.

File	Purpose and how to use it
docs/customer/demo/video/v5/features/features-01.png	Video and slide scene: One packet supplies the model input. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/features/features-02.png	Video and slide scene: Rescale the bytes without changing their order. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/features/features-03.png	Video and slide scene: Assemble 378 tokens from one packet. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/features/features-04.png	Video and slide scene: Empty streams still have learned prefixes. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/features/features-05.png	Video and slide scene: Four blocks lead to two class scores. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/features/features-06.png	Video and slide scene: Transfer compatible weights into the packet model. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/features/features-07.png	Video and slide scene: Explain the complete feature path. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/features/features-08.png	Video and slide scene: Your turn · 10 seconds. Labeled practice prompt.
docs/customer/demo/video/v5/hardware/hardware-01.png	Video and slide scene: Start with the complete processing path. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/hardware/hardware-02.png	Video and slide scene: What a SmartNIC could contribute. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/hardware/hardware-03.png	Video and slide scene: What an NPU port would require. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/hardware/hardware-04.png	Video and slide scene: Optimization needs a new measurement plan. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/hardware/hardware-05.png	Video and slide scene: The major research gaps remain visible. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/hardware/hardware-06.png	Video and slide scene: Say precisely what has been built. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/hardware/hardware-07.png	Video and slide scene: Teach the complete story to the next person. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/hardware/hardware-practice.png	Video and slide scene: Your turn: explain it aloud. Labeled practice prompt.
docs/customer/demo/video/v5/learning/learning-01.png	Video and slide scene: Preserve disagreements in the learning target. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/learning/learning-02.png	Video and slide scene: Compare six training arms. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/learning/learning-03.png	Video and slide scene: One update is one learning step. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/learning/learning-04.png	Video and slide scene: Caps and replacement sampling define exposure. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/learning/learning-05.png	Video and slide scene: Validation chooses; frozen test groups measure. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/learning/learning-06.png	Video and slide scene: Freeze the experiment and retain its receipts. Narration and primary references appear in the transcript.

File	Purpose and how to use it
docs/customer/demo/video/v5/frames/learning-07.png	Video and slide scene: Distinguish training, selection, and inference. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/learning-08.png	Video and slide scene: Your turn · 10 seconds. Labeled practice prompt.
docs/customer/demo/video/v5/frames/orientation-01.png	Video and slide scene: Follow one packet to a decision. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/orientation-02.png	Video and slide scene: The paper, the implementation, and the evidence. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/orientation-03.png	Video and slide scene: One supported route from both CSVs. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/orientation-04.png	Video and slide scene: What joint pretrained means. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/orientation-05.png	Video and slide scene: Learning a rule and using a saved rule. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/orientation-06.png	Video and slide scene: Two repositories, one packet workflow. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/orientation-07.png	Video and slide scene: Explain the delivered capability precisely. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/orientation-pause.png	Video and slide scene: Explain it back. Labeled practice prompt.
docs/customer/demo/video/v5/frames/paper-01.png	Video and slide scene: Three views of traffic in the paper. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/paper-02.png	Video and slide scene: Mamba carries a numerical state forward. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/paper-03.png	Video and slide scene: A packet row cannot establish a whole flow. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/paper-04.png	Video and slide scene: Pretraining learns to reconstruct hidden content. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/paper-05.png	Video and slide scene: Supervised learning teaches the packet categories. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/paper-06.png	Video and slide scene: What the packet adaptation actually observes. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/paper-07.png	Video and slide scene: A similar percentage is not the same experiment. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/paper-pause.png	Video and slide scene: Explain it back. Labeled practice prompt.
docs/customer/demo/video/v5/frames/repository-01.png	Video and slide scene: Two repos, one packet application. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/repository-02.png	Video and slide scene: The model and data files have distinct jobs. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/repository-03.png	Video and slide scene: Tools, configs and requirements support execution. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/repository-04.png	Video and slide scene: Evidence and local runs are different objects. Narration and primary references appear in the transcript.

File	Purpose and how to use it
docs/customer/demo/video/v5/frames/repository-05.png	Video and slide scene: The client manifest identifies the delivery. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/repository-06.png	Video and slide scene: Updates flow from source to client. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/repository-07.png	Video and slide scene: Find the right download and its scope. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/repository-practice.png	Video and slide scene: Your turn: explain it aloud. Labeled practice prompt.
docs/customer/demo/video/v5/frames/results-01.png	Video and slide scene: The selected joint model's measured results. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/results-02.png	Video and slide scene: Class balance and duplicate weighting answer different things. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/results-03.png	Video and slide scene: Pretrained models show transfer tradeoffs. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/results-04.png	Video and slide scene: All scratch arms trained; their outcomes were weak. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/results-05.png	Video and slide scene: Nine simple controls challenge neural superiority. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/results-06.png	Video and slide scene: Class agreement and numerical agreement are distinct. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/results-07.png	Video and slide scene: State what the evidence still cannot establish. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/results-08.png	Video and slide scene: Your turn · 10 seconds. Labeled practice prompt.
docs/customer/demo/video/v5/frames/setup-01.png	Video and slide scene: Start from the client repository. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/setup-02.png	Video and slide scene: Check the portable parts first. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/setup-03.png	Video and slide scene: Prepare the native execution environment. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/setup-04.png	Video and slide scene: Acquire only the required initialization. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/setup-05.png	Video and slide scene: Prepare both inputs and run the native gate. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/setup-06.png	Video and slide scene: Freeze, train, select and predict. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/setup-07.png	Video and slide scene: Read a failed check before retrying. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/setup-practice.png	Video and slide scene: Your turn: explain it aloud. Labeled practice prompt.
docs/customer/demo/video/v5/frames/unsw-csv-01.png	Video and slide scene: UNSW uses the same row structure. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/unsw-csv-02.png	Video and slide scene: Map source labels to the shared binary task. Narration and primary references appear in the transcript.

File	Purpose and how to use it
docs/customer/demo/video/v5/frames/unsw-csv-03.png	Video and slide scene: Combine packet examples without inventing connections. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/unsw-csv-04.png	Video and slide scene: Find exact payload duplicates across both sources. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/unsw-csv-05.png	Video and slide scene: Shared payloads can carry conflicting answers. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/unsw-csv-06.png	Video and slide scene: Preserve disagreement as label proportions. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/unsw-csv-07.png	Video and slide scene: Keep exact duplicates inside one partition. Narration and primary references appear in the transcript.
docs/customer/demo/video/v5/frames/unsw-csv-pause.png	Video and slide scene: Explain it back. Labeled practice prompt.
docs/customer/demo/video/v5/media-manifest.json	Rendered source, reference, scene and media hashes plus timing; integrity evidence, not a claim of human review.
docs/customer/demo/video/v5/motion-manifest.json	Time-bound schematic movement, emphasis and source-identified recorded clip insertion.
docs/customer/demo/video/v5/narration-manifest.json	Source-bound per-scene voice identity, word cues and audio hashes; cloud model revision is unavailable.
docs/customer/demo/video/v5/poster.png	Opening frame displayed before the current video starts.
docs/customer/demo/video/v5/recordings/capture.json	Capture provenance, browser actions and hashes tied to the actual compressed packet predictions.
docs/customer/demo/video/v5/recordings/packet-demo.png	Actual offline packet-viewer screenshot showing source selection, metrics and saved predictions.
docs/customer/demo/video/v5/recordings/packet-demo.webm	Short real browser interaction changing datasets and opening the recorded error; no fresh inference occurs.
docs/customer/demo/video/v5/recordings/packet-error.png	Actual detail view of a saved attack prediction whose supplied group label is benign.
docs/customer/demo/video/v5/transcript.md	Complete current narration and practice text in playback order, with on-screen teaching and linked evidence.
docs/customer/video-course-v5-source.json	Reviewed Codex lesson source: 12 chapters, narration, diagram data, both repositories, seven customer questions, primary JSON fact bindings and source references.
docs/customer/video-verification-v5.md	Current video reproduction commands, actual executed audit results and limits, including the distinction from a complete human watch.
docs/research/video-v5-browser.json	Actual current-video browser playback, seek and responsive-layout observations.
docs/research/video-v5-council.json	Non-mutual council record and exact blockers, with the direct evidence-based resolution; no consensus is claimed.
docs/research/video-v5-decode.json	Full encoded MP4 inspection: duration, stream structure, loudness, silences and decode errors.
docs/research/video-v5-publication.json	Dated source/client CI, release and deployed-page/download observations; publication checks are distinct from scientific model results.
docs/research/video-v5-review.json	Editorial, visual, audio/ASR, source-binding and scope review record for the rendered current lesson.

File	Purpose and how to use it
docs/research/video-v5-speech.json	Independent local ASR of all spoken passages decoded from the completed MP4, with recognized words, timings and explicit machine-review limits.
docs/research/video-v5-webm-decode.json	Full VP9/Opus fallback stream probe and decode result for the current browser video.
docs/superpowers/plans/2026-09-14-packet-video-v5.md	Scope and acceptance plan for the current CSV lesson, animation, companion documents, audits and publication.
tests/test_packet_video_motion.py	Regression checks for packet diagram geometry, data-supplied dimensions, exact cue anchors, intact groups and motion containment.
tests/test_packet_video_publication.py	Rejects missing packet summaries and verifies current publication metadata without changing archived flow edition behavior.
tests/test_packet_video_source.py	Rejects changed packet facts/contracts and unknown workflows; verifies independent packet lesson loading and dynamic publication summary.
tests/test_packet_video_verification.py	Negative tests for the current video verifier, including changed evidence, missing topics and stale source/artifact bindings.
tools/packet_video_motion.py	Five scene-specific packet diagrams and timed ASS animations: separate payload/metadata paths, encoder, training, duplicate grouping and repository export.
tools/verify_video_course_v5.py	Current packet-lesson checks, primary evidence bindings, topic coverage and source/companion artifact identity verification; --source-only supports pre-render review.
docs/research/video-v5/decode.json	Bound acceptance of both encoded-media inspections, linked to the exact completed source and media hashes.
docs/research/video-v5/browser.json	Bound acceptance of actual desktop/mobile playback, seeking and caption checks; uses the separate detailed browser observation.
docs/research/video-v5/captions.json	Source and encoded-speech caption review, with machine-recognition disagreements distinguished from narration defects.
docs/research/video-v5/claims.json	Primary-fact and required-question review of the final rendered lesson and both repositories.
docs/research/video-v5/rendered_samples.json	Visual review of every scene and meaningful animation samples; identifies reviewed frames without claiming human full-watch.
docs/research/video-v5/documents.json	PowerPoint notes/image and PDF/handbook checks tied to the completed companion artifacts.
tools/audit_video_speech.py	Independently transcribes every spoken scene from an identity-checked encoded video using a locally cached speech-recognition model. Records recognized text/times and source hashes; this is diagnostic, not narration authorship or human acceptance.
tests/test_video_speech_audit.py	Rejects mismatched media/manifest hashes, wrong scene counts and invalid time spans before expensive speech recognition. Does not substitute a mock result for actual audio verification.
.github/workflows/publish-video-v5.yml	Manually publishes the reviewed v5 video and companion files from main after source/media, page and guide checks. It creates the versioned release once and does not overwrite earlier releases.
docs/customer/video-release-v5.md	Concise public release notes with absolute watch, repository and audit links, used by the v5 publication workflow.